

# Legyen Ön is Milliomos

1.0

Készítette Doxygen 1.10.0



|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| <b>1. Névtérmutató</b>                                              | <b>1</b>  |
| 1.1. Package List                                                   | 1         |
| <b>2. Hierarchikus mutató</b>                                       | <b>3</b>  |
| 2.1. Osztályhierarchia                                              | 3         |
| <b>3. Osztálymutató</b>                                             | <b>5</b>  |
| 3.1. Osztálylista                                                   | 5         |
| <b>4. Fájlmutató</b>                                                | <b>7</b>  |
| 4.1. Fájllista                                                      | 7         |
| <b>5. Névterek dokumentációja</b>                                   | <b>9</b>  |
| 5.1. millionaire csomag                                             | 9         |
| 5.2. millionaire.controller csomag                                  | 9         |
| 5.3. millionaire.model csomag                                       | 10        |
| 5.4. millionaire.view csomag                                        | 10        |
| <b>6. Osztályok dokumentációja</b>                                  | <b>11</b> |
| 6.1. millionaire.controller.AddQuestionController osztályreferencia | 11        |
| 6.1.1. Részletes leírás                                             | 12        |
| 6.1.2. Konstruktork és destruktorok dokumentációja                  | 12        |
| 6.1.2.1. AddQuestionController()                                    | 12        |
| 6.1.3. Tagfüggvények dokumentációja                                 | 12        |
| 6.1.3.1. getCategories()                                            | 12        |
| 6.1.3.2. getDifficulties()                                          | 13        |
| 6.1.3.3. handleCancel()                                             | 13        |
| 6.1.3.4. handleSubmit()                                             | 14        |
| 6.1.3.5. showDialog()                                               | 14        |
| 6.2. millionaire.model.AddQuestionModel osztályreferencia           | 15        |
| 6.2.1. Részletes leírás                                             | 16        |
| 6.2.2. Konstruktork és destruktorok dokumentációja                  | 16        |
| 6.2.2.1. AddQuestionModel()                                         | 16        |
| 6.2.3. Tagfüggvények dokumentációja                                 | 16        |
| 6.2.3.1. addQuestion()                                              | 16        |
| 6.2.3.2. getCategories()                                            | 18        |
| 6.2.3.3. getDifficulties()                                          | 18        |
| 6.2.3.4. validateInputs()                                           | 18        |
| 6.3. millionaire.view.AddQuestionView osztályreferencia             | 19        |
| 6.3.1. Részletes leírás                                             | 21        |
| 6.3.2. Konstruktork és destruktorok dokumentációja                  | 22        |
| 6.3.2.1. AddQuestionView()                                          | 22        |
| 6.3.3. Tagfüggvények dokumentációja                                 | 22        |
| 6.3.3.1. clearFields()                                              | 22        |

|           |                                                         |    |
|-----------|---------------------------------------------------------|----|
| 6.3.3.2.  | closeWindow()                                           | 22 |
| 6.3.3.3.  | showError()                                             | 23 |
| 6.3.3.4.  | showSuccess()                                           | 23 |
| 6.4.      | millionaire.controller.GameController osztályreferencia | 24 |
| 6.4.1.    | Részletes leírás                                        | 25 |
| 6.4.2.    | Konstruktorok és destruktorok dokumentációja            | 25 |
| 6.4.2.1.  | GameController()                                        | 25 |
| 6.4.3.    | Tagfüggvények dokumentációja                            | 26 |
| 6.4.3.1.  | getCurrentLevel()                                       | 26 |
| 6.4.3.2.  | getCurrentQuestion()                                    | 26 |
| 6.4.3.3.  | getGameView()                                           | 27 |
| 6.4.3.4.  | getPlayer()                                             | 27 |
| 6.4.3.5.  | handleAnswer()                                          | 28 |
| 6.4.3.6.  | setAnswerButtonsEnabled()                               | 28 |
| 6.4.3.7.  | updateGameTime()                                        | 29 |
| 6.5.      | millionaire.model.GameModel osztályreferencia           | 29 |
| 6.5.1.    | Részletes leírás                                        | 31 |
| 6.5.2.    | Konstruktorok és destruktorok dokumentációja            | 31 |
| 6.5.2.1.  | GameModel()                                             | 31 |
| 6.5.3.    | Tagfüggvények dokumentációja                            | 32 |
| 6.5.3.1.  | getCurrentPrize()                                       | 32 |
| 6.5.3.2.  | getCurrentQuestion()                                    | 32 |
| 6.5.3.3.  | getLastSafePrize()                                      | 33 |
| 6.5.3.4.  | getPlayer()                                             | 33 |
| 6.5.3.5.  | getPrizeLevels()                                        | 33 |
| 6.5.3.6.  | isGameOver()                                            | 34 |
| 6.5.3.7.  | isTimeUp()                                              | 34 |
| 6.5.3.8.  | processAnswer()                                         | 35 |
| 6.5.3.9.  | selectNextQuestion()                                    | 35 |
| 6.5.3.10. | updatePlayTime()                                        | 36 |
| 6.6.      | millionaire.model.GameState felsoroló referencia        | 37 |
| 6.6.1.    | Részletes leírás                                        | 38 |
| 6.6.2.    | Tagfüggvények dokumentációja                            | 38 |
| 6.6.2.1.  | isGameEnding()                                          | 38 |
| 6.6.3.    | Adattagok dokumentációja                                | 38 |
| 6.6.3.1.  | CORRECT                                                 | 38 |
| 6.6.3.2.  | LOST                                                    | 38 |
| 6.6.3.3.  | WON                                                     | 38 |
| 6.7.      | millionaire.view.GameView osztályreferencia             | 39 |
| 6.7.1.    | Részletes leírás                                        | 41 |
| 6.7.2.    | Konstruktorok és destruktorok dokumentációja            | 41 |
| 6.7.2.1.  | GameView()                                              | 41 |

|                                                                   |    |
|-------------------------------------------------------------------|----|
| 6.7.3. Tagfüggvények dokumentációja                               | 42 |
| 6.7.3.1. closeGame()                                              | 42 |
| 6.7.3.2. getAnswerButtons()                                       | 42 |
| 6.7.3.3. setAnswerButtonsEnabled()                                | 42 |
| 6.7.3.4. showAnswerResult()                                       | 43 |
| 6.7.3.5. showGameEndDialog()                                      | 44 |
| 6.7.3.6. stopTimer()                                              | 44 |
| 6.7.3.7. updateFinalPlaytime()                                    | 44 |
| 6.7.3.8. updatePrizeLadder()                                      | 45 |
| 6.7.3.9. updateQuestion()                                         | 45 |
| 6.8. millionaire.controller.LifelineController osztályreferencia  | 46 |
| 6.8.1. Részletes leírás                                           | 47 |
| 6.8.2. Konstruktork és destruktorok dokumentációja                | 48 |
| 6.8.2.1. LifelineController()                                     | 48 |
| 6.8.3. Tagfüggvények dokumentációja                               | 48 |
| 6.8.3.1. getLifelinePanel()                                       | 48 |
| 6.8.3.2. setGameContext()                                         | 49 |
| 6.8.3.3. useAudienceHelp()                                        | 49 |
| 6.8.3.4. useFiftyFifty()                                          | 50 |
| 6.9. millionaire.model.LifelineModel osztályreferencia            | 51 |
| 6.9.1. Részletes leírás                                           | 51 |
| 6.9.2. Tagfüggvények dokumentációja                               | 52 |
| 6.9.2.1. getAudienceResult()                                      | 52 |
| 6.9.2.2. getFiftyFiftyResult()                                    | 53 |
| 6.9.2.3. isAudienceHelpAvailable()                                | 53 |
| 6.9.2.4. isFiftyFiftyAvailable()                                  | 54 |
| 6.10. millionaire.view.LifelineView osztályreferencia             | 55 |
| 6.10.1. Részletes leírás                                          | 55 |
| 6.10.2. Konstruktork és destruktorok dokumentációja               | 56 |
| 6.10.2.1. LifelineView()                                          | 56 |
| 6.10.3. Tagfüggvények dokumentációja                              | 56 |
| 6.10.3.1. createLifelinePanel()                                   | 56 |
| 6.10.3.2. showAudienceResults()                                   | 57 |
| 6.10.3.3. updateAnswerButtons()                                   | 57 |
| 6.10.3.4. updateButtonStates()                                    | 58 |
| 6.11. millionaire.Main osztályreferencia                          | 58 |
| 6.11.1. Részletes leírás                                          | 59 |
| 6.11.2. Tagfüggvények dokumentációja                              | 59 |
| 6.11.2.1. isDeveloperMode()                                       | 59 |
| 6.11.2.2. main()                                                  | 60 |
| 6.12. millionaire.controller.MainMenuController osztályreferencia | 60 |
| 6.12.1. Részletes leírás                                          | 61 |

|                                                                 |    |
|-----------------------------------------------------------------|----|
| 6.12.2. Konstruktorkok és destruktorok dokumentációja           | 62 |
| 6.12.2.1. MainMenuController()                                  | 62 |
| 6.12.3. Tagfüggvények dokumentációja                            | 62 |
| 6.12.3.1. exitGame()                                            | 62 |
| 6.12.3.2. showAddQuestionDialog()                               | 62 |
| 6.12.3.3. showMenu()                                            | 63 |
| 6.12.3.4. showRules()                                           | 63 |
| 6.12.3.5. showScoreboard()                                      | 64 |
| 6.12.3.6. startNewGame()                                        | 64 |
| 6.13. millionaire.model.MainMenuModel osztályreferencia         | 65 |
| 6.13.1. Részletes leírás                                        | 65 |
| 6.13.2. Konstruktorkok és destruktorok dokumentációja           | 65 |
| 6.13.2.1. MainMenuModel()                                       | 65 |
| 6.13.3. Tagfüggvények dokumentációja                            | 66 |
| 6.13.3.1. getQuestionRepository()                               | 66 |
| 6.14. millionaire.view.MainMenuView osztályreferencia           | 66 |
| 6.14.1. Részletes leírás                                        | 68 |
| 6.14.2. Konstruktorkok és destruktorok dokumentációja           | 68 |
| 6.14.2.1. MainMenuView()                                        | 68 |
| 6.14.3. Tagfüggvények dokumentációja                            | 68 |
| 6.14.3.1. hideWindow()                                          | 68 |
| 6.14.3.2. showWindow()                                          | 69 |
| 6.15. millionaire.controller.PlayerController osztályreferencia | 69 |
| 6.15.1. Részletes leírás                                        | 70 |
| 6.15.2. Konstruktorkok és destruktorok dokumentációja           | 70 |
| 6.15.2.1. PlayerController()                                    | 70 |
| 6.15.3. Tagfüggvények dokumentációja                            | 71 |
| 6.15.3.1. cancelPlayerCreation()                                | 71 |
| 6.15.3.2. createPlayer()                                        | 71 |
| 6.15.3.3. showPlayerCreationDialog()                            | 72 |
| 6.16. millionaire.model.PlayerModel osztályreferencia           | 73 |
| 6.16.1. Részletes leírás                                        | 75 |
| 6.16.2. Konstruktorkok és destruktorok dokumentációja           | 75 |
| 6.16.2.1. PlayerModel() [1/2]                                   | 75 |
| 6.16.2.2. PlayerModel() [2/2]                                   | 76 |
| 6.16.3. Tagfüggvények dokumentációja                            | 76 |
| 6.16.3.1. compareTo()                                           | 76 |
| 6.16.3.2. equals()                                              | 77 |
| 6.16.3.3. getCurrentLevel()                                     | 78 |
| 6.16.3.4. getFormattedPlaytime()                                | 78 |
| 6.16.3.5. getFormattedPrize()                                   | 78 |
| 6.16.3.6. getName()                                             | 79 |

|                                                                                     |    |
|-------------------------------------------------------------------------------------|----|
| 6.16.3.7. <code>getPlaytime()</code> . . . . .                                      | 79 |
| 6.16.3.8. <code>getPrize()</code> . . . . .                                         | 79 |
| 6.16.3.9. <code>getStartTime()</code> . . . . .                                     | 80 |
| 6.16.3.10. <code>incrementLevel()</code> . . . . .                                  | 80 |
| 6.16.3.11. <code>setCurrentLevel()</code> . . . . .                                 | 80 |
| 6.16.3.12. <code>setName()</code> . . . . .                                         | 80 |
| 6.16.3.13. <code>setPlaytime()</code> . . . . .                                     | 81 |
| 6.16.3.14. <code>setPrize()</code> . . . . .                                        | 81 |
| 6.16.3.15. <code>updatePlaytime()</code> . . . . .                                  | 81 |
| 6.17. <code>millionaire.view.PlayerView</code> osztályreferencia . . . . .          | 82 |
| 6.17.1. Részletes leírás . . . . .                                                  | 83 |
| 6.17.2. Konstruktork és destruktorok dokumentációja . . . . .                       | 84 |
| 6.17.2.1. <code>PlayerView()</code> . . . . .                                       | 84 |
| 6.17.3. Tagfüggvények dokumentációja . . . . .                                      | 84 |
| 6.17.3.1. <code>closeDialog()</code> . . . . .                                      | 84 |
| 6.17.3.2. <code>getPlayerName()</code> . . . . .                                    | 85 |
| 6.17.3.3. <code>showError()</code> . . . . .                                        | 85 |
| 6.18. <code>millionaire.model.PrizeLevel</code> osztályreferencia . . . . .         | 85 |
| 6.18.1. Részletes leírás . . . . .                                                  | 86 |
| 6.18.2. Konstruktork és destruktorok dokumentációja . . . . .                       | 86 |
| 6.18.2.1. <code>PrizeLevel()</code> . . . . .                                       | 86 |
| 6.18.3. Tagfüggvények dokumentációja . . . . .                                      | 87 |
| 6.18.3.1. <code>getLevel()</code> . . . . .                                         | 87 |
| 6.18.3.2. <code>getPrize()</code> . . . . .                                         | 87 |
| 6.18.3.3. <code>isSafeSpot()</code> . . . . .                                       | 87 |
| 6.19. <code>millionaire.model.Question</code> osztályreferencia . . . . .           | 88 |
| 6.19.1. Részletes leírás . . . . .                                                  | 90 |
| 6.19.2. Tagfüggvények dokumentációja . . . . .                                      | 90 |
| 6.19.2.1. <code>getCategory()</code> . . . . .                                      | 90 |
| 6.19.2.2. <code>getCorrectAnswer()</code> . . . . .                                 | 90 |
| 6.19.2.3. <code>getDifficulty()</code> . . . . .                                    | 91 |
| 6.19.2.4. <code>getOptionA()</code> . . . . .                                       | 91 |
| 6.19.2.5. <code>getOptionB()</code> . . . . .                                       | 91 |
| 6.19.2.6. <code>getOptionC()</code> . . . . .                                       | 92 |
| 6.19.2.7. <code>getOptionD()</code> . . . . .                                       | 92 |
| 6.19.2.8. <code>getQuestion()</code> . . . . .                                      | 93 |
| 6.20. <code>millionaire.model.QuestionRepository</code> osztályreferencia . . . . . | 93 |
| 6.20.1. Részletes leírás . . . . .                                                  | 95 |
| 6.20.2. Konstruktork és destruktorok dokumentációja . . . . .                       | 95 |
| 6.20.2.1. <code>QuestionRepository()</code> . . . . .                               | 95 |
| 6.20.3. Tagfüggvények dokumentációja . . . . .                                      | 95 |
| 6.20.3.1. <code>addQuestion()</code> . . . . .                                      | 95 |

|                                                                     |     |
|---------------------------------------------------------------------|-----|
| 6.20.3.2. getAllQuestions()                                         | 96  |
| 6.20.3.3. getQuestionsByCategory()                                  | 96  |
| 6.20.3.4. getQuestionsByDifficulty()                                | 96  |
| 6.20.4. Adattagok dokumentációja                                    | 97  |
| 6.20.4.1. CATEGORIES                                                | 97  |
| 6.20.4.2. DIFFICULTIES                                              | 97  |
| 6.21. millionaire.controller.RulesController osztályreferencia      | 98  |
| 6.21.1. Részletes leírás                                            | 98  |
| 6.21.2. Konstruktorok és destruktorok dokumentációja                | 98  |
| 6.21.2.1. RulesController()                                         | 98  |
| 6.21.3. Tagfüggvények dokumentációja                                | 99  |
| 6.21.3.1. handleCloseRequest()                                      | 99  |
| 6.21.3.2. showRules()                                               | 99  |
| 6.22. millionaire.model.RulesModel osztályreferencia                | 100 |
| 6.22.1. Részletes leírás                                            | 100 |
| 6.22.2. Konstruktorok és destruktorok dokumentációja                | 100 |
| 6.22.2.1. RulesModel()                                              | 100 |
| 6.22.3. Tagfüggvények dokumentációja                                | 101 |
| 6.22.3.1. getRulesContent()                                         | 101 |
| 6.22.3.2. isContentLoaded()                                         | 101 |
| 6.23. millionaire.view.RulesView osztályreferencia                  | 101 |
| 6.23.1. Részletes leírás                                            | 103 |
| 6.23.2. Konstruktorok és destruktorok dokumentációja                | 103 |
| 6.23.2.1. RulesView()                                               | 103 |
| 6.23.3. Tagfüggvények dokumentációja                                | 103 |
| 6.23.3.1. closeWindow()                                             | 103 |
| 6.23.3.2. displayRules()                                            | 104 |
| 6.23.3.3. showError()                                               | 104 |
| 6.24. millionaire.controller.ScoreboardController osztályreferencia | 104 |
| 6.24.1. Részletes leírás                                            | 105 |
| 6.24.2. Konstruktorok és destruktorok dokumentációja                | 106 |
| 6.24.2.1. ScoreboardController()                                    | 106 |
| 6.24.3. Tagfüggvények dokumentációja                                | 106 |
| 6.24.3.1. addScore()                                                | 106 |
| 6.24.3.2. handleClose()                                             | 106 |
| 6.24.3.3. showScoreboard()                                          | 107 |
| 6.25. millionaire.model.ScoreboardModel osztályreferencia           | 108 |
| 6.25.1. Részletes leírás                                            | 108 |
| 6.25.2. Konstruktorok és destruktorok dokumentációja                | 109 |
| 6.25.2.1. ScoreboardModel()                                         | 109 |
| 6.25.3. Tagfüggvények dokumentációja                                | 109 |
| 6.25.3.1. addScore()                                                | 109 |



|                                                                                                     |            |
|-----------------------------------------------------------------------------------------------------|------------|
| 6.25.3.2. <a href="#">getPlayers()</a>                                                              | 110        |
| 6.26. <a href="#">millionaire.view.ScoreboardView osztályreferencia</a>                             | 110        |
| 6.26.1. <a href="#">Részletes leírás</a>                                                            | 111        |
| 6.26.2. <a href="#">Konstruktorok és destruktorok dokumentációja</a>                                | 112        |
| 6.26.2.1. <a href="#">ScoreboardView()</a>                                                          | 112        |
| 6.26.3. <a href="#">Tagfüggvények dokumentációja</a>                                                | 113        |
| 6.26.3.1. <a href="#">closeWindow()</a>                                                             | 113        |
| 6.26.3.2. <a href="#">updateScores()</a>                                                            | 113        |
| 6.27. <a href="#">millionaire.controller.TimerController osztályreferencia</a>                      | 114        |
| 6.27.1. <a href="#">Részletes leírás</a>                                                            | 114        |
| 6.27.2. <a href="#">Konstruktorok és destruktorok dokumentációja</a>                                | 115        |
| 6.27.2.1. <a href="#">TimerController()</a>                                                         | 115        |
| 6.27.3. <a href="#">Tagfüggvények dokumentációja</a>                                                | 115        |
| 6.27.3.1. <a href="#">getElapsedTime()</a>                                                          | 115        |
| 6.27.3.2. <a href="#">getView()</a>                                                                 | 116        |
| 6.27.3.3. <a href="#">startTimer()</a>                                                              | 116        |
| 6.27.3.4. <a href="#">stopTimer()</a>                                                               | 117        |
| 6.28. <a href="#">millionaire.model.TimerModel osztályreferencia</a>                                | 117        |
| 6.28.1. <a href="#">Részletes leírás</a>                                                            | 118        |
| 6.28.2. <a href="#">Konstruktorok és destruktorok dokumentációja</a>                                | 119        |
| 6.28.2.1. <a href="#">TimerModel()</a>                                                              | 119        |
| 6.28.3. <a href="#">Tagfüggvények dokumentációja</a>                                                | 119        |
| 6.28.3.1. <a href="#">getElapsedTime()</a>                                                          | 119        |
| 6.28.3.2. <a href="#">getStartTime()</a>                                                            | 119        |
| 6.28.3.3. <a href="#">start()</a>                                                                   | 120        |
| 6.28.3.4. <a href="#">stop()</a>                                                                    | 120        |
| 6.28.3.5. <a href="#">updateElapsedTime()</a>                                                       | 120        |
| 6.29. <a href="#">millionaire.view.TimerView osztályreferencia</a>                                  | 121        |
| 6.29.1. <a href="#">Részletes leírás</a>                                                            | 122        |
| 6.29.2. <a href="#">Konstruktorok és destruktorok dokumentációja</a>                                | 122        |
| 6.29.2.1. <a href="#">TimerView()</a>                                                               | 122        |
| 6.29.3. <a href="#">Tagfüggvények dokumentációja</a>                                                | 122        |
| 6.29.3.1. <a href="#">updateDisplay()</a>                                                           | 122        |
| <b>7. Fájlok dokumentációja</b>                                                                     | <b>123</b> |
| 7.1. <a href="#">src/main/java/millionaire/controller/AddQuestionController.java fájlreferencia</a> | 123        |
| 7.2. <a href="#">AddQuestionController.java</a>                                                     | 123        |
| 7.3. <a href="#">src/main/java/millionaire/controller/GameController.java fájlreferencia</a>        | 124        |
| 7.4. <a href="#">GameController.java</a>                                                            | 124        |
| 7.5. <a href="#">src/main/java/millionaire/controller/LifelineController.java fájlreferencia</a>    | 126        |
| 7.6. <a href="#">LifelineController.java</a>                                                        | 126        |
| 7.7. <a href="#">src/main/java/millionaire/controller/MainMenuController.java fájlreferencia</a>    | 127        |

|                                                                                     |     |
|-------------------------------------------------------------------------------------|-----|
| 7.8. MainMenuController.java                                                        | 128 |
| 7.9. src/main/java/millionaire/controller/PlayerController.java fájlreferencia      | 128 |
| 7.10. PlayerController.java                                                         | 129 |
| 7.11. src/main/java/millionaire/controller/RulesController.java fájlreferencia      | 129 |
| 7.12. RulesController.java                                                          | 129 |
| 7.13. src/main/java/millionaire/controller/ScoreboardController.java fájlreferencia | 130 |
| 7.14. ScoreboardController.java                                                     | 130 |
| 7.15. src/main/java/millionaire/controller/TimerController.java fájlreferencia      | 131 |
| 7.16. TimerController.java                                                          | 131 |
| 7.17. src/main/java/millionaire/Main.java fájlreferencia                            | 132 |
| 7.18. Main.java                                                                     | 132 |
| 7.19. src/main/java/millionaire/model/AddQuestionModel.java fájlreferencia          | 132 |
| 7.20. AddQuestionModel.java                                                         | 133 |
| 7.21. src/main/java/millionaire/model/GameModel.java fájlreferencia                 | 133 |
| 7.22. GameModel.java                                                                | 134 |
| 7.23. src/main/java/millionaire/model/GameState.java fájlreferencia                 | 135 |
| 7.24. GameState.java                                                                | 135 |
| 7.25. src/main/java/millionaire/model/LifelineModel.java fájlreferencia             | 135 |
| 7.26. LifelineModel.java                                                            | 136 |
| 7.27. src/main/java/millionaire/model/MainMenuModel.java fájlreferencia             | 137 |
| 7.28. MainMenuModel.java                                                            | 137 |
| 7.29. src/main/java/millionaire/model/PlayerModel.java fájlreferencia               | 137 |
| 7.30. PlayerModel.java                                                              | 138 |
| 7.31. src/main/java/millionaire/model/PrizeLevel.java fájlreferencia                | 139 |
| 7.32. PrizeLevel.java                                                               | 139 |
| 7.33. src/main/java/millionaire/model/Question.java fájlreferencia                  | 139 |
| 7.34. Question.java                                                                 | 140 |
| 7.35. src/main/java/millionaire/model/QuestionRepository.java fájlreferencia        | 141 |
| 7.36. QuestionRepository.java                                                       | 142 |
| 7.37. src/main/java/millionaire/model/RulesModel.java fájlreferencia                | 143 |
| 7.38. RulesModel.java                                                               | 143 |
| 7.39. src/main/java/millionaire/model/ScoreboardModel.java fájlreferencia           | 144 |
| 7.40. ScoreboardModel.java                                                          | 144 |
| 7.41. src/main/java/millionaire/model/TimerModel.java fájlreferencia                | 146 |
| 7.42. TimerModel.java                                                               | 146 |
| 7.43. src/main/java/millionaire/view/AddQuestionView.java fájlreferencia            | 146 |
| 7.44. AddQuestionView.java                                                          | 147 |
| 7.45. src/main/java/millionaire/view/GameView.java fájlreferencia                   | 149 |
| 7.46. GameView.java                                                                 | 149 |
| 7.47. src/main/java/millionaire/view/LifelineView.java fájlreferencia               | 153 |
| 7.48. LifelineView.java                                                             | 153 |
| 7.49. src/main/java/millionaire/view/MainMenuView.java fájlreferencia               | 155 |

---

|                                                                                   |            |
|-----------------------------------------------------------------------------------|------------|
| 7.50. MainMenuView.java . . . . .                                                 | 155        |
| 7.51. src/main/java/millionaire/view/PlayerView.java fájlreferencia . . . . .     | 157        |
| 7.52. PlayerView.java . . . . .                                                   | 157        |
| 7.53. src/main/java/millionaire/view/RulesView.java fájlreferencia . . . . .      | 158        |
| 7.54. RulesView.java . . . . .                                                    | 158        |
| 7.55. src/main/java/millionaire/view/ScoreboardView.java fájlreferencia . . . . . | 160        |
| 7.56. ScoreboardView.java . . . . .                                               | 160        |
| 7.57. src/main/java/millionaire/view/TimerView.java fájlreferencia . . . . .      | 162        |
| 7.58. TimerView.java . . . . .                                                    | 162        |
| <b>Tárgymutató</b>                                                                | <b>163</b> |



# 1. fejezet

## Névtérmutató

### 1.1. Package List

A csomagok rövid leírásai (ha léteznek):

|                                                  |    |
|--------------------------------------------------|----|
| <a href="#">millionaire</a> . . . . .            | 9  |
| <a href="#">millionaire.controller</a> . . . . . | 9  |
| <a href="#">millionaire.model</a> . . . . .      | 10 |
| <a href="#">millionaire.view</a> . . . . .       | 10 |



## 2. fejezet

# Hierarchikus mutató

### 2.1. Osztályhierarchia

Majdnem (de nem teljesen) betűrendbe szedett leszármazási lista:

|                                                        |     |
|--------------------------------------------------------|-----|
| millionaire.controller.AddQuestionController . . . . . | 11  |
| millionaire.model.AddQuestionModel . . . . .           | 15  |
| Comparable                                             |     |
| millionaire.model.PlayerModel . . . . .                | 73  |
| millionaire.controller.GameController . . . . .        | 24  |
| millionaire.model.GameModel . . . . .                  | 29  |
| millionaire.model.GameState . . . . .                  | 37  |
| JDialog                                                |     |
| millionaire.view.PlayerView . . . . .                  | 82  |
| millionaire.controller.LifelineController . . . . .    | 46  |
| millionaire.model.LifelineModel . . . . .              | 51  |
| millionaire.view.LifelineView . . . . .                | 55  |
| millionaire.Main . . . . .                             | 58  |
| millionaire.controller.MainMenuController . . . . .    | 60  |
| millionaire.model.MainMenuModel . . . . .              | 65  |
| millionaire.controller.PlayerController . . . . .      | 69  |
| millionaire.model.PrizeLevel . . . . .                 | 85  |
| millionaire.model.Question . . . . .                   | 88  |
| millionaire.model.QuestionRepository . . . . .         | 93  |
| millionaire.controller.RulesController . . . . .       | 98  |
| millionaire.model.RulesModel . . . . .                 | 100 |
| millionaire.controller.ScoreboardController . . . . .  | 104 |
| millionaire.model.ScoreboardModel . . . . .            | 108 |
| millionaire.controller.TimerController . . . . .       | 114 |
| millionaire.model.TimerModel . . . . .                 | 117 |
| JFrame                                                 |     |
| millionaire.view.AddQuestionView . . . . .             | 19  |
| millionaire.view.GameView . . . . .                    | 39  |
| millionaire.view.MainMenuView . . . . .                | 66  |
| millionaire.view.RulesView . . . . .                   | 101 |
| millionaire.view.ScoreboardView . . . . .              | 110 |
| JPanel                                                 |     |
| millionaire.view.TimerView . . . . .                   | 121 |





## 3. fejezet

# Osztálymutató

### 3.1. Osztálylista

Az összes osztály, struktúra, unió és interfész listája rövid leírásokkal:

|                                                              |    |
|--------------------------------------------------------------|----|
| <a href="#">millionaire.controller.AddQuestionController</a> |    |
| Új kérdések hozzáadását vezérlő osztály                      | 11 |
| <a href="#">millionaire.model.AddQuestionModel</a>           |    |
| Új kérdések hozzáadását kezelő modell osztály                | 15 |
| <a href="#">millionaire.view.AddQuestionView</a>             |    |
| Új kérdések hozzáadására szolgáló grafikus felület           | 19 |
| <a href="#">millionaire.controller.GameController</a>        |    |
| A játék fő vezérlő osztálya                                  | 24 |
| <a href="#">millionaire.model.GameModel</a>                  |    |
| A játék logikáját és állapotát kezelő osztály                | 29 |
| <a href="#">millionaire.model.GameState</a>                  |    |
| Enum osztály, amely a játék állapotait reprezentálja         | 37 |
| <a href="#">millionaire.view.GameView</a>                    |    |
| A játék fő grafikus felülete                                 | 39 |
| <a href="#">millionaire.controller.LifelineController</a>    |    |
| A játék segítségét kezelő vezérlő osztály                    | 46 |
| <a href="#">millionaire.model.LifelineModel</a>              |    |
| Segítségek (lifeline) logikáját kezelő osztály               | 51 |
| <a href="#">millionaire.view.LifelineView</a>                |    |
| A játék segítségét megjelenítő és kezelő felület             | 55 |
| <a href="#">millionaire.Main</a>                             |    |
| A "Legyen Ön is Milliomos!" kvízzjáték főprogramja           | 58 |
| <a href="#">millionaire.controller.MainMenuController</a>    |    |
| A főmenü vezérlő osztálya                                    | 60 |
| <a href="#">millionaire.model.MainMenuModel</a>              |    |
| A főmenü modell osztálya                                     | 65 |
| <a href="#">millionaire.view.MainMenuView</a>                |    |
| A játék főmenüjének grafikus felülete                        | 66 |
| <a href="#">millionaire.controller.PlayerController</a>      |    |
| A játékos létrehozását kezelő vezérlő osztály                | 69 |
| <a href="#">millionaire.model.PlayerModel</a>                |    |
| <a href="#">millionaire.view.PlayerView</a>                  |    |
| Játékos létrehozására szolgáló dialógusablak                 | 82 |
| <a href="#">millionaire.model.PrizeLevel</a>                 |    |
| Nyereményszintet reprezentáló osztály                        | 85 |

|                                                             |     |
|-------------------------------------------------------------|-----|
| <a href="#">millionaire.model.Question</a>                  |     |
| Egy kérdést reprezentáló osztály . . . . .                  | 88  |
| <a href="#">millionaire.model.QuestionRepository</a>        |     |
| Kérdések perzisztens tárolását végző osztály . . . . .      | 93  |
| <a href="#">millionaire.controller.RulesController</a>      |     |
| A játékszabályokat kezelő vezérlő osztály . . . . .         | 98  |
| <a href="#">millionaire.model.RulesModel</a>                |     |
| Játékszabályok kezelését végző osztály . . . . .            | 100 |
| <a href="#">millionaire.view.RulesView</a>                  |     |
| A játékszabályok megjelenítésére szolgáló ablak . . . . .   | 101 |
| <a href="#">millionaire.controller.ScoreboardController</a> |     |
| A dicsőséglista kezelését végző vezérlő osztály . . . . .   | 104 |
| <a href="#">millionaire.model.ScoreboardModel</a>           |     |
| Dicsőséglista kezelését végző osztály . . . . .             | 108 |
| <a href="#">millionaire.view.ScoreboardView</a>             |     |
| A dicsőséglista megjelenítésére szolgáló ablak . . . . .    | 110 |
| <a href="#">millionaire.controller.TimerController</a>      |     |
| Az időmérő vezérlője . . . . .                              | 114 |
| <a href="#">millionaire.model.TimerModel</a>                |     |
| Az időmérő adatmodellje . . . . .                           | 117 |
| <a href="#">millionaire.view.TimerView</a>                  |     |
| Az időmérő megjelenítése . . . . .                          | 121 |

## 4. fejezet

# Fájlmutató

### 4.1. Fájllista

Az összes fájl listája rövid leírásokkal:

|                                                                 |     |
|-----------------------------------------------------------------|-----|
| src/main/java/millionaire/Main.java                             | 132 |
| src/main/java/millionaire/controller/AddQuestionController.java | 123 |
| src/main/java/millionaire/controller/GameController.java        | 124 |
| src/main/java/millionaire/controller/LifelineController.java    | 126 |
| src/main/java/millionaire/controller/MainMenuController.java    | 127 |
| src/main/java/millionaire/controller/PlayerController.java      | 128 |
| src/main/java/millionaire/controller/RulesController.java       | 129 |
| src/main/java/millionaire/controller/ScoreboardController.java  | 130 |
| src/main/java/millionaire/controller/TimerController.java       | 131 |
| src/main/java/millionaire/model/AddQuestionModel.java           | 132 |
| src/main/java/millionaire/model/GameModel.java                  | 133 |
| src/main/java/millionaire/model/GameState.java                  | 135 |
| src/main/java/millionaire/model/LifelineModel.java              | 135 |
| src/main/java/millionaire/model/MainMenuModel.java              | 137 |
| src/main/java/millionaire/model/PlayerModel.java                | 137 |
| src/main/java/millionaire/model/PrizeLevel.java                 | 139 |
| src/main/java/millionaire/model/Question.java                   | 139 |
| src/main/java/millionaire/model/QuestionRepository.java         | 141 |
| src/main/java/millionaire/model/RulesModel.java                 | 143 |
| src/main/java/millionaire/model/ScoreboardModel.java            | 144 |
| src/main/java/millionaire/model/TimerModel.java                 | 146 |
| src/main/java/millionaire/view/AddQuestionView.java             | 146 |
| src/main/java/millionaire/view/GameView.java                    | 149 |
| src/main/java/millionaire/view/LifelineView.java                | 153 |
| src/main/java/millionaire/view/MainMenuView.java                | 155 |
| src/main/java/millionaire/view/PlayerView.java                  | 157 |
| src/main/java/millionaire/view/RulesView.java                   | 158 |
| src/main/java/millionaire/view/ScoreboardView.java              | 160 |
| src/main/java/millionaire/view/TimerView.java                   | 162 |



## 5. fejezet

# Névterek dokumentációja

### 5.1. millionaire csomag

#### Csomagok

- package [controller](#)
- package [model](#)
- package [view](#)

#### Osztályok

- class [Main](#)  
*A "Legyen Ön is Milliomos!" kvízzjáték főprogramja.*

### 5.2. millionaire.controller csomag

#### Osztályok

- class [AddQuestionController](#)  
*Új kérdések hozzáadását vezérlő osztály.*
- class [GameController](#)  
*A játék fő vezérlő osztálya.*
- class [LifelineController](#)  
*A játék segítségait kezelő vezérlő osztály.*
- class [MainMenuController](#)  
*A főmenü vezérlő osztálya.*
- class [PlayerController](#)  
*A játékos létrehozását kezelő vezérlő osztály.*
- class [RulesController](#)  
*A játékszabályokat kezelő vezérlő osztály.*
- class [ScoreboardController](#)  
*A dicsőséglista kezelését végző vezérlő osztály.*
- class [TimerController](#)  
*Az időmérő vezérlője.*

## 5.3. millionaire.model csomag

### Osztályok

- class [AddQuestionModel](#)  
*Új kérdések hozzáadását kezelő modell osztály.*
- class [GameModel](#)  
*A játék logikáját és állapotát kezelő osztály.*
- enum [GameState](#)  
*Enum osztály, amely a játék állapotait reprezentálja.*
- class [LifelineModel](#)  
*Segítségek (lifeline) logikáját kezelő osztály.*
- class [MainMenuModel](#)  
*A főmenü modell osztálya.*
- class [PlayerModel](#)
- class [PrizeLevel](#)  
*Nyereményszintet reprezentáló osztály.*
- class [Question](#)  
*Egy kérdést reprezentáló osztály.*
- class [QuestionRepository](#)  
*Kérdések perzisztens tárolását végző osztály.*
- class [RulesModel](#)  
*Játékszabályok kezelését végző osztály.*
- class [ScoreboardModel](#)  
*Dicsőségtábla kezelését végző osztály.*
- class [TimerModel](#)  
*Az időmérő adatmodellje.*

## 5.4. millionaire.view csomag

### Osztályok

- class [AddQuestionView](#)  
*Új kérdések hozzáadására szolgáló grafikus felület.*
- class [GameView](#)  
*A játék fő grafikus felülete.*
- class [LifelineView](#)  
*A játék segítségeit megjelenítő és kezelő felület.*
- class [MainMenuView](#)  
*A játék főmenüjének grafikus felülete.*
- class [PlayerView](#)  
*Játékos létrehozására szolgáló dialógusablak.*
- class [RulesView](#)  
*A játékszabályok megjelenítésére szolgáló ablak.*
- class [ScoreboardView](#)  
*A dicsőséglista megjelenítésére szolgáló ablak.*
- class [TimerView](#)  
*Az időmérő megjelenítése.*

## 6. fejezet

# Osztályok dokumentációja

### 6.1. millionaire.controller.AddQuestionController osztályreferencia

Új kérdések hozzáadását vezérlő osztály.

A millionaire.controller.AddQuestionController osztály együttműködési diagramja:

| millionaire.controller.Add<br>QuestionController                                                                                                                                                                                                                                                                                                                                                                                       |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div>+ AddQuestionController<br/>(QuestionRepository repository)</div> <div>+ void handleSubmit(String<br/>question, String optionA,<br/>String optionB, String optionC,<br/>String optionD, String correctAnswer,<br/>String difficulty, String category)</div> <div>+ void handleCancel()</div> <div>+ List&lt; String &gt; getDifficulties()</div> <div>+ List&lt; String &gt; getCategories()</div> <div>+ void showDialog()</div> |

#### Publikus tagfüggvények

- [AddQuestionController](#) ([QuestionRepository](#) repository)  
*Konstruktor az AddQuestionController létrehozásához.*
- void [handleSubmit](#) (String question, String optionA, String optionB, String optionC, String optionD, String correctAnswer, String difficulty, String category)

- *A kérdés hozzáadása gombra kattintáskor meghívott metódus.*
- void `handleCancel ()`  
*Bezárja a kérdés hozzáadási ablakot.*
- List< String > `getDifficulties ()`  
*Lekérdezi az elérhető nehézségi szinteket.*
- List< String > `getCategories ()`  
*Lekérdezi az elérhető kategóriákat.*
- void `showDialog ()`  
*Megjeleníti a kérdés hozzáadási ablakot.*

### 6.1.1. Részletes leírás

Új kérdések hozzáadását vezérlő osztály.

Ez az osztály felelős a kérdés hozzáadási folyamat vezérléséért:

- Kezeli a felhasználói inputot a kérdés hozzáadási nézetből
- Validálja a bevitt adatokat
- Továbbítja az adatokat a modellnek feldolgozásra
- Visszajelzést ad a felhasználónak a művelet eredményéről

Definíció a(z) `AddQuestionController.java` fájl 18. sorában.

### 6.1.2. Konstruktorok és destruktorok dokumentációja

#### 6.1.2.1. AddQuestionController()

```
millionaire.controller.AddQuestionController.AddQuestionController (
    QuestionRepository repository )
```

Konstruktor az AddQuestionController létrehozásához.

Paraméterek

|                         |                               |
|-------------------------|-------------------------------|
| <code>repository</code> | A kérdéseket tároló adatbázis |
|-------------------------|-------------------------------|

Definíció a(z) `AddQuestionController.java` fájl 34. sorában.

### 6.1.3. Tagfüggvények dokumentációja

#### 6.1.3.1. getCategories()

```
List< String > millionaire.controller.AddQuestionController.getCategories ( )
```

Lekérdezi az elérhető kategóriákat.



**Visszatérési érték**

A kategóriák listája

Definíció a(z) [AddQuestionController.java](#) fájl 95. sorában.

A függvény hívási gráfja:

**6.1.3.2. getDifficulties()**

```
List< String > millionaire.controller.AddQuestionController.getDifficulties ( )
```

Lekérdezi az elérhető nehézségi szinteket.

**Visszatérési érték**

A nehézségi szintek listája

Definíció a(z) [AddQuestionController.java](#) fájl 86. sorában.

A függvény hívási gráfja:

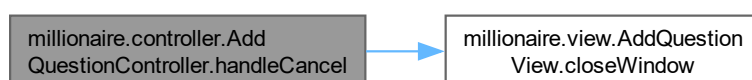
**6.1.3.3. handleCancel()**

```
void millionaire.controller.AddQuestionController.handleCancel ( )
```

Bezárja a kérdés hozzáadási ablakot.

Definíció a(z) [AddQuestionController.java](#) fájl 77. sorában.

A függvény hívási gráfja:



#### 6.1.3.4. handleSubmit()

```
void millionaire.controller.AddQuestionController.handleSubmit (
    String question,
    String optionA,
    String optionB,
    String optionC,
    String optionD,
    String correctAnswer,
    String difficulty,
    String category )
```

A kérdés hozzáadása gombra kattintáskor meghívott metódus.

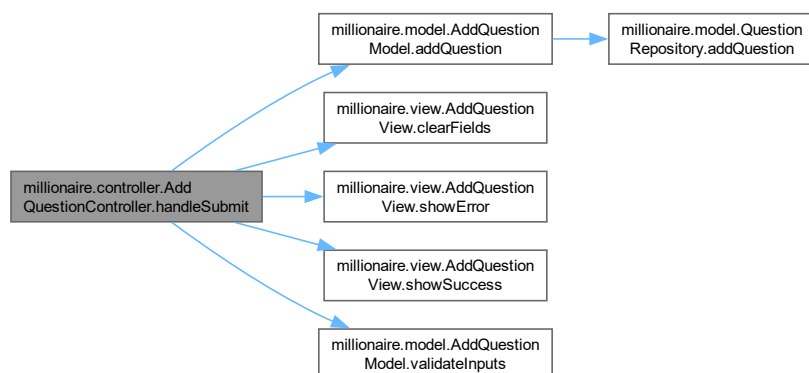
Ellenőrzi a bemeneti mezők érvényességét, majd hozzáadja a kérdést a repository-hoz. Sikeres hozzáadás esetén visszajelzést ad és törli a mezőket. Hiba esetén hibaüzenetet jelenít meg.

##### Paraméterek

|                      |                            |
|----------------------|----------------------------|
| <i>question</i>      | A kérdés szövege           |
| <i>optionA</i>       | Az A válaszlehetőség       |
| <i>optionB</i>       | A B válaszlehetőség        |
| <i>optionC</i>       | A C válaszlehetőség        |
| <i>optionD</i>       | A D válaszlehetőség        |
| <i>correctAnswer</i> | A helyes válasz betűjele   |
| <i>difficulty</i>    | A kérdés nehézségi szintje |
| <i>category</i>      | A kérdés kategóriája       |

Definíció a(z) [AddQuestionController.java](#) fájl 56. sorában.

A függvény hívási gráfja:



#### 6.1.3.5. showDialog()

```
void millionaire.controller.AddQuestionController.showDialog ( )
```

Megjeleníti a kérdés hozzáadási ablakot.

Definíció a(z) [AddQuestionController.java](#) fájl 103. sorában.

A függvény hívó gráfja:



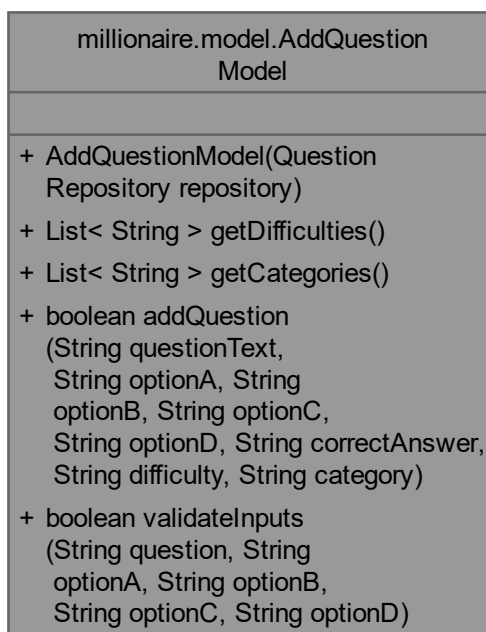
Ez a dokumentáció az osztályról a következő fájl alapján készült:

- [src/main/java/millionaire/controller/AddQuestionController.java](#)

## 6.2. millionaire.model.AddQuestionModel osztályreferencia

Új kérdések hozzáadását kezelő modell osztály.

A millionaire.model.AddQuestionModel osztály együttműködési diagramja:



## Publikus tagfüggvények

- [AddQuestionModel](#) ([QuestionRepository](#) repository)  
*Konstruktor az AddQuestionModel létrehozásához.*
- List< String > [getDifficulties](#) ()  
*Visszaadja a kérdések nehézségeit.*
- List< String > [getCategories](#) ()  
*Visszaadja a kérdések kategóriáit.*
- boolean [addQuestion](#) (String questionText, String optionA, String optionB, String optionC, String optionD, String correctAnswer, String difficulty, String category)  
*A QuestionBuilder segítségével létrehoz egy új kérdést, majd hozzáadja a kérdéseket tároló repository-hoz.*
- boolean [validateInputs](#) (String question, String optionA, String optionB, String optionC, String optionD)  
*Ellenőrzi, hogy a megadott bemeneti mezők érvényesek-e.*

### 6.2.1. Részletes leírás

Új kérdések hozzáadását kezelő modell osztály.

Ez az osztály felelős az új kérdések létrehozásáért és validálásáért, valamint a kérdések tárolóhoz való hozzáadásáért.

Definíció a(z) [AddQuestionModel.java](#) fájl 12. sorában.

### 6.2.2. Konstruktorok és destruktorok dokumentációja

#### 6.2.2.1. AddQuestionModel()

```
millionaire.model.AddQuestionModel.AddQuestionModel (
    QuestionRepository repository )
```

Konstruktor az AddQuestionModel létrehozásához.

Paraméterek

|                   |                               |
|-------------------|-------------------------------|
| <i>repository</i> | A kérdéseket tároló adatbázis |
|-------------------|-------------------------------|

Definíció a(z) [AddQuestionModel.java](#) fájl 20. sorában.

### 6.2.3. Tagfüggvények dokumentációja

#### 6.2.3.1. addQuestion()

```
boolean millionaire.model.AddQuestionModel.addQuestion (
    String questionText,
    String optionA,
    String optionB,
    String optionC,
    String optionD,
```

```
String correctAnswer,
String difficulty,
String category )
```

A QuestionBuilder segítségével létrehoz egy új kérdést, majd hozzáadja a kérdéseket tároló repository-hoz.

#### Paraméterek

|                      |                                           |
|----------------------|-------------------------------------------|
| <i>questionText</i>  | A kérdés szövege                          |
| <i>optionA</i>       | A kérdéshez tartozó A válasz              |
| <i>optionB</i>       | A kérdéshez tartozó B válasz              |
| <i>optionC</i>       | A kérdéshez tartozó C válasz              |
| <i>optionD</i>       | A kérdéshez tartozó D válasz              |
| <i>correctAnswer</i> | A helyes válasz betűjele (A, B, C vagy D) |
| <i>difficulty</i>    | A kérdés nehézségi szintje (1-15)         |
| <i>category</i>      | A kérdés kategóriája                      |

#### Visszatérési érték

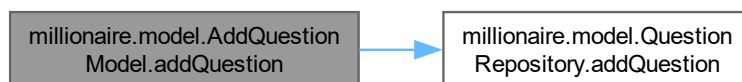
True, ha a kérdés hozzáadása a tárolóhoz sikeres, egyébként false

#### Kivételek

|                                 |                                            |
|---------------------------------|--------------------------------------------|
| <i>NumberFormatException</i>    | Ha a nehézségi szint nem alakítható számmá |
| <i>IllegalArgumentException</i> | Ha a builder érvénytelen adatokat kap      |

Definíció a(z) [AddQuestionModel.java](#) fájl 57. sorában.

A függvény hívási gráfja:



A függvény hívó gráfja:



### 6.2.3.2. getCategories()

```
List< String > millionaire.model.AddQuestionModel.getCategories ( )
```

Visszaadja a kérdések kategóriáit.

#### Visszatérési érték

A kérdések kategóriái.

Definíció a(z) [AddQuestionModel.java](#) fájl 38. sorában.

A függvény hívó gráfja:



### 6.2.3.3. getDifficulties()

```
List< String > millionaire.model.AddQuestionModel.getDifficulties ( )
```

Visszaadja a kérdések nehézségeit.

#### Visszatérési érték

A kérdések nehézségei.

Definíció a(z) [AddQuestionModel.java](#) fájl 29. sorában.

A függvény hívó gráfja:



### 6.2.3.4. validateInputs()

```
boolean millionaire.model.AddQuestionModel.validateInputs (
    String question,
    String optionA,
    String optionB,
    String optionC,
    String optionD )
```

Ellenőrzi, hogy a megadott bemeneti mezők érvényesek-e.

Egy bemenet akkor érvényes, ha egyik kötelező mező sem üres. Az üres sztringeket és a csak whitespace karaktereket tartalmazó sztringeket is üresnek tekintjük.

## Paraméterek

|                 |                              |
|-----------------|------------------------------|
| <i>question</i> | A feltenni kívánt kérdés     |
| <i>optionA</i>  | A kérdéshez tartozó A válasz |
| <i>optionB</i>  | A kérdéshez tartozó B válasz |
| <i>optionC</i>  | A kérdéshez tartozó C válasz |
| <i>optionD</i>  | A kérdéshez tartozó D válasz |

## Visszatérési érték

True, ha minden szükséges bemeneti mező ki van töltve, egyébként false

Definíció a(z) [AddQuestionModel.java](#) fájl 92. sorában.

A függvény hívó gráfja:



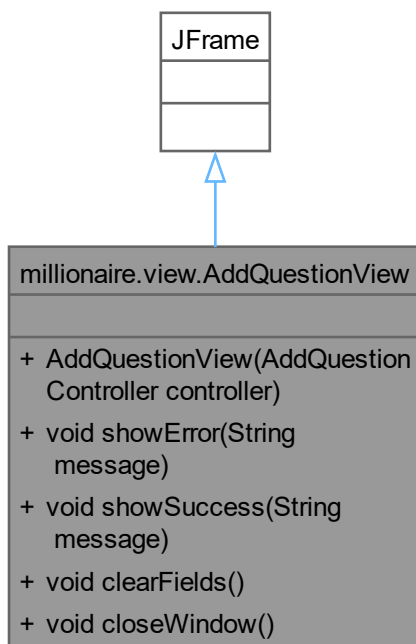
Ez a dokumentáció az osztályról a következő fájl alapján készült:

- [src/main/java/millionaire/model/AddQuestionModel.java](#)

## 6.3. millionaire.view.AddQuestionView osztályreferencia

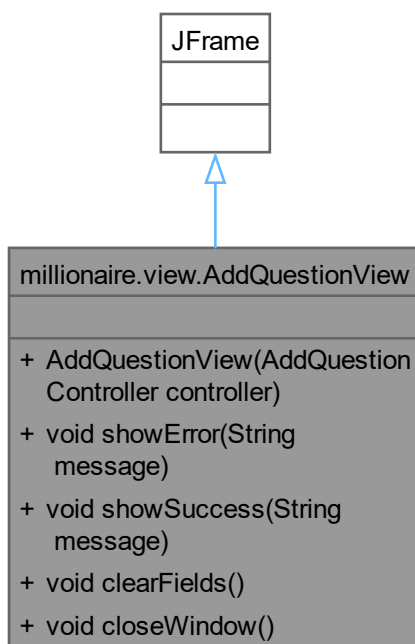
Új kérdések hozzáadására szolgáló grafikus felület.

A millionaire.view.AddQuestionView osztály származási diagramja:





A millionaire.view.AddQuestionView osztály együttműködési diagramja:



#### Publikus tagfüggvények

- `AddQuestionView` (`AddQuestionController` controller)  
*Konstruktor az AddQuestionView létrehozásához.*
- `void showError` (String message)  
*Hibaüzenet megjelenítése.*
- `void showSuccess` (String message)  
*Sikeres művelet visszajelzése.*
- `void clearFields` ()  
*Törli az űrlap mezőinek tartalmát.*
- `void closeWindow` ()  
*Bezárja az ablakot.*

#### 6.3.1. Részletes leírás

Új kérdések hozzáadására szolgáló grafikus felület.

Ez az osztály felelős az új kérdések hozzáadásához szükséges felhasználói felület megjelenítéséért és kezeléséért. A felület tartalmaz:

- Beviteli mezőket a kérdés adatainak megadásához
- Legördülő menüket a kategória és nehézség kiválasztásához
- Gombokat a művelet végrehajtásához vagy megszakításához

Definíció a(z) `AddQuestionView.java` fájl 18. sorában.

## 6.3.2. Konstruktorkok és destruktorkok dokumentációja

### 6.3.2.1. AddQuestionView()

```
millionaire.view.AddQuestionView.AddQuestionView (
    AddQuestionController controller )
```

Konstruktor az AddQuestionView létrehozásához.

Paraméterek

|                   |                                        |
|-------------------|----------------------------------------|
| <i>controller</i> | A kérdés hozzáadását kezelő controller |
|-------------------|----------------------------------------|

Definíció a(z) [AddQuestionView.java](#) fájl 42. sorában.

## 6.3.3. Tagfüggvények dokumentációja

### 6.3.3.1. clearFields()

```
void millionaire.view.AddQuestionView.clearFields ( )
```

Törli az űrlap mezőinek tartalmát.

Alaphelyzetbe állítja a beviteli mezőket és a legördülő menüket

Definíció a(z) [AddQuestionView.java](#) fájl 298. sorában.

A függvény hívó gráfja:



### 6.3.3.2. closeWindow()

```
void millionaire.view.AddQuestionView.closeWindow ( )
```

Bezárja az ablakot.

Felszabadítja az ablakhoz tartozó erőforrásokat és eltávolítja az ablakot a képernyőről.

Definíció a(z) [AddQuestionView.java](#) fájl 315. sorában.

A függvény hívó gráfja:



#### 6.3.3.3. showError()

```
void millionaire.view.AddQuestionView.showError (
    String message )
```

Hibaüzenet megjelenítése.

Felugró ablakban jeleníti meg a hibaüzenetet.

##### Paraméterek

|                |                             |
|----------------|-----------------------------|
| <i>message</i> | A megjelenítendő hibaüzenet |
|----------------|-----------------------------|

Definíció a(z) [AddQuestionView.java](#) fájl 276. sorában.

A függvény hívó gráfja:



#### 6.3.3.4. showSuccess()

```
void millionaire.view.AddQuestionView.showSuccess (
    String message )
```

Sikeres művelet visszajelzése.

Felugró ablakban jeleníti meg a sikeres művelet üzenetét.

##### Paraméterek

|                |                         |
|----------------|-------------------------|
| <i>message</i> | A megjelenítendő üzenet |
|----------------|-------------------------|

Definíció a(z) [AddQuestionView.java](#) fájl 288. sorában.

A függvény hívó gráfja:



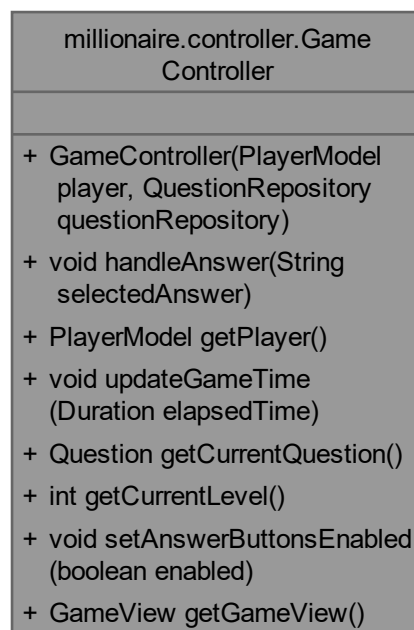
Ez a dokumentáció az osztályról a következő fájl alapján készült:

- `src/main/java/millionaire/view/AddQuestionView.java`

## 6.4. millionaire.controller.GameController osztályreferencia

A játék fő vezérlő osztálya.

A millionaire.controller.GameController osztály együttműködési diagramja:



## Publikus tagfüggvények

- [GameController](#) ([PlayerModel](#) player, [QuestionRepository](#) questionRepository)

*Konstruktor a GameController létrehozásához.*

- void [handleAnswer](#) (String selectedAnswer)

*Kezeli a játékos által választott választ.*

- [PlayerModel](#) [getPlayer](#) ()

*Visszaadja az aktuális játékos modelljét.*

- void [updateGameTime](#) (Duration elapsedTime)

*Frissíti a játékidőt a modellben.*

- [Question](#) [getCurrentQuestion](#) ()

*Visszaadja az aktuális kérdést.*

- int [getCurrentLevel](#) ()

*Visszaadja az aktuális szintet.*

- void [setAnswerButtonsEnabled](#) (boolean enabled)

*Beállítja a válaszgombok engedélyezettségi állapotát.*

- [GameView](#) [getGameView](#) ()

*Visszaadja a játék nézetét.*

### 6.4.1. Részletes leírás

A játék fő vezérlő osztálya.

Ez az osztály felelős a teljes játékmenet vezérléséért:

- Koordinálja a játék állapotának változásait
- Kezeli a játékos válaszait
- Irányítja a segítségék használatát
- Frissíti a játék nézetét
- Kezeli a játék végét és az eredmények mentését

Definíció a(z) [GameController.java](#) fájl 21. sorában.

### 6.4.2. Konstruktorok és destruktorok dokumentációja

#### 6.4.2.1. GameController()

```
millionaire.controller.GameController.GameController (
    PlayerModel player,
    QuestionRepository questionRepository )
```

Konstruktor a GameController létrehozásához.

#### Paraméterek

|                           |                               |
|---------------------------|-------------------------------|
| <i>player</i>             | A játékos modellje            |
| <i>questionRepository</i> | A kérdéseket tároló adatbázis |

Definíció a(z) [GameController.java](#) fájl 33. sorában.

### 6.4.3. Tagfüggvények dokumentációja

#### 6.4.3.1. `getCurrentLevel()`

```
int millionaire.controller.GameController.getCurrentLevel ( )
```

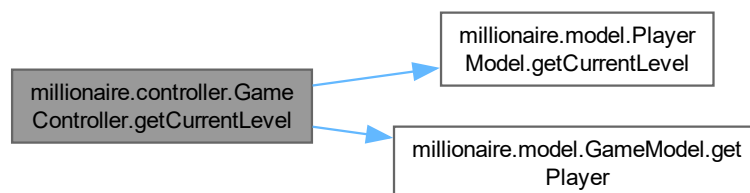
Visszaadja az aktuális szintet.

##### Visszatérési érték

Az aktuális szint

Definíció a(z) [GameController.java](#) fájl 219. sorában.

A függvény hívási gráfja:



#### 6.4.3.2. `getCurrentQuestion()`

```
Question millionaire.controller.GameController.getCurrentQuestion ( )
```

Visszaadja az aktuális kérdést.

##### Visszatérési érték

Az aktuális kérdés

Definíció a(z) [GameController.java](#) fájl 210. sorában.

A függvény hívási gráfja:



#### 6.4.3.3. `getGameView()`

```
GameView millionaire.controller.GameController.getGameView ( )
```

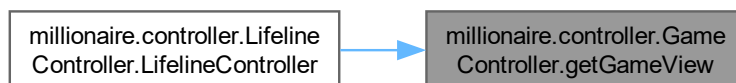
Visszaadja a játék nézetét.

Visszatérési érték

A játék nézete

Definíció a(z) [GameController.java](#) fájl 237. sorában.

A függvény hívó gráfja:



#### 6.4.3.4. `getPlayer()`

```
PlayerModel millionaire.controller.GameController.getPlayer ( )
```

Visszaadja az aktuális játékos modelljét.

Visszatérési érték

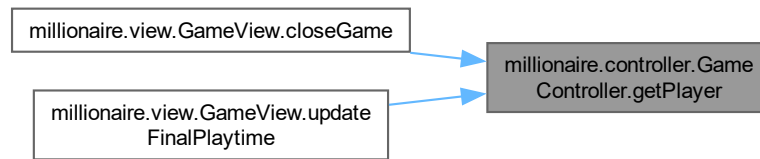
A játékos modellje

Definíció a(z) [GameController.java](#) fájl 181. sorában.

A függvény hívási gráfja:



A függvény hívó gráfja:



#### 6.4.3.5. handleAnswer()

```
void millionaire.controller.GameController.handleAnswer (
    String selectedAnswer )
```

Kezeli a játékos által választott választ.

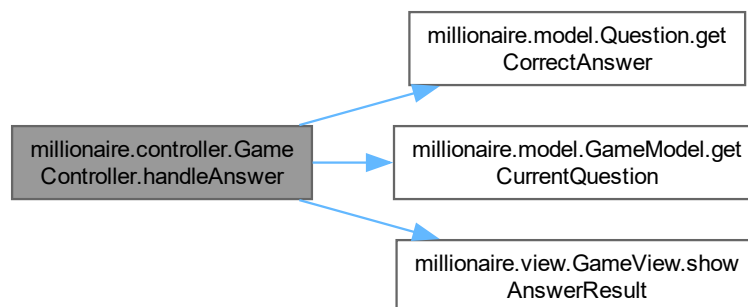
Letiltja a választógombokat a feldolgozás idejére, megjeleníti a válasz helyességét, majd késleltetés után folytatja a játékot vagy befejezi azt.

Paraméterek

|                       |                              |
|-----------------------|------------------------------|
| <i>selectedAnswer</i> | A választott válasz betűjele |
|-----------------------|------------------------------|

Definíció a(z) [GameController.java](#) fájl 61. sorában.

A függvény hívási gráfja:



#### 6.4.3.6. setAnswerButtonsEnabled()

```
void millionaire.controller.GameController.setAnswerButtonsEnabled (
    boolean enabled )
```



Beállítja a választógombok engedélyezettségi állapotát.

#### Paraméterek

|                |                              |
|----------------|------------------------------|
| <i>enabled</i> | Az engedélyezettség állapota |
|----------------|------------------------------|

Definíció a(z) [GameController.java](#) fájl 228. sorában.

A függvény hívási gráfja:



#### 6.4.3.7. updateGameTime()

```
void millionaire.controller.GameController.updateGameTime (
    Duration elapsedTime )
```

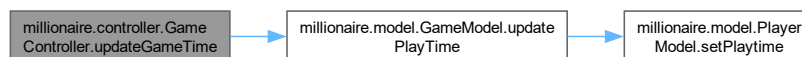
Frissíti a játékidőt a modellben.

#### Paraméterek

|                    |                    |
|--------------------|--------------------|
| <i>elapsedTime</i> | Az eltelt játékidő |
|--------------------|--------------------|

Definíció a(z) [GameController.java](#) fájl 190. sorában.

A függvény hívási gráfja:



Ez a dokumentáció az osztályról a következő fájl alapján készült:

- `src/main/java/millionaire/controller/GameController.java`

## 6.5. millionaire.model.GameModel osztályreferencia

A játék logikáját és állapotát kezelő osztály.

A millionaire.model.GameModel osztály együttműködési diagramja:

| millionaire.model.GameModel                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>+ GameModel(QuestionRepository questionRepository, PlayerModel player)</li> <li>+ Question selectNextQuestion()</li> <li>+ GameState processAnswer (String selectedAnswer)</li> <li>+ long getLastSafePrize()</li> <li>+ long getCurrentPrize()</li> <li>+ PlayerModel getPlayer()</li> <li>+ Question getCurrentQuestion()</li> <li>+ boolean isGameOver()</li> <li>+ void updatePlayTime (Duration elapsedTime)</li> <li>+ boolean isTimeUp(Duration currentTime)</li> <li>+ static List&lt; PrizeLevel &gt; getPrizeLevels()</li> </ul> |

### Publikus tagfüggvények

- [GameModel](#) ([QuestionRepository](#) questionRepository, [PlayerModel](#) player)  
*Konstruktor a játékmódel létrehozásához.*
- [Question](#) selectNextQuestion ()  
*Kiválaszt egy következő kérdést az aktuális szintnek megfelelően.*
- [GameState](#) processAnswer (String selectedAnswer)  
*Feldolgozza a játékos választát.*
- long [getLastSafePrize](#) ()  
*Meghatározza az utolsó elért garantált nyereményt.*
- long [getCurrentPrize](#) ()  
*Visszaadja az aktuális szinthez tartozó nyereményösszeget.*
- [PlayerModel](#) getPlayer ()  
*Visszaadja az aktuális játékost.*
- [Question](#) getCurrentQuestion ()  
*Visszaadja az aktuális kérdést.*
- boolean [isGameOver](#) ()  
*Visszaadja, hogy véget ért-e a játék.*
- void [updatePlayTime](#) (Duration elapsedTime)  
*Frissíti a játékos játékidjét.*
- boolean [isTimeUp](#) (Duration currentTime)  
*Ellenőrzi, hogy a játék időlimitje lejárt-e (ha van)*

### Statikus publikus tagfüggvények

- static List< [PrizeLevel](#) > [getPrizeLevels](#) ()  
*Létrehozza a nyereményszintek listáját.*

#### 6.5.1. Részletes leírás

A játék logikáját és állapotát kezelő osztály.

Ez az osztály felelős a játék fő működéséért, beleértve:

- Kérdések kiválasztását
- Válaszok feldolgozását
- Nyeremények kezelését
- Játékállapot követését

Definíció a(z) [GameModel.java](#) fájl 17. sorában.

#### 6.5.2. Konstruktorok és destruktorok dokumentációja

##### 6.5.2.1. GameModel()

```
millionaire.model.GameModel.GameModel (
    QuestionRepository questionRepository,
    PlayerModel player )
```

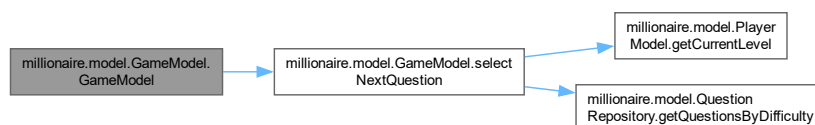
Konstruktor a játékmódel létrehozásához.

##### Paraméterek

|                           |                               |
|---------------------------|-------------------------------|
| <i>questionRepository</i> | A kérdéseket tároló adatbázis |
| <i>player</i>             | Az aktuális játékos modellje  |

Definíció a(z) [GameModel.java](#) fájl 42. sorában.

A függvény hívási gráfja:



### 6.5.3. Tagfüggvények dokumentációja

#### 6.5.3.1. `getCurrentPrize()`

```
long millionaire.model.GameModel.getCurrentPrize ( )
```

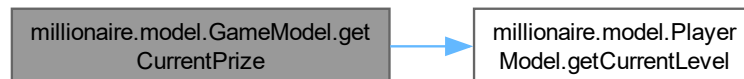
Visszaadja az aktuális szinthez tartozó nyereményösszeget.

Visszatérési érték

Az aktuális nyereményösszeg

Definíció a(z) [GameModel.java](#) fájl 113. sorában.

A függvény hívási gráfja:



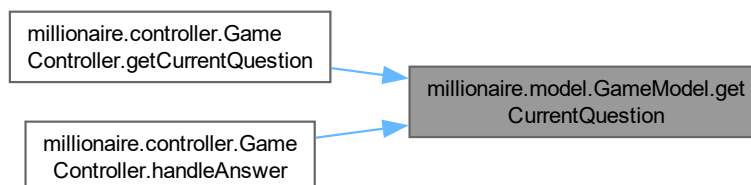
#### 6.5.3.2. `getCurrentQuestion()`

```
Question millionaire.model.GameModel.getCurrentQuestion ( )
```

Visszaadja az aktuális kérdést.

Definíció a(z) [GameModel.java](#) fájl 123. sorában.

A függvény hívó gráfja:



### 6.5.3.3. getLastSafePrize()

```
long millionaire.model.GameModel.getLastSafePrize ( )
```

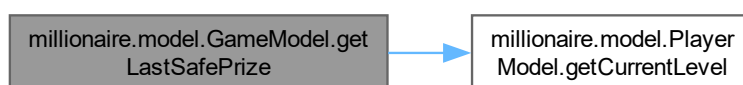
Meghatározza az utolsó elért garantált nyereményt.

Visszatérési érték

Az utolsó garantált nyeremény összege

Definíció a(z) [GameModel.java](#) fájl 99. sorában.

A függvény hívási gráfja:



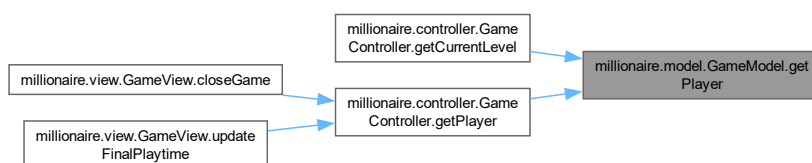
### 6.5.3.4. getPlayer()

```
PlayerModel millionaire.model.GameModel.getPlayer ( )
```

Visszaadja az aktuális játékost.

Definíció a(z) [GameModel.java](#) fájl 119. sorában.

A függvény hívó gráfja:



### 6.5.3.5. getPrizeLevels()

```
static List< PrizeLevel > millionaire.model.GameModel.getPrizeLevels ( ) [static]
```

Létrehozza a nyereményszintek listáját.

Visszatérési érték

Nyereményszintek listája a megfelelő információkkal

Definíció a(z) [GameModel.java](#) fájl 134. sorában.

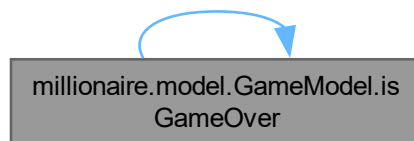
#### 6.5.3.6. isGameOver()

```
boolean millionaire.model.GameModel.isGameOver ( )
```

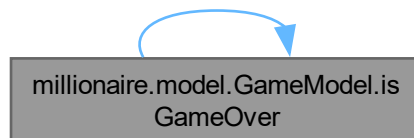
Visszaadja, hogy véget ért-e a játék.

Definíció a(z) [GameModel.java](#) fájl 127. sorában.

A függvény hívási gráfja:



A függvény hívó gráfja:



#### 6.5.3.7. isTimeUp()

```
boolean millionaire.model.GameModel.isTimeUp (
    Duration currentTime )
```

Ellenőrzi, hogy a játék időlimitje lejárt-e (ha van)

##### Paraméterek

|                    |                      |
|--------------------|----------------------|
| <i>currentTime</i> | Az aktuális játékidő |
|--------------------|----------------------|

##### Visszatérési érték

true ha lejárt az idő, false ha nem

Definíció a(z) [GameModel.java](#) fájl 160. sorában.

### 6.5.3.8. processAnswer()

```
GameState millionaire.model.GameModel.processAnswer (
    String selectedAnswer )
```

Feldolgozza a játékos választát.

#### Paraméterek

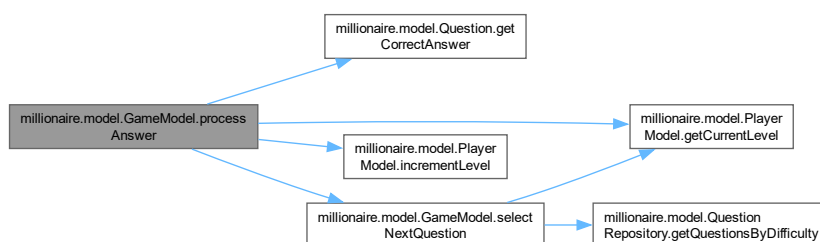
|                       |                                            |
|-----------------------|--------------------------------------------|
| <i>selectedAnswer</i> | A játékos által választott válasz betűjele |
|-----------------------|--------------------------------------------|

#### Visszatérési érték

A játék állapota a válasz feldolgozása után

Definíció a(z) [GameModel.java](#) fájl 72. sorában.

A függvény hívási gráfja:



### 6.5.3.9. selectNextQuestion()

```
Question millionaire.model.GameModel.selectNextQuestion ( )
```

Kiválaszt egy következő kérdést az aktuális szintnek megfelelően.

#### Visszatérési érték

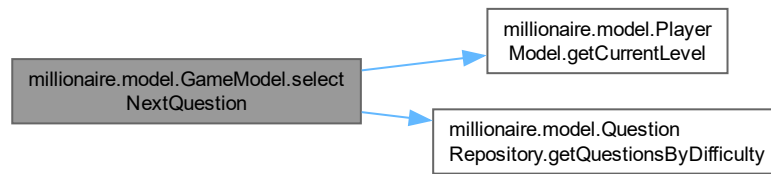
A kiválasztott kérdés

#### Kivételek

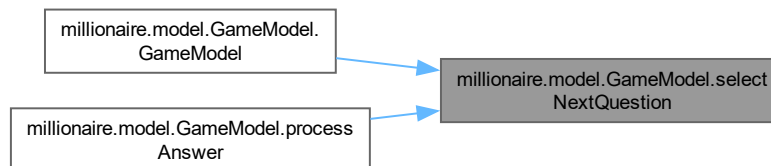
|                              |                                           |
|------------------------------|-------------------------------------------|
| <i>IllegalStateException</i> | Ha nincs elérhető kérdés az adott szinten |
|------------------------------|-------------------------------------------|

Definíció a(z) [GameModel.java](#) fájl 57. sorában.

A függvény hívási gráfja:



A függvény hívó gráfja:



#### 6.5.3.10. updatePlayTime()

```
void millionaire.model.GameModel.updatePlayTime (
    Duration elapsedTime )
```

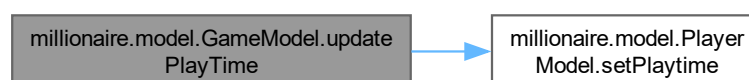
Frissíti a játékos játékidőjét.

Paraméterek

|                    |                    |
|--------------------|--------------------|
| <i>elapsedTime</i> | Az eltelt játékidő |
|--------------------|--------------------|

Definíció a(z) [GameModel.java](#) fájl 150. sorában.

A függvény hívási gráfja:





A függvény hívó gráfja:



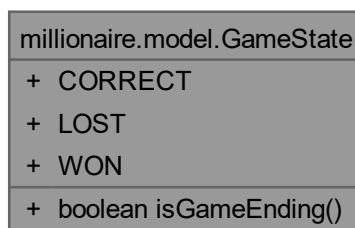
Ez a dokumentáció az osztályról a következő fájl alapján készült:

- `src/main/java/millionaire/model/`[GameModel.java](#)

## 6.6. millionaire.model.GameState felsoroló referencia

Enum osztály, amely a játék állapotait reprezentálja.

A millionaire.model.GameState osztály együttműködési diagramja:



### Publikus tagfüggvények

- boolean [isGameEnding](#) ()

*Megadja, hogy a játék olyan állapotban van-e, amikor véget kell érnie.*

### Publikus attribútumok

- [CORRECT](#)
- [LOST](#)
- [WON](#)

### 6.6.1. Részletes leírás

Enum osztály, amely a játék állapotait reprezentálja.

Definíció a(z) [GameState.java](#) fájl 7. sorában.

### 6.6.2. Tagfüggvények dokumentációja

#### 6.6.2.1. isGameEnding()

```
boolean millionaire.model.GameState.isGameEnding ( )
```

Megadja, hogy a játék olyan állapotban van-e, amikor véget kell érnie.

Visszatérési érték

True, ha a játék véget ért, false egyébként.

Definíció a(z) [GameState.java](#) fájl 30. sorában.

### 6.6.3. Adattagok dokumentációja

#### 6.6.3.1. CORRECT

```
millionaire.model.GameState.CORRECT
```

Ha nem az utolsó kérdésre válaszolt a játékos, és helyes választ adott. A játék folytatódik a következő kérdéssel.

Definíció a(z) [GameState.java](#) fájl 12. sorában.

#### 6.6.3.2. LOST

```
millionaire.model.GameState.LOST
```

Ha a játékos rossz választ adott, vagy az utolsó kérdésre helytelen választ adott. A játék véget ér, a játékos vagy nem nyer semmit, vagy a legutóbb elért biztos nyereményt kapja.

Definíció a(z) [GameState.java](#) fájl 18. sorában.

#### 6.6.3.3. WON

```
millionaire.model.GameState.WON
```

A játékos a 15. kérdésre helyes választ adott, így megnyerte a főnyereményt. A játék véget ér, a játékos megszerezte a főnyereményt.

Definíció a(z) [GameState.java](#) fájl 24. sorában.

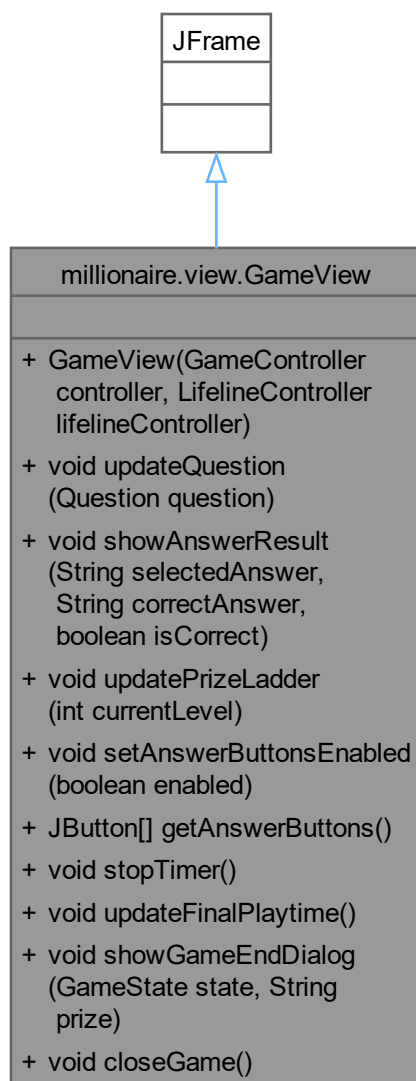
A dokumentáció ehhez az enum-hoz a következő fájl alapján készült:

- `src/main/java/millionaire/model/GameState.java`

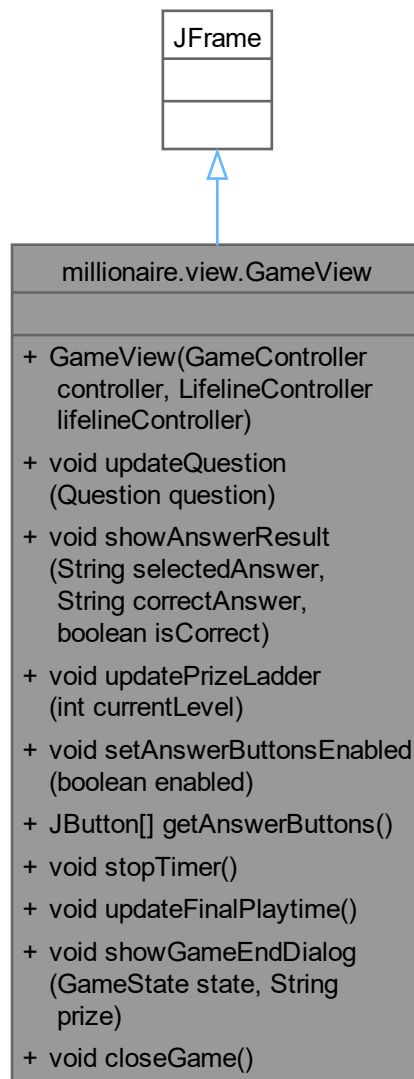
## 6.7. millionaire.view.GameView osztályreferencia

A játék fő grafikus felülete.

A millionaire.view.GameView osztály származási diagramja:



A millionaire.view.GameView osztály együttműködési diagramja:



### Publikus tagfüggvények

- `GameView` (`GameController` controller, `LifelineController` lifelineController)  
*Konstruktor a GameView létrehozásához.*
- void `updateQuestion` (`Question` question)  
*Frissíti a kérdést és a válaszlehetőségeket.*
- void `showAnswerResult` (String selectedAnswer, String correctAnswer, boolean isCorrect)  
*Megjeleníti a választott válasz eredményét.*
- void `updatePrizeLadder` (int currentLevel)  
*Frissíti a nyereménylétrát.*
- void `setAnswerButtonsEnabled` (boolean enabled)

- *Beállítja a választógombok engedélyezettségi állapotát.*  
• JButton[] [getAnswerButtons](#) ()  
*Visszaadja a választógombokat.*
- void [stopTimer](#) ()  
*Leállítja az időmérőt.*
- void [updateFinalPlaytime](#) ()  
*Frissíti a végleges játékidjét a játékosnak.*
- void [showGameEndDialog](#) (GameState state, String prize)  
*Megjeleníti a játék végeredményét.*
- void [closeGame](#) ()  
*Bezárja a játék ablakát.*

### 6.7.1. Részletes leírás

A játék fő grafikus felülete.

Ez az osztály felelős a játék fő képernyőjének megjelenítéséért és kezeléséért. A felület tartalmazza:

- A kérdés megjelenítését
- A négy válaszlehetőség gombjait
- A nyereményletrát
- A segítség ikonjait
- A játékállapot megjelenítését

Definíció a(z) [GameView.java](#) fájl 26. sorában.

### 6.7.2. Konstruktorok és destruktorok dokumentációja

#### 6.7.2.1. GameView()

```
millionaire.view.GameView.GameView (
    GameController controller,
    LifelineController lifelineController )
```

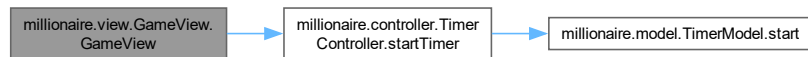
Konstruktor a GameView létrehozásához.

#### Paraméterek

|                           |                                  |
|---------------------------|----------------------------------|
| <i>controller</i>         | A játékot vezérlő controller     |
| <i>lifelineController</i> | A segítségeket kezelő controller |

Definíció a(z) [GameView.java](#) fájl 52. sorában.

A függvény hívási gráfja:



### 6.7.3. Tagfüggvények dokumentációja

#### 6.7.3.1. closeGame()

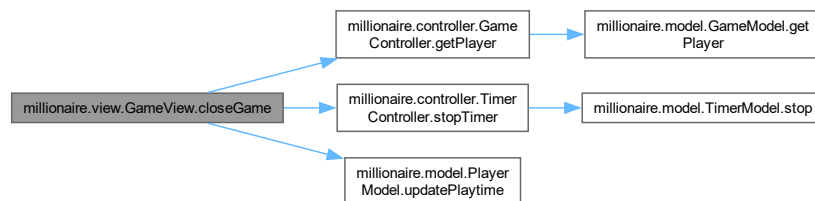
```
void millionaire.view.GameView.closeGame ( )
```

Bezárja a játék ablakát.

Felszabadítja az erőforrásokat és eltávolítja az ablakot a képernyőről.

Definíció a(z) [GameView.java](#) fájl 473. sorában.

A függvény hívási gráfja:



#### 6.7.3.2. getAnswerButtons()

```
JButton[] millionaire.view.GameView.getAnswerButtons ( )
```

Visszaadja a válaszgombokat.

**Visszatérési érték**

A válaszgombok tömbje

Definíció a(z) [GameView.java](#) fájl 425. sorában.

#### 6.7.3.3. setAnswerButtonsEnabled()

```
void millionaire.view.GameView.setAnswerButtonsEnabled (
    boolean enabled )
```

Beállítja a válaszgombok engedélyezettségi állapotát.

## Paraméterek

|                |                              |
|----------------|------------------------------|
| <i>enabled</i> | Az engedélyezettség állapota |
|----------------|------------------------------|

Definíció a(z) [GameView.java](#) fájl 413. sorában.

A függvény hívó gráfja:



## 6.7.3.4. showAnswerResult()

```

void millionaire.view.GameView.showAnswerResult (
    String selectedAnswer,
    String correctAnswer,
    boolean isCorrect )
  
```

Megjeleníti a választott válasz eredményét.

- Letiltja a választógombokat
- Színezi a választott választ (zöld/piros)
- Helytelen válasz esetén megjelöli a helyes választ

## Paraméterek

|                       |                                 |
|-----------------------|---------------------------------|
| <i>selectedAnswer</i> | A választott válasz betűjele    |
| <i>correctAnswer</i>  | A helyes válasz betűjele        |
| <i>isCorrect</i>      | A válasz helyességét jelző flag |

Definíció a(z) [GameView.java](#) fájl 348. sorában.

A függvény hívó gráfja:



#### 6.7.3.5. showGameEndDialog()

```
void millionaire.view.GameView.showGameEndDialog (
    GameState state,
    String prize )
```

Megjeleníti a játék végeredményét.

Felugró ablakban jeleníti meg:

- A játék végeredményét (nyert/vesztett)
- Az elért nyereményt

##### Paraméterek

|              |                     |
|--------------|---------------------|
| <i>state</i> | A játék végállapota |
| <i>prize</i> | Az elért nyeremény  |

Definíció a(z) [GameView.java](#) fájl 456. sorában.

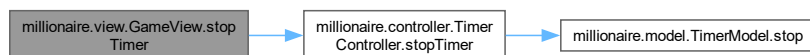
#### 6.7.3.6. stopTimer()

```
void millionaire.view.GameView.stopTimer ( )
```

Leállítja az időmérőt.

Definíció a(z) [GameView.java](#) fájl 433. sorában.

A függvény hívási gráfja:



#### 6.7.3.7. updateFinalPlaytime()

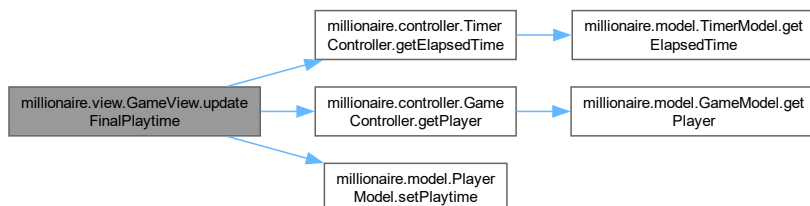
```
void millionaire.view.GameView.updateFinalPlaytime ( )
```

Frissíti a végleges játékidjét a játékosnak.

Definíció a(z) [GameView.java](#) fájl 441. sorában.



A függvény hívási gráfja:



### 6.7.3.8. updatePrizeLadder()

```
void millionaire.view.GameView.updatePrizeLadder (
    int currentLevel )
```

Frissíti a nyeremenylétrát.

Kiemeli az aktuális szintet és frissíti a megjelenítést

Paraméterek

|                           |                         |
|---------------------------|-------------------------|
| <code>currentLevel</code> | Az aktuális szint száma |
|---------------------------|-------------------------|

Definíció a(z) [GameView.java](#) fájl 369. sorában.

### 6.7.3.9. updateQuestion()

```
void millionaire.view.GameView.updateQuestion (
    Question question )
```

Frissíti a kérdést és a válaszlehetőségeket.

Megjeleníti:

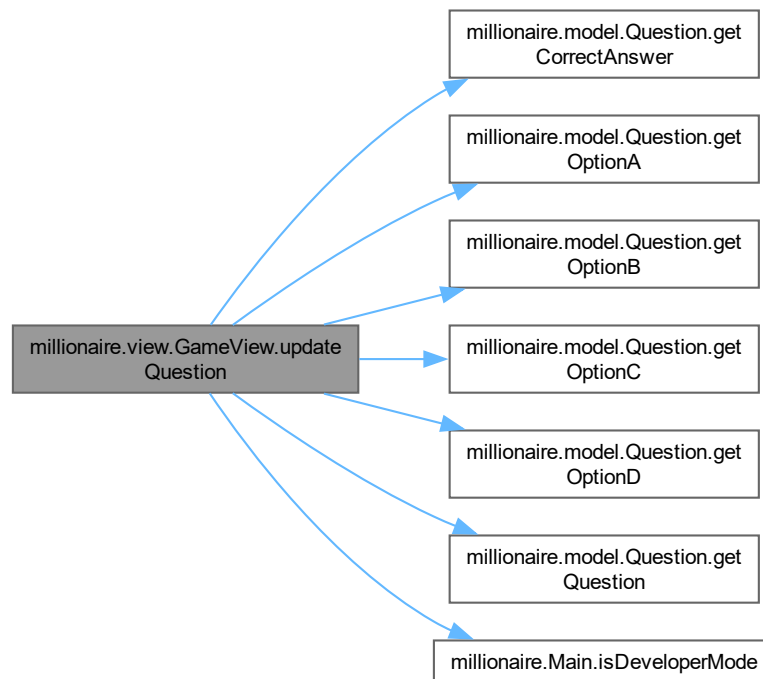
- Az új kérdés szövegét
- Az új válaszlehetőségeket
- Alaphelyzetbe állítja a választógombokat

Paraméterek

|                       |                       |
|-----------------------|-----------------------|
| <code>question</code> | Az új kérdés objektum |
|-----------------------|-----------------------|

Definíció a(z) [GameView.java](#) fájl 321. sorában.

A függvény hívási gráfja:



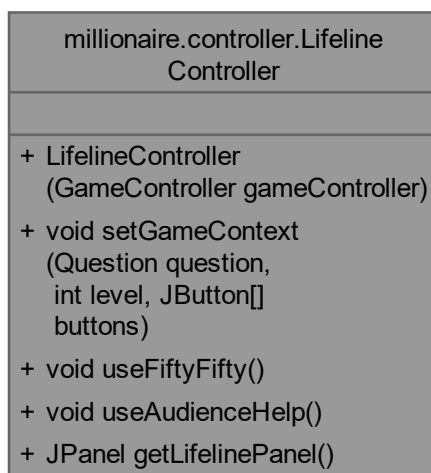
Ez a dokumentáció az osztályról a következő fájl alapján készült:

- `src/main/java/millionaire/view/GameView.java`

## 6.8. millionaire.controller.LifelineController osztályreferencia

A játék segítségét kezelő vezérlő osztály.

A millionaire.controller.LifelineController osztály együttműködési diagramja:



#### Publikus tagfüggvények

- [LifelineController](#) ([GameController](#) gameController)  
*Konstruktor a LifelineController létrehozásához.*
- void [setGameContext](#) ([Question](#) question, int level, JButton[] buttons)  
*Beállítja a játék aktuális kontextusát.*
- void [useFiftyFifty](#) ()  
*A felezés segítség használata.*
- void [useAudienceHelp](#) ()  
*A közönség segítségének használata.*
- JPanel [getLifelinePanel](#) ()  
*Visszaadja a segítségek paneljét.*

#### 6.8.1. Részletes leírás

A játék segítségeit kezelő vezérlő osztály.

Ez az osztály felelős a játék során használható segítségek kezeléséért:

- Felezés segítség használata
- Közönség segítségének használata
- Segítségek állapotának követése
- Segítségek eredményének megjelenítése

Definíció a(z) [LifelineController.java](#) fájl 23. sorában.

## 6.8.2. Konstruktorkok és destruktorkok dokumentációja

### 6.8.2.1. LifelineController()

```
millionaire.controller.LifelineController.LifelineController (
    GameController gameController )
```

Konstruktork a LifelineController létrehozásához.

#### Paraméterek

|                             |                  |
|-----------------------------|------------------|
| <code>gameController</code> | A fő játékvezérő |
|-----------------------------|------------------|

Definíció a(z) [LifelineController.java](#) fájl 36. sorában.

A függvény hívási gráfja:



## 6.8.3. Tagfüggvények dokumentációja

### 6.8.3.1. getLifelinePanel()

```
JPanel millionaire.controller.LifelineController.getLifelinePanel ( )
```

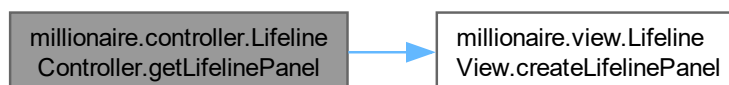
Visszaadja a segítség paneljét.

#### Visszatérési érték

A segítség panelje

Definíció a(z) [LifelineController.java](#) fájl 120. sorában.

A függvény hívási gráfja:



### 6.8.3.2. setGameContext()

```
void millionaire.controller.LifelineController.setGameContext (
    Question question,
    int level,
    JButton[] buttons )
```

Beállítja a játék aktuális kontextusát.

Frissíti:

- Az aktuális kérdést
- Az aktuális szintet
- A válaszgombokat
- A segítség elérhetőségét

#### Paraméterek

|                 |                    |
|-----------------|--------------------|
| <i>question</i> | Az aktuális kérdés |
| <i>level</i>    | Az aktuális szint  |
| <i>buttons</i>  | A válaszgombok     |

Definíció a(z) [LifelineController.java](#) fájl 56. sorában.

### 6.8.3.3. useAudienceHelp()

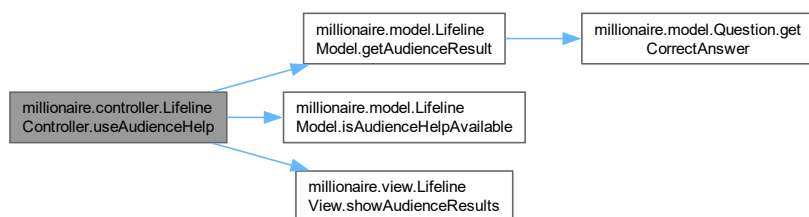
```
void millionaire.controller.LifelineController.useAudienceHelp ( )
```

A közönség segítségének használata.

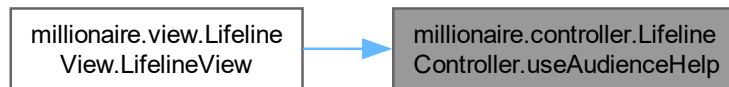
Generál egy szavazási eredményt, megjeleníti az eredményt.

Definíció a(z) [LifelineController.java](#) fájl 88. sorában.

A függvény hívási gráfja:



A függvény hívó gráfja:



#### 6.8.3.4. useFiftyFifty()

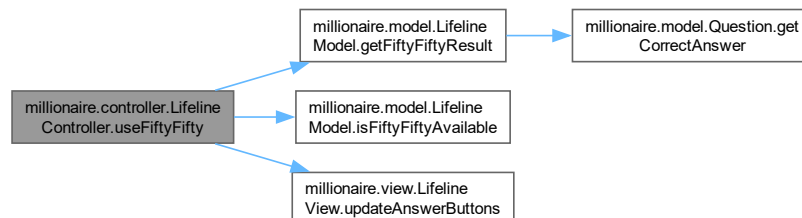
```
void millionaire.controller.LifelineController.useFiftyFifty ( )
```

A felezés segítség használata.

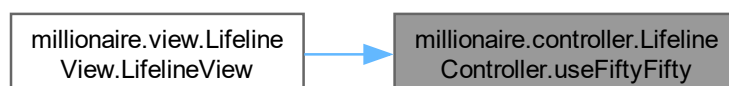
Eltávolít két helytelen választ, frissíti a gombok állapotát.

Definíció a(z) [LifelineController.java](#) fájl 69. sorában.

A függvény hívási gráfja:



A függvény hívó gráfja:



Ez a dokumentáció az osztályról a következő fájl alapján készült:

- `src/main/java/millionaire/controller/LifelineController.java`

## 6.9. millionaire.model.LifelineModel osztályreferencia

Segítségek (lifeline) logikáját kezelő osztály.

A millionaire.model.LifelineModel osztály együttműködési diagramja:

| millionaire.model.Lifeline<br>Model                                                                                                                                                                                                             |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| + Map< String, Boolean<br>> getFiftyFiftyResult<br>(Question question)<br>+ Map< String, Integer<br>> getAudienceResult<br>(Question question, int<br>currentLevel)<br>+ boolean isFiftyFiftyAvailable()<br>+ boolean isAudienceHelpAvailable() |

### Publikus tagfüggvények

- Map< String, Boolean > [getFiftyFiftyResult](#) ([Question](#) question)  
*Felezés segítség végrehajtása.*
- Map< String, Integer > [getAudienceResult](#) ([Question](#) question, int currentLevel)  
*Közönség segítségének szimulálása.*
- boolean [isFiftyFiftyAvailable](#) ()  
*Ellenőrzi, hogy a felezés segítség még használható-e.*
- boolean [isAudienceHelpAvailable](#) ()  
*Ellenőrzi, hogy a közönség segítség még használható-e.*

### 6.9.1. Részletes leírás

Segítségek (lifeline) logikáját kezelő osztály.

Ez az osztály felelős a játékban használható segítségek (felezés, közönség) működésének kezeléséért és a válaszok generálásáért.

Definíció a(z) [LifelineModel.java](#) fájl 12. sorában.

## 6.9.2. Tagfüggvények dokumentációja

### 6.9.2.1. getAudienceResult()

```
Map< String, Integer > millionaire.model.LifelineModel.getAudienceResult (
    Question question,
    int currentLevel )
```

Közönség segítségének szimulálása.

A játék aktuális szintjétől függően generál szavazati arányokat. A magasabb szinteken a közönség kevésbé biztos a helyes válaszban.

#### Paraméterek

|                     |                          |
|---------------------|--------------------------|
| <i>question</i>     | Az aktuális kérdés       |
| <i>currentLevel</i> | A játék aktuális szintje |

#### Visszatérési érték

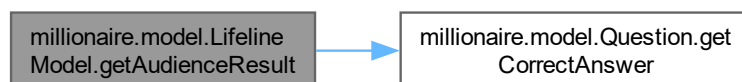
Map, ami tartalmazza a válaszlehetőségekhez tartozó százalékos arányokat

#### Kivételek

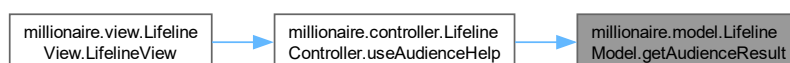
|                              |                                         |
|------------------------------|-----------------------------------------|
| <i>IllegalStateException</i> | Ha a közönség segítséget már használták |
|------------------------------|-----------------------------------------|

Definíció a(z) [LifelineModel.java](#) fájl 63. sorában.

A függvény hívási gráfja:



A függvény hívó gráfja:





### 6.9.2.2. getFiftyFiftyResult()

```
Map< String, Boolean > millionaire.model.LifelineModel.getFiftyFiftyResult (
    Question question )
```

Felezés segítség végrehajtása.

Két rossz választ eltávolít a négy lehetőség közül, meghagyva a helyes választ és egy véletlenszerűen kiválasztott rossz választ.

#### Paraméterek

|                 |                    |
|-----------------|--------------------|
| <i>question</i> | Az aktuális kérdés |
|-----------------|--------------------|

#### Visszatérési érték

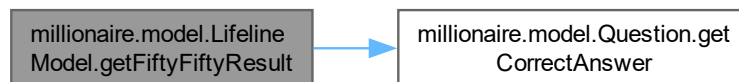
Map, ami tartalmazza minden válaszlehetőséghez, hogy látható-e

#### Kivételek

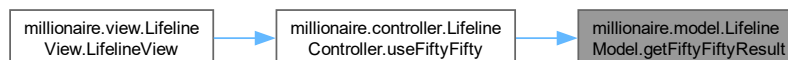
|                              |                                        |
|------------------------------|----------------------------------------|
| <i>IllegalStateException</i> | Ha a felezés segítséget már használták |
|------------------------------|----------------------------------------|

Definíció a(z) [LifelineModel.java](#) fájl 28. sorában.

A függvény hívási gráfja:



A függvény hívó gráfja:



### 6.9.2.3. isAudienceHelpAvailable()

```
boolean millionaire.model.LifelineModel.isAudienceHelpAvailable ( )
```

Ellenőrzi, hogy a közönség segítség még használható-e.

**Visszatérési érték**

true ha a segítség még nem volt használva, egyébként false

Definíció a(z) [LifelineModel.java](#) fájl 142. sorában.

A függvény hívó gráfja:

**6.9.2.4. isFiftyFiftyAvailable()**

```
boolean millionaire.model.LifelineModel.isFiftyFiftyAvailable ( )
```

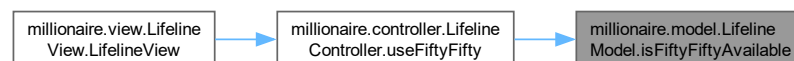
Ellenőrzi, hogy a felezés segítség még használható-e.

**Visszatérési érték**

true ha a segítség még nem volt használva, egyébként false

Definíció a(z) [LifelineModel.java](#) fájl 133. sorában.

A függvény hívó gráfja:



Ez a dokumentáció az osztályról a következő fájl alapján készült:

- `src/main/java/millionaire/model/LifelineModel.java`

## 6.10. millionaire.view.LifelineView osztályreferencia

A játék segítségeit megjelenítő és kezelő felület.

A millionaire.view.LifelineView osztály együttműködési diagramja:

| millionaire.view.LifelineView                                                                                                                                                                                                                                                                                                                                                                                                                        |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>+ LifelineView(JFrame parentFrame, LifelineController controller)</li> <li>+ JPanel createLifelinePanel()</li> <li>+ void updateButtonStates (boolean fiftyFiftyAvailable, boolean audienceHelpAvailable)</li> <li>+ void updateAnswerButtons (JButton[] answerButtons, Map&lt; String, Boolean &gt; visibilityMap)</li> <li>+ void showAudienceResults (Map&lt; String, Integer &gt; percentages)</li> </ul> |

### Publikus tagfüggvények

- [LifelineView](#) (JFrame parentFrame, [LifelineController](#) controller)  
*Konstruktor a LifelineView létrehozásához.*
- JPanel [createLifelinePanel](#) ()  
*Létrehozza a segítségek paneljét.*
- void [updateButtonStates](#) (boolean fiftyFiftyAvailable, boolean audienceHelpAvailable)  
*Frissíti a segítség gombok állapotát.*
- void [updateAnswerButtons](#) (JButton[] answerButtons, Map< String, Boolean > visibilityMap)  
*Frissíti a válaszgombokat a felezés segítség használatakor.*
- void [showAudienceResults](#) (Map< String, Integer > percentages)  
*Megjeleníti a közönség szavazatainak eredményét.*

### 6.10.1. Részletes leírás

A játék segítségeit megjelenítő és kezelő felület.

Ez az osztály felelős a játék során használható segítségek (felezés, közönség) grafikus megjelenítéséért és interakcióinak kezeléséért. A felület tartalmazza:

- A segítségeket aktiváló gombokat
- A segítségek állapotának megjelenítését
- A közönség szavazatainak vizualizációját

Definíció a(z) [LifelineView.java](#) fájl 18. sorában.

## 6.10.2. Konstruktorok és destruktorok dokumentációja

### 6.10.2.1. LifelineView()

```
millionaire.view.LifelineView.LifelineView (
    JFrame parentFrame,
    LifelineController controller )
```

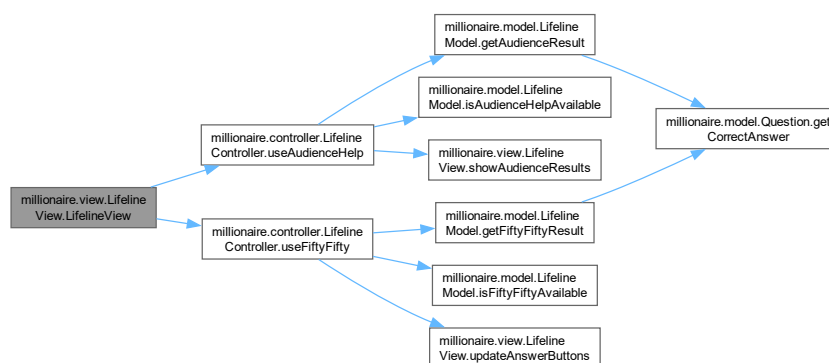
Konstruktor a LifelineView létrehozásához.

#### Paraméterek

|                    |                                  |
|--------------------|----------------------------------|
| <i>parentFrame</i> | A szülő ablak                    |
| <i>controller</i>  | A segítségeket kezelő controller |

Definíció a(z) [LifelineView.java](#) fájl 36. sorában.

A függvény hívási gráfja:



## 6.10.3. Tagfüggvények dokumentációja

### 6.10.3.1. createLifelinePanel()

```
JPanel millionaire.view.LifelineView.createLifelinePanel ( )
```

Létrehozza a segítség paneljét.

Létrehozza a segítség gombokat tartalmazó panelt, megfelelő elrendezéssel és stílusbeállításokkal.

**Visszatérési érték**

A létrehozott panel a segítség gombokkal

Definíció a(z) [LifelineView.java](#) fájl 106. sorában.

A függvény hívó gráfja:

**6.10.3.2. showAudienceResults()**

```
void millionaire.view.LifelineView.showAudienceResults (
    Map< String, Integer > percentages )
```

Megjeleníti a közönség szavazatainak eredményét.

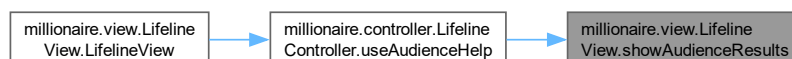
Létrehoz egy dialógusablakot, amely megjeleníti a közönség szavazatainak százalékos megoszlását.

**Paraméterek**

|                    |                                                     |
|--------------------|-----------------------------------------------------|
| <i>percentages</i> | A válaszokra adott szavazatok százalékos megoszlása |
|--------------------|-----------------------------------------------------|

Definíció a(z) [LifelineView.java](#) fájl 163. sorában.

A függvény hívó gráfja:

**6.10.3.3. updateAnswerButtons()**

```
void millionaire.view.LifelineView.updateAnswerButtons (
    JButton[] answerButtons,
    Map< String, Boolean > visibilityMap )
```

Frissíti a válaszgombokat a felezés segítség használatakor.

A felezés segítség használatakor:

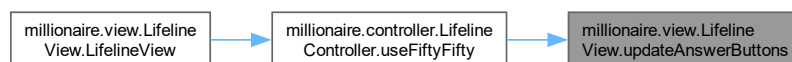
- Letiltja a kiválasztott rossz válaszokat
- Törli a letiltott válaszok szövegét

#### Paraméterek

|                      |                                           |
|----------------------|-------------------------------------------|
| <i>answerButtons</i> | A válasszgombok tömbje                    |
| <i>visibilityMap</i> | A válaszok láthatóságát tartalmazó térkép |

Definíció a(z) [LifelineView.java](#) fájl 145. sorában.

A függvény hívó gráfja:



#### 6.10.3.4. updateButtonStates()

```
void millionaire.view.LifelineView.updateButtonStates (
    boolean fiftyFiftyAvailable,
    boolean audienceHelpAvailable )
```

Frissíti a segítség gombok állapotát.

Frissíti a gombok:

- Engedélyezettségi állapotát
- Színeit A felhasznált segítség gombjai letiltásra kerülnek.

#### Paraméterek

|                              |                                  |
|------------------------------|----------------------------------|
| <i>fiftyFiftyAvailable</i>   | A felezés segítség elérhetősége  |
| <i>audienceHelpAvailable</i> | A közönség segítség elérhetősége |

Definíció a(z) [LifelineView.java](#) fájl 126. sorában.

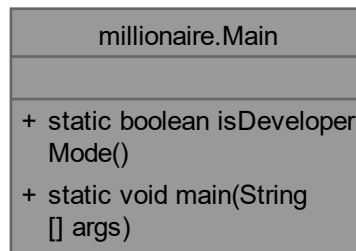
Ez a dokumentáció az osztályról a következő fájl alapján készült:

- src/main/java/millionaire/view/[LifelineView.java](#)

## 6.11. millionaire.Main osztályreferencia

A "Legyen Ön is Milliomos!" kvízzjáték főprogramja.

A millionaire.Main osztály együttműködési diagramja:



### Statikus publikus tagfüggvények

- static boolean `isDeveloperMode()`  
*Visszaadja, hogy a program fejlesztői módban fut-e.*
- static void `main(String[] args)`  
*A program belépési pontja.*

#### 6.11.1. Részletes leírás

A "Legyen Ön is Milliomos!" kvízzjáték főprogramja.

##### Szerző

Zeisz Ottó (HP9G2J)

##### Dátum

2024. november 24.

Ez az osztály a program belépési pontja, amely elindítja a főmenüt és biztosítja, hogy a felhasználói felület az Event Dispatch Thread-en fusson.

Definíció a(z) `Main.java` fájl 15. sorában.

#### 6.11.2. Tagfüggvények dokumentációja

##### 6.11.2.1. `isDeveloperMode()`

```
static boolean millionaire.Main.isDeveloperMode ( ) [static]
```

Visszaadja, hogy a program fejlesztői módban fut-e.

**Visszatérési érték**

true ha fejlesztői módban fut, false ha nem

Definíció a(z) [Main.java](#) fájl 22. sorában.

A függvény hívó gráfja:

**6.11.2.2. main()**

```
static void millionaire.Main.main (
    String[] args ) [static]
```

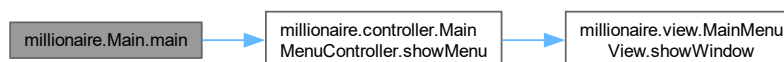
A program belépési pontja.

**Paraméterek**

|             |                                                                              |
|-------------|------------------------------------------------------------------------------|
| <i>args</i> | Parancssori argumentumok: Ha args[0] == "dev", akkor fejlesztői módban indul |
|-------------|------------------------------------------------------------------------------|

Definíció a(z) [Main.java](#) fájl 31. sorában.

A függvény hívási gráfja:



Ez a dokumentáció az osztályról a következő fájl alapján készült:

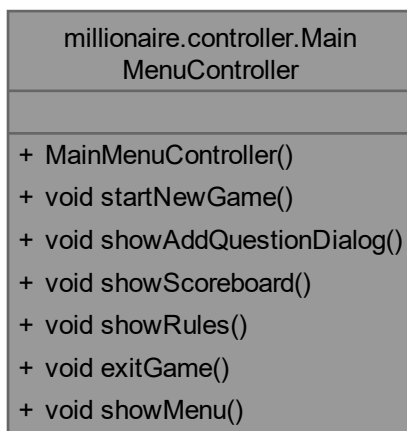
- src/main/java/millionaire/[Main.java](#)

**6.12. millionaire.controller.MainMenuController osztályreferencia**

A főmenü vezérlő osztálya.



A millionaire.controller.MainMenuController osztály együttműködési diagramja:



#### Publikus tagfüggvények

- [MainMenuController](#) ()  
*Konstruktor a MainMenuController létrehozásához.*
- void [startNewGame](#) ()  
*Új játék indítása.*
- void [showAddQuestionDialog](#) ()  
*Megjeleníti a kérdés hozzáadási dialogust.*
- void [showScoreboard](#) ()  
*Megjeleníti a dicsőséglistát.*
- void [showRules](#) ()  
*Megjeleníti a játékszabályokat.*
- void [exitGame](#) ()  
*Kilép a játékból.*
- void [showMenu](#) ()  
*Megjeleníti a főmenüt.*

#### 6.12.1. Részletes leírás

A főmenü vezérlő osztálya.

Ez az osztály felelős a főmenü működésének vezérléséért:

- Új játék indítása
- Kérdés hozzáadási felület megnyitása
- Dicsőséglista megjelenítése
- Játékszabályok megjelenítése
- Játékból való kilépés

Definíció a(z) [MainMenuController.java](#) fájl 18. sorában.

## 6.12.2. Konstruktorkok és destruktorkok dokumentációja

### 6.12.2.1. MainMenuController()

```
millionaire.controller.MainMenuController.MainMenuController ( )
```

Konstruktork a MainMenuController létrehozásához.

Definíció a(z) [MainMenuController.java](#) fájl 25. sorában.

## 6.12.3. Tagfüggvények dokumentációja

### 6.12.3.1. exitGame()

```
void millionaire.controller.MainMenuController.exitGame ( )
```

Kilép a játékból.

Definíció a(z) [MainMenuController.java](#) fájl 79. sorában.

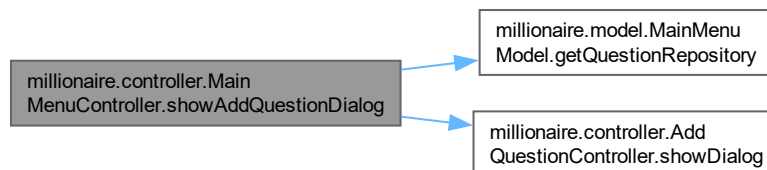
### 6.12.3.2. showAddQuestionDialog()

```
void millionaire.controller.MainMenuController.showAddQuestionDialog ( )
```

Megjeleníti a kérdés hozzáadási dialógust.

Definíció a(z) [MainMenuController.java](#) fájl 51. sorában.

A függvény hívási gráfja:



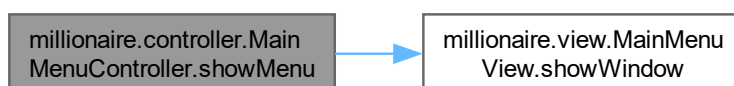
### 6.12.3.3. showMenu()

```
void millionaire.controller.MainMenuController.showMenu ( )
```

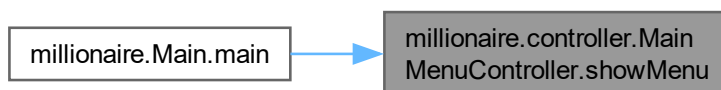
Megjeleníti a főmenüt.

Definíció a(z) [MainMenuController.java](#) fájl 87. sorában.

A függvény hívási gráfja:



A függvény hívó gráfja:



### 6.12.3.4. showRules()

```
void millionaire.controller.MainMenuController.showRules ( )
```

Megjeleníti a játékszabályokat.

Definíció a(z) [MainMenuController.java](#) fájl 70. sorában.

A függvény hívási gráfja:



#### 6.12.3.5. showScoreboard()

```
void millionaire.controller.MainMenuController.showScoreboard ( )
```

Megjeleníti a dicsőséglistát.

Definíció a(z) [MainMenuController.java](#) fájl 61. sorában.

A függvény hívási gráfja:



#### 6.12.3.6. startNewGame()

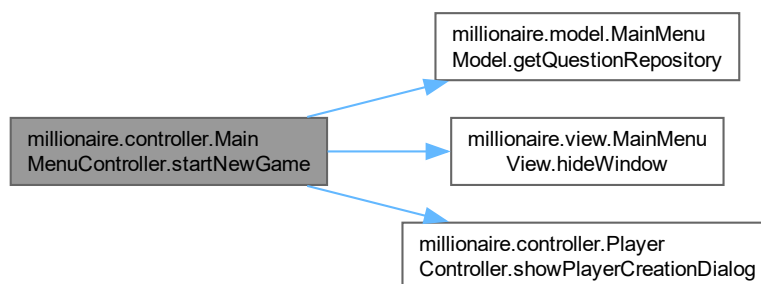
```
void millionaire.controller.MainMenuController.startNewGame ( )
```

Új játék indítása.

- Megjeleníti a játékos létrehozási dialógust
- Sikeres létrehozás esetén elindítja a játékot

Definíció a(z) [MainMenuController.java](#) fájl 37. sorában.

A függvény hívási gráfja:



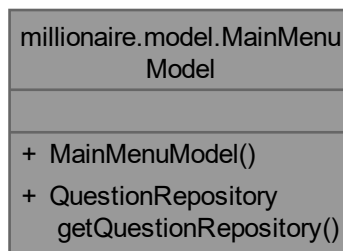
Ez a dokumentáció az osztályról a következő fájl alapján készült:

- `src/main/java/millionaire/controller/MainMenuController.java`

## 6.13. millionaire.model.MainMenuModel osztályreferencia

A főmenü modell osztálya.

A millionaire.model.MainMenuModel osztály együttműködési diagramja:



### Publikus tagfüggvények

- [MainMenuModel \(\)](#)  
*Konstruktor a MainMenuModel létrehozásához.*
- [QuestionRepository getQuestionRepository \(\)](#)  
*Visszaadja a kérdések tárolóját.*

### 6.13.1. Részletes leírás

A főmenü modell osztálya.

Ez az osztály felelős a főmenü adatmodelljének kezeléséért, elsősorban a kérdések tárolójának inicializálásáért és elérhetővé tételéért.

Definíció a(z) [MainMenuModel.java](#) fájl 10. sorában.

### 6.13.2. Konstruktorok és destruktorok dokumentációja

#### 6.13.2.1. MainMenuModel()

```
millionaire.model.MainMenuModel.MainMenuModel ( )
```

Konstruktor a MainMenuModel létrehozásához.

Inicializálja a kérdések tárolóját.

Definíció a(z) [MainMenuModel.java](#) fájl 18. sorában.

### 6.13.3. Tagfüggvények dokumentációja

#### 6.13.3.1. `getRepository()`

```
QuestionRepository millionaire.model.MainMenuModel.getRepository ( )
```

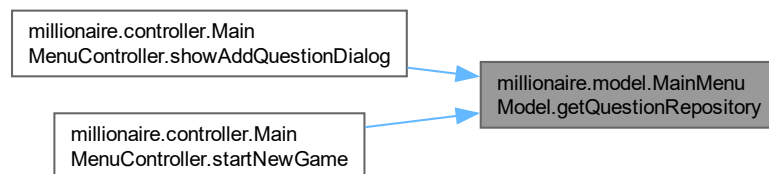
Visszaadja a kérdések tárolóját.

##### Visszatérési érték

A kérdéseket tároló `QuestionRepository` példány

Definíció a(z) `MainMenuModel.java` fájl 26. sorában.

A függvény hívó gráfja:



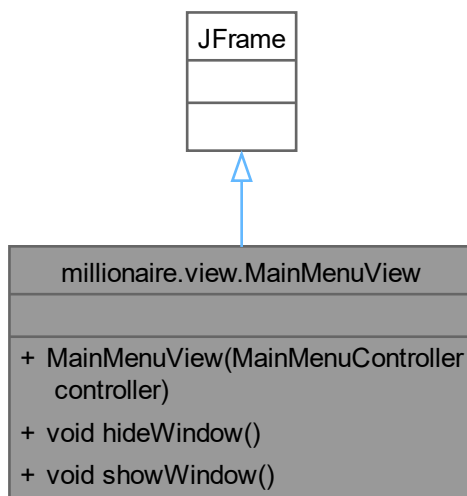
Ez a dokumentáció az osztályról a következő fájl alapján készült:

- `src/main/java/millionaire/model/MainMenuModel.java`

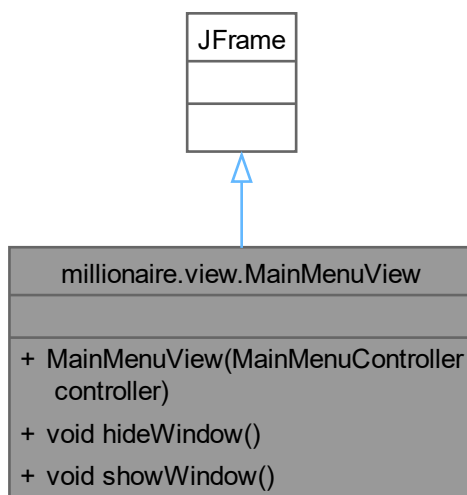
## 6.14. `millionaire.view.MainMenuView` osztályreferencia

A játék főmenüjének grafikus felülete.

A millionaire.view.MainMenuView osztály származási diagramja:



A millionaire.view.MainMenuView osztály együttműködési diagramja:



#### Publikus tagfüggvények

- [MainMenuView](#) ([MainMenuController](#) controller)  
*Konstruktor a MainMenuView létrehozásához.*

- void `hideWindow` ()  
*Elrejt a főmenü ablakát.*
- void `showWindow` ()  
*Megjeleníti a főmenü ablakát.*

### 6.14.1. Részletes leírás

A játék főmenüjének grafikus felülete.

Ez az osztály felelős a játék főmenüjének megjelenítéséért és kezeléséért. A főmenü tartalmazza:

- A játék címét
- Az összes főmenü opciót (új játék, kérdés hozzáadása, dicsőséglista, kézikönyv, kilépés)
- Az opciók közötti navigáció kezelését

Definíció a(z) `MainMenuView.java` fájl 17. sorában.

### 6.14.2. Konstruktork és destruktork dokumentációja

#### 6.14.2.1. MainMenuView()

```
millionaire.view.MainMenuView.MainMenuView (  
    MainMenuController controller )
```

Konstruktork a MainMenuView létrehozásához.

Inicializálja az ablakot és beállítja annak komponenseit.

#### Paraméterek

|                         |                              |
|-------------------------|------------------------------|
| <code>controller</code> | A főmenüt vezérlő controller |
|-------------------------|------------------------------|

Definíció a(z) `MainMenuView.java` fájl 36. sorában.

### 6.14.3. Tagfüggvények dokumentációja

#### 6.14.3.1. hideWindow()

```
void millionaire.view.MainMenuView.hideWindow ( )
```

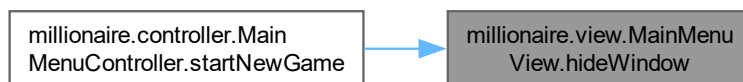
Elrejt a főmenü ablakát.

Az ablakot láthatatlanná teszi, de nem zárja be. Új játék indításakor használatos.

Definíció a(z) `MainMenuView.java` fájl 238. sorában.



A függvény hívó gráfja:



#### 6.14.3.2. showWindow()

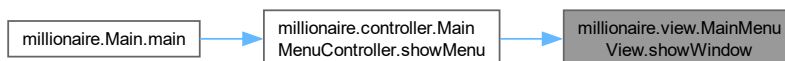
```
void millionaire.view.MainMenuView.showWindow ( )
```

Megjeleníti a főmenü ablakát.

Az ablakot láthatóvá teszi. A játékból való visszatéréskor használatos.

Definíció a(z) [MainMenuView.java](#) fájl 248. sorában.

A függvény hívó gráfja:



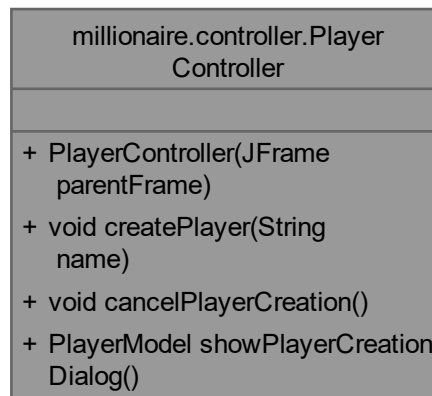
Ez a dokumentáció az osztályról a következő fájl alapján készült:

- `src/main/java/millionaire/view/MainMenuView.java`

## 6.15. millionaire.controller.PlayerController osztályreferencia

A játékos létrehozását kezelő vezérlő osztály.

A `millionaire.controller.PlayerController` osztály együttműködési diagramja:



#### Publikus tagfüggvények

- `PlayerController` (`JFrame parentFrame`)  
*Konstruktor a `PlayerController` létrehozásához.*
- `void createPlayer` (`String name`)  
*Létrehoz egy új játékost.*
- `void cancelPlayerCreation` ()  
*Megszakítja a játékos létrehozását.*
- `PlayerModel showPlayerCreationDialog` ()  
*Megjeleníti a játékos létrehozási dialógust.*

#### 6.15.1. Részletes leírás

A játékos létrehozását kezelő vezérlő osztály.

Ez az osztály felelős:

- A játékos létrehozási folyamat vezérléséért
- A játékos nevének validálásáért
- A játékos objektum létrehozásáért

Definíció a(z) `PlayerController.java` fájl 17. sorában.

#### 6.15.2. Konstruktorok és destruktorok dokumentációja

##### 6.15.2.1. `PlayerController()`

```
millionaire.controller.PlayerController.PlayerController (
    JFrame parentFrame )
```

Konstruktor a `PlayerController` létrehozásához.

## Paraméterek

|                    |               |
|--------------------|---------------|
| <i>parentFrame</i> | A szülő ablak |
|--------------------|---------------|

Definíció a(z) [PlayerController.java](#) fájl 27. sorában.

### 6.15.3. Tagfüggvények dokumentációja

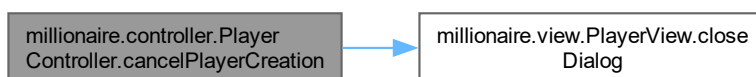
#### 6.15.3.1. cancelPlayerCreation()

```
void millionaire.controller.PlayerController.cancelPlayerCreation ( )
```

Megszakítja a játékos létrehozását.

Definíció a(z) [PlayerController.java](#) fájl 55. sorában.

A függvény hívási gráfja:



#### 6.15.3.2. createPlayer()

```
void millionaire.controller.PlayerController.createPlayer (
    String name )
```

Létrehoz egy új játékost.

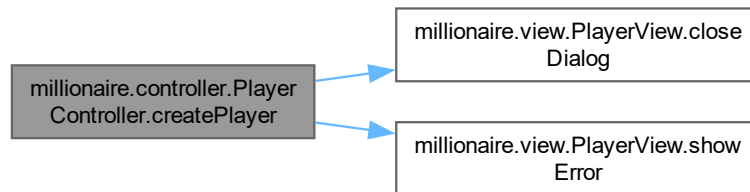
Ellenőrzi a név érvényességét, létrehozza a játékos objektumot.

## Paraméterek

|             |                |
|-------------|----------------|
| <i>name</i> | A játékos neve |
|-------------|----------------|

Definíció a(z) [PlayerController.java](#) fájl 40. sorában.

A függvény hívási gráfja:



#### 6.15.3.3. showPlayerCreationDialog()

```
PlayerModel millionaire.controller.PlayerController.showPlayerCreationDialog ( )
```

Megjeleníti a játékos létrehozási dialógust.

##### Visszatérési érték

A létrehozott játékos, vagy null ha megszakították

Definíció a(z) [PlayerController.java](#) fájl 65. sorában.

A függvény hívó gráfja:

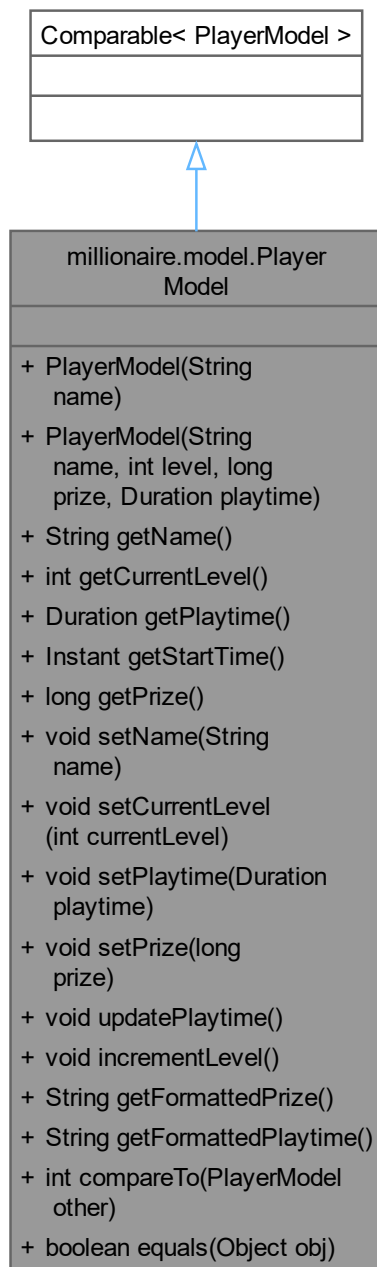


Ez a dokumentáció az osztályról a következő fájl alapján készült:

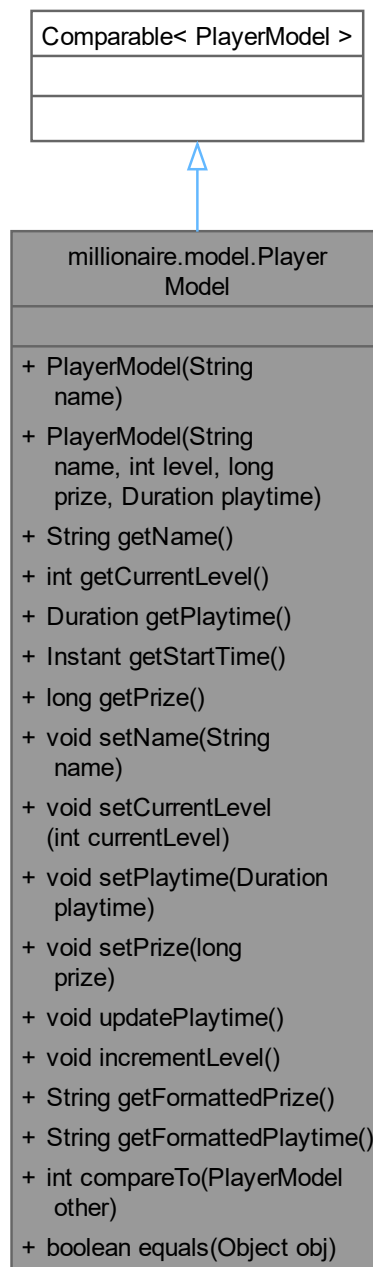
- [src/main/java/millionaire/controller/PlayerController.java](#)

## 6.16. millionaire.model.PlayerModel osztályreferencia

A millionaire.model.PlayerModel osztály származási diagramja:



A millionaire.model.PlayerModel osztály együttműködési diagramja:



### Publikus tagfüggvények

- `PlayerModel` (String name)  
*Konstruktor új játékos létrehozásához.*
- `PlayerModel` (String name, int level, long prize, Duration playtime)  
*Konstruktor meglévő játékos adatainak betöltéséhez.*
- String `getName` ()

- Visszaadja az aktuális játékos nevét.*
- int [getCurrentLevel](#) ()
- Visszaadja az aktuális játékos jelenlegi szintjét (1-15).*
- Duration [getPlaytime](#) ()
- Visszaadja az aktuális játékos játékidőjét.*
- Instant [getStartTime](#) ()
- Visszaadja az aktuális játékos játékának kezdetének időpontját.*
- long [getPrize](#) ()
- Visszaadja az aktuális játékos nyereményét.*
- void [setName](#) (String name)
- Beállítja az aktuális játékos nevét.*
- void [setCurrentLevel](#) (int currentLevel)
- Beállítja az aktuális játékos jelenlegi szintjét.*
- void [setPlaytime](#) (Duration playtime)
- Beállítja az aktuális játékos játékidőjét.*
- void [setPrize](#) (long prize)
- Beállítja az aktuális játékos nyereményét.*
- void [updatePlaytime](#) ()
- Játékidő frissítése a kezdési idő alapján.*
- void [incrementLevel](#) ()
- Növeli az aktuális játékos szintjét eggyel, ha helyesen válaszol egy kérdésre.*
- String [getFormattedPrize](#) ()
- Visszaadja az aktuális játékos nyereményét formázva.*
- String [getFormattedPlaytime](#) ()
- Játékidő formázott megjelenítése.*
- int [compareTo](#) (PlayerModel other)
- Játékosok összehasonlítása először a szintjük, majd a játékidőjük alapján, ha a szintjük megegyezik.*
- boolean [equals](#) (Object obj)
- Két játékos egyenlőségének vizsgálata a nevük alapján.*

### 6.16.1. Részletes leírás

Definíció a(z) [PlayerModel.java](#) fájl 9. sorában.

### 6.16.2. Konstruktorkok és destruktorkok dokumentációja

#### 6.16.2.1. PlayerModel() [1/2]

```
millionaire.model.PlayerModel.PlayerModel (
    String name )
```

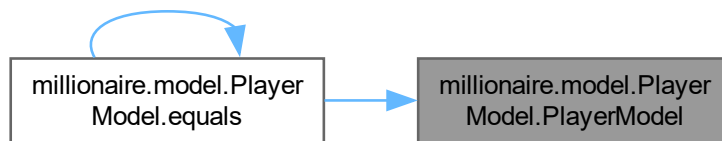
Konstruktork új játékos létrehozásához.

Paraméterek

|             |                |
|-------------|----------------|
| <i>name</i> | A játékos neve |
|-------------|----------------|

Definíció a(z) [PlayerModel.java](#) fájl 21. sorában.

A függvény hívó gráfja:



#### 6.16.2.2. PlayerModel() [2/2]

```

millionaire.model.PlayerModel.PlayerModel (
    String name,
    int level,
    long prize,
    Duration playtime )
  
```

Konstruktor meglévő játékos adatainak betöltéséhez.

##### Paraméterek

|                 |                      |
|-----------------|----------------------|
| <i>name</i>     | A játékos neve       |
| <i>level</i>    | A játékos szintje    |
| <i>prize</i>    | A játékos nyereménye |
| <i>playtime</i> | A játékos játékidője |

Definíció a(z) [PlayerModel.java](#) fájl 37. sorában.

### 6.16.3. Tagfüggvények dokumentációja

#### 6.16.3.1. compareTo()

```

int millionaire.model.PlayerModel.compareTo (
    PlayerModel other )
  
```

Játékosok összehasonlítása először a szintjük, majd a játékidőjük alapján, ha a szintjük megegyezik.

##### Paraméterek

|              |                                                                |
|--------------|----------------------------------------------------------------|
| <i>other</i> | A másik játékos, amivel összehasonlítjuk az aktuális játékost. |
|--------------|----------------------------------------------------------------|



**Visszatérési érték**

Az összehasonlítás eredménye.

Definíció a(z) [PlayerModel.java](#) fájl 155. sorában.

**6.16.3.2. equals()**

```
boolean millionaire.model.PlayerModel.equals (
    Object obj )
```

Két játékos egyenlőségének vizsgálata a nevük alapján.

**Paraméterek**

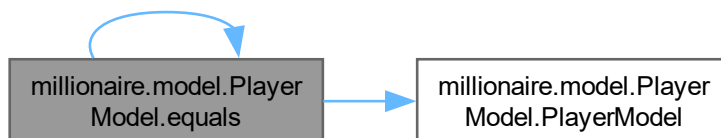
|            |                                                                |
|------------|----------------------------------------------------------------|
| <i>obj</i> | A másik játékos, amivel összehasonlítjuk az aktuális játékost. |
|------------|----------------------------------------------------------------|

**Visszatérési érték**

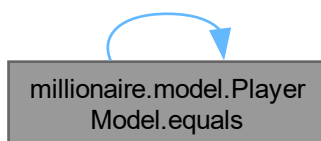
Az egyenlőség eredménye.

Definíció a(z) [PlayerModel.java](#) fájl 171. sorában.

A függvény hívási gráfja:



A függvény hívó gráfja:



### 6.16.3.3. getCurrentLevel()

```
int millionaire.model.PlayerModel.getCurrentLevel ( )
```

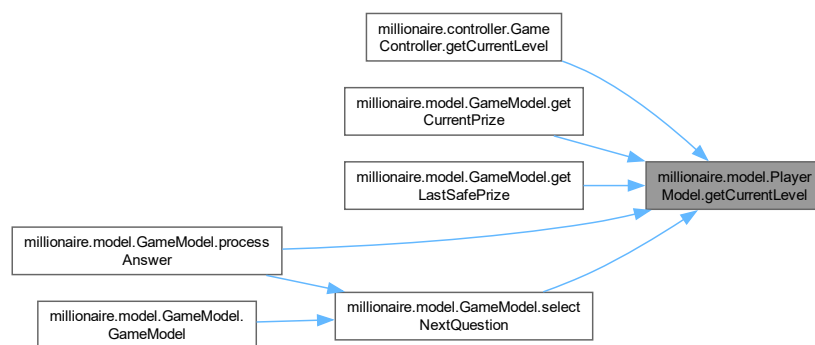
Visszaadja az aktuális játékos jelenlegi szintjét (1-15).

#### Visszatérési érték

A játékos jelenlegi szintje.

Definíció a(z) [PlayerModel.java](#) fájl 57. sorában.

A függvény hívó gráfja:



### 6.16.3.4. getFormattedPlaytime()

```
String millionaire.model.PlayerModel.getFormattedPlaytime ( )
```

Játékidő formázott megjelenítése.

#### Visszatérési érték

A játékidő "perc:másodperc" formátumban

Definíció a(z) [PlayerModel.java](#) fájl 138. sorában.

### 6.16.3.5. getFormattedPrize()

```
String millionaire.model.PlayerModel.getFormattedPrize ( )
```

Visszaadja az aktuális játékos nyereményét formázva.

#### Visszatérési érték

A játékos nyereménye formázva.

Definíció a(z) [PlayerModel.java](#) fájl 129. sorában.

### 6.16.3.6. getName()

```
String millionaire.model.PlayerModel.getName ( )
```

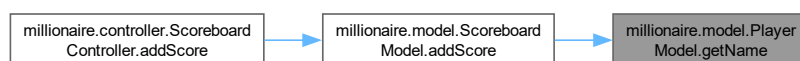
Visszaadja az aktuális játékos nevét.

#### Visszatérési érték

A játékos neve.

Definíció a(z) [PlayerModel.java](#) fájl 50. sorában.

A függvény hívó gráfja:



### 6.16.3.7. getPlaytime()

```
Duration millionaire.model.PlayerModel.getPlaytime ( )
```

Visszaadja az aktuális játékos játékidéjét.

#### Visszatérési érték

A játékos játékidéje.

Definíció a(z) [PlayerModel.java](#) fájl 64. sorában.

### 6.16.3.8. getPrize()

```
long millionaire.model.PlayerModel.getPrize ( )
```

Visszaadja az aktuális játékos nyereményét.

#### Visszatérési érték

A játékos nyereménye.

Definíció a(z) [PlayerModel.java](#) fájl 78. sorában.

#### 6.16.3.9. `getStartTime()`

```
Instant millionaire.model.PlayerModel.getStartTime ( )
```

Visszaadja az aktuális játékos játékának kezdetének időpontját.

##### Visszatérési érték

A játék kezdetének időpontja.

Definíció a(z) [PlayerModel.java](#) fájl 71. sorában.

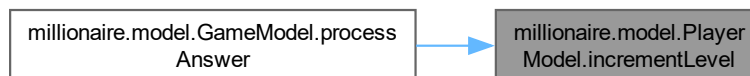
#### 6.16.3.10. `incrementLevel()`

```
void millionaire.model.PlayerModel.incrementLevel ( )
```

Növeli az aktuális játékos szintjét eggyel, ha helyesen válaszol egy kérdésre.

Definíció a(z) [PlayerModel.java](#) fájl 122. sorában.

A függvény hívó gráfja:



#### 6.16.3.11. `setCurrentLevel()`

```
void millionaire.model.PlayerModel.setCurrentLevel (
    int currentLevel )
```

Beállítja az aktuális játékos jelenlegi szintjét.

##### Paraméterek

|                     |                              |
|---------------------|------------------------------|
| <i>currentLevel</i> | A játékos jelenlegi szintje. |
|---------------------|------------------------------|

Definíció a(z) [PlayerModel.java](#) fájl 92. sorában.

#### 6.16.3.12. `setName()`

```
void millionaire.model.PlayerModel.setName (
    String name )
```

Beállítja az aktuális játékos nevét.

**Visszatérési érték**

A játékos neve.

Definíció a(z) [PlayerModel.java](#) fájl 85. sorában.

**6.16.3.13. setPlaytime()**

```
void millionaire.model.PlayerModel.setPlaytime (
    Duration playtime )
```

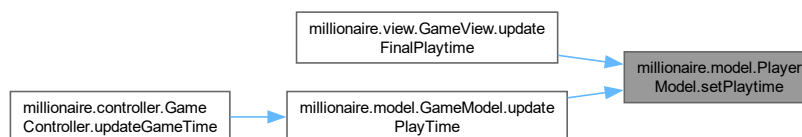
Beállítja az aktuális játékos játékidőjét.

**Paraméterek**

|                 |                       |
|-----------------|-----------------------|
| <i>playtime</i> | A játékos játékidője. |
|-----------------|-----------------------|

Definíció a(z) [PlayerModel.java](#) fájl 99. sorában.

A függvény hívó gráfja:

**6.16.3.14. setPrize()**

```
void millionaire.model.PlayerModel.setPrize (
    long prize )
```

Beállítja az aktuális játékos nyereményét.

**Paraméterek**

|              |                       |
|--------------|-----------------------|
| <i>prize</i> | A játékos nyereménye. |
|--------------|-----------------------|

Definíció a(z) [PlayerModel.java](#) fájl 105. sorában.

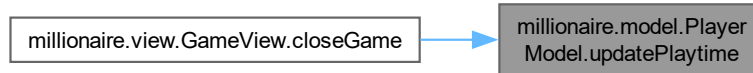
**6.16.3.15. updatePlaytime()**

```
void millionaire.model.PlayerModel.updatePlaytime ( )
```

Játékidő frissítése a kezdési idő alapján.

Definíció a(z) [PlayerModel.java](#) fájl 111. sorában.

A függvény hívó gráfja:



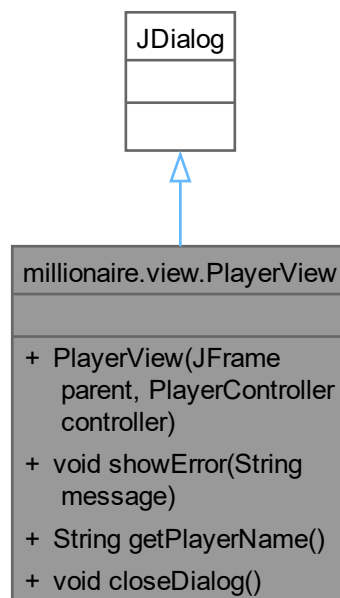
Ez a dokumentáció az osztályról a következő fájl alapján készült:

- [src/main/java/millionaire/model/PlayerModel.java](#)

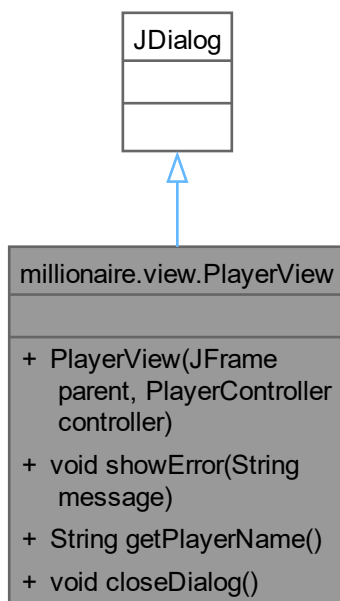
## 6.17. millionaire.view.PlayerView osztályreferencia

Játékos létrehozására szolgáló dialógusablak.

A millionaire.view.PlayerView osztály származási diagramja:



A millionaire.view.PlayerView osztály együttműködési diagramja:



### Publikus tagfüggvények

- `PlayerView` (`JFrame` parent, `PlayerController` controller)  
*Konstruktor a `PlayerView` létrehozásához.*
- `void showError` (`String` message)  
*Hibaüzenet megjelenítése.*
- `String getPlayerName` ()  
*A játékos nevének lekérdezése.*
- `void closeDialog` ()  
*Bezárja a dialógusablakot.*

#### 6.17.1. Részletes leírás

Játékos létrehozására szolgáló dialógusablak.

Ez az osztály felelős az új játékos létrehozásához szükséges felhasználói felület megjelenítéséért és kezeléséért. A felület tartalmazza:

- Beviteli mezőt a játékos nevének megadásához
- OK és Mégse gombokat a művelet végrehajtásához vagy megszakításához

Definíció a(z) `PlayerView.java` fájl 16. sorában.

## 6.17.2. Konstruktorkok és destruktorkok dokumentációja

### 6.17.2.1. `PlayerView()`

```
millionaire.view.PlayerView.PlayerView (
    JFrame parent,
    PlayerController controller )
```

Konstruktor a `PlayerView` létrehozásához.

Létrehoz egy modális dialógusablakot a játékos nevének bekérésére.

#### Paraméterek

|                   |                                          |
|-------------------|------------------------------------------|
| <i>parent</i>     | A szülő ablak                            |
| <i>controller</i> | A játékos létrehozását kezelő controller |

Definíció a(z) [PlayerView.java](#) fájl 35. sorában.

## 6.17.3. Tagfüggvények dokumentációja

### 6.17.3.1. `closeDialog()`

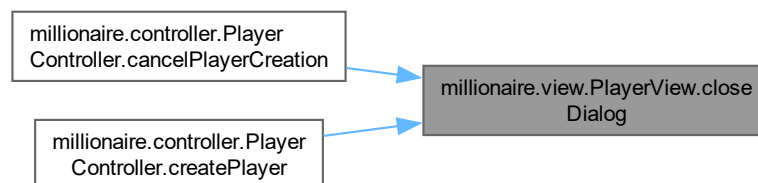
```
void millionaire.view.PlayerView.closeDialog ( )
```

Bezárja a dialógusablakot.

Felszabadítja az ablakhoz tartozó erőforrásokat és eltávolítja az ablakot a képernyőről.

Definíció a(z) [PlayerView.java](#) fájl 137. sorában.

A függvény hívó gráfja:





### 6.17.3.2. getPlayerName()

```
String millionaire.view.PlayerView.getPlayerName ( )
```

A játékos nevének lekérdezése.

Visszaadja a beviteli mezőbe írt nevet, eltávolítva az esetleges felesleges szóközöket.

#### Visszatérési érték

A megadott játékosnév

Definíció a(z) [PlayerView.java](#) fájl 127. sorában.

### 6.17.3.3. showError()

```
void millionaire.view.PlayerView.showError (
    String message )
```

Hibaüzenet megjelenítése.

Felugró ablakban jeleníti meg a hibaüzenetet.

#### Paraméterek

|                |                             |
|----------------|-----------------------------|
| <i>message</i> | A megjelenítendő hibaüzenet |
|----------------|-----------------------------|

Definíció a(z) [PlayerView.java](#) fájl 115. sorában.

A függvény hívó gráfja:



Ez a dokumentáció az osztályról a következő fájl alapján készült:

- `src/main/java/millionaire/view/PlayerView.java`

## 6.18. millionaire.model.PrizeLevel osztályreferencia

Nyereményszintet reprezentáló osztály.

A `millionaire.model.PrizeLevel` osztály együttműködési diagramja:

| millionaire.model.PrizeLevel                                                                                                                                                                                                                       |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>+ <code>PrizeLevel(int level, long prize, boolean isSafeSpot)</code></li> <li>+ <code>int getLevel()</code></li> <li>+ <code>long getPrize()</code></li> <li>+ <code>boolean isSafeSpot()</code></li> </ul> |

#### Publikus tagfüggvények

- [PrizeLevel](#) (int level, long prize, boolean isSafeSpot)  
*Konstruktor egy nyereményszint létrehozásához.*
- int [getLevel](#) ()  
*Megadja a játékos által elért szintet.*
- long [getPrize](#) ()  
*Megadja a játékos által elért nyereményt.*
- boolean [isSafeSpot](#) ()  
*Megadja, hogy a játékos által elért szint szakaszhatároló-e.*

### 6.18.1. Részletes leírás

Nyereményszintet reprezentáló osztály.

Ez az osztály egy nyereményszintet reprezentál a játékban. Minden szinthez tartozik:

- egy sorszám (1-15)
- egy nyereményösszeg
- egy jelző, hogy szakaszhatároló-e (garantált nyereménnyel járó szint)

Definíció a(z) [PrizeLevel.java](#) fájl 13. sorában.

### 6.18.2. Konstruktorok és destruktorok dokumentációja

#### 6.18.2.1. PrizeLevel()

```
millionaire.model.PrizeLevel.PrizeLevel (
    int level,
    long prize,
    boolean isSafeSpot )
```

Konstruktor egy nyereményszint létrehozásához.

## Paraméterek

|                   |                                                                |
|-------------------|----------------------------------------------------------------|
| <i>level</i>      | A szint sorszáma (1-15)                                        |
| <i>prize</i>      | A szinthez tartozó nyereményösszeg                             |
| <i>isSafeSpot</i> | Igaz, ha ez egy szakaszhatároló szint (garantált nyereménnyel) |

Definíció a(z) [PrizeLevel.java](#) fájl 25. sorában.

### 6.18.3. Tagfüggvények dokumentációja

#### 6.18.3.1. `getLevel()`

```
int millionaire.model.PrizeLevel.getLevel ( )
```

Megadja a játékos által elért szintet.

**Visszatérési érték**

Az elért szint.

Definíció a(z) [PrizeLevel.java](#) fájl 35. sorában.

#### 6.18.3.2. `getPrize()`

```
long millionaire.model.PrizeLevel.getPrize ( )
```

Megadja a játékos által elért nyereményt.

**Visszatérési érték**

Az elért nyeremény.

Definíció a(z) [PrizeLevel.java](#) fájl 41. sorában.

#### 6.18.3.3. `isSafeSpot()`

```
boolean millionaire.model.PrizeLevel.isSafeSpot ( )
```

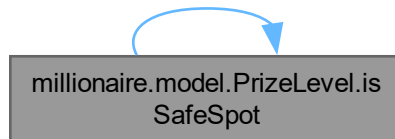
Megadja, hogy a játékos által elért szint szakaszhatároló-e.

**Visszatérési érték**

Igaz, ha a szint szakaszhatároló (garantált nyereménnyel), egyébként hamis.

Definíció a(z) [PrizeLevel.java](#) fájl 47. sorában.

A függvény hívási gráfja:



A függvény hívó gráfja:



Ez a dokumentáció az osztályról a következő fájl alapján készült:

- `src/main/java/millionaire/model/PrizeLevel.java`

## 6.19. millionaire.model.Question osztályreferencia

Egy kérdést reprezentáló osztály.

A millionaire.model.Question osztály együttműködési diagramja:

| millionaire.model.Question                                                                                                                                                                                   |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| + int getDifficulty()<br>+ String getQuestion()<br>+ String getOptionA()<br>+ String getOptionB()<br>+ String getOptionC()<br>+ String getOptionD()<br>+ String getCorrectAnswer()<br>+ String getCategory() |

## Osztályok

- class **QuestionBuilder**

*Builder osztály a Question példányok létrehozásához.*

## Publikus tagfüggvények

- int [getDifficulty](#) ()  
*A kérdéshez tartozó nehézségi szintet adja vissza.*
- String [getQuestion](#) ()  
*A kérdés szövegét adja vissza.*
- String [getOptionA](#) ()  
*A kérdéshez tartozó 'A' válaszlehetőség szövegét adja vissza.*
- String [getOptionB](#) ()  
*A kérdéshez tartozó 'B' válaszlehetőség szövegét adja vissza.*
- String [getOptionC](#) ()  
*A kérdéshez tartozó 'C' válaszlehetőség szövegét adja vissza.*
- String [getOptionD](#) ()  
*A kérdéshez tartozó 'D' válaszlehetőség szövegét adja vissza.*
- String [getCorrectAnswer](#) ()  
*A kérdéshez tartozó helyes válasz betűjelét adja vissza.*
- String [getCategory](#) ()  
*A kérdés kategóriáját adja vissza.*

### 6.19.1. Részletes leírás

Egy kérdést reprezentáló osztály.

Ez az osztály egy játékbeli kérdést reprezentál, annak minden tulajdonságával:

- nehézségi szint (1-15)
- kérdés szövege
- négy válaszlehetőség (A, B, C, D)
- helyes válasz betűjele
- kategória

Az osztály immutable (megváltoztathatatlan), és Builder pattern segítségével példányosítható.

Definíció a(z) [Question.java](#) fájl 16. sorában.

### 6.19.2. Tagfüggvények dokumentációja

#### 6.19.2.1. getCategory()

```
String millionaire.model.Question.getCategory ( )
```

A kérdés kategóriáját adja vissza.

**Visszatérési érték**

A kérdés kategóriája.

Definíció a(z) [Question.java](#) fájl 96. sorában.

#### 6.19.2.2. getCorrectAnswer()

```
String millionaire.model.Question.getCorrectAnswer ( )
```

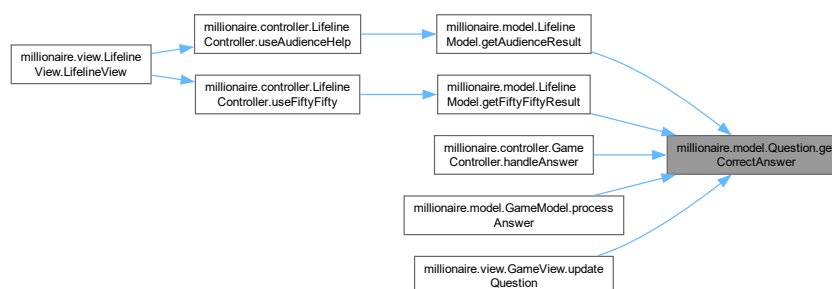
A kérdéshez tartozó helyes válasz betűjelét adja vissza.

**Visszatérési érték**

A helyes válasz betűjele.

Definíció a(z) [Question.java](#) fájl 89. sorában.

A függvény hívó gráfja:



### 6.19.2.3. getDifficulty()

```
int millionaire.model.Question.getDifficulty ( )
```

A kérdéshez tartozó nehézségi szintet adja vissza.

#### Visszatérési érték

A nehézségi szint.

Definíció a(z) [Question.java](#) fájl 47. sorában.

### 6.19.2.4. getOptionA()

```
String millionaire.model.Question.getOptionA ( )
```

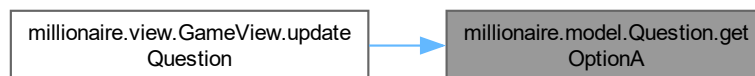
A kérdéshez tartozó 'A' válaszlehetőség szövegét adja vissza.

#### Visszatérési érték

Az 'A' válaszlehetőség szövege.

Definíció a(z) [Question.java](#) fájl 61. sorában.

A függvény hívó gráfja:



### 6.19.2.5. getOptionB()

```
String millionaire.model.Question.getOptionB ( )
```

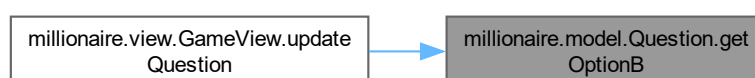
A kérdéshez tartozó 'B' válaszlehetőség szövegét adja vissza.

#### Visszatérési érték

Az 'B' válaszlehetőség szövege.

Definíció a(z) [Question.java](#) fájl 68. sorában.

A függvény hívó gráfja:



#### 6.19.2.6. `getOptionC()`

```
String millionaire.model.Question.getOptionC ( )
```

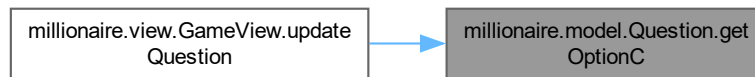
A kérdéshez tartozó 'C' válaszlehetőség szövegét adja vissza.

##### Visszatérési érték

Az 'C' válaszlehetőség szövege.

Definíció a(z) [Question.java](#) fájl 75. sorában.

A függvény hívó gráfja:



#### 6.19.2.7. `getOptionD()`

```
String millionaire.model.Question.getOptionD ( )
```

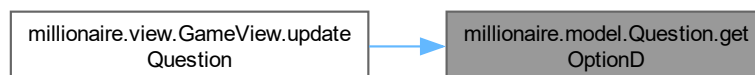
A kérdéshez tartozó 'D' válaszlehetőség szövegét adja vissza.

##### Visszatérési érték

Az 'D' válaszlehetőség szövege.

Definíció a(z) [Question.java](#) fájl 82. sorában.

A függvény hívó gráfja:





#### 6.19.2.8. getQuestion()

```
String millionaire.model.Question.getQuestion ( )
```

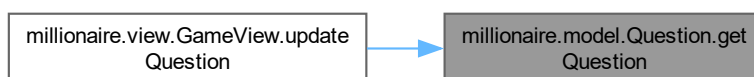
A kérdés szövegét adja vissza.

##### Visszatérési érték

A kérdés szövege.

Definíció a(z) [Question.java](#) fájl 54. sorában.

A függvény hívó gráfja:



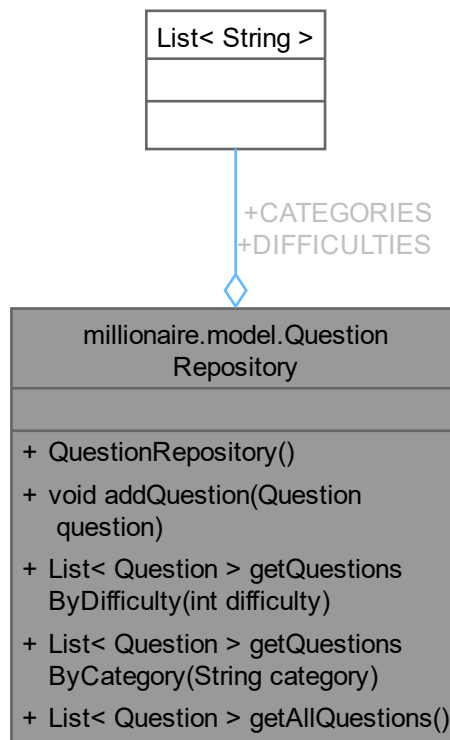
Ez a dokumentáció az osztályról a következő fájl alapján készült:

- [src/main/java/millionaire/model/Question.java](#)

## 6.20. millionaire.model.QuestionRepository osztályreferencia

Kérdések perzisztens tárolását végző osztály.

A `millionaire.model.QuestionRepository` osztály együttműködési diagramja:



### Publikus tagfüggvények

- `QuestionRepository ()`  
*Konstruktor a QuestionRepository létrehozásához.*
- `void addQuestion (Question question)`  
*Hozzáad egy új kérdést a gyűjteményhez.*
- `List< Question > getQuestionsByDifficulty (int difficulty)`  
*Lekérdezi az adott nehézségi szintű kérdéseket.*
- `List< Question > getQuestionsByCategory (String category)`  
*Lekérdezi az adott kategóriába tartozó kérdéseket.*
- `List< Question > getAllQuestions ()`  
*Visszaadja az összes tárolt kérdést.*

### Statikus publikus attribútumok

- `static final List< String > CATEGORIES`  
*Elérhető kategóriák listája.*
- `static final List< String > DIFFICULTIES`  
*Elérhető nehézségi szintek listája.*

### 6.20.1. Részletes leírás

Kérdések perzisztens tárolását végző osztály.

Ez az osztály felelős:

- A kérdések JSON fájlból való betöltéséért
- A kérdések JSON fájlba mentéséért
- A kérdések memóriában való tárolásáért
- A kérdések szűréséért különböző szempontok alapján

Definíció a(z) [QuestionRepository.java](#) fájl 24. sorában.

### 6.20.2. Konstruktorok és destruktorok dokumentációja

#### 6.20.2.1. QuestionRepository()

```
millionaire.model.QuestionRepository.QuestionRepository ( )
```

Konstruktor a QuestionRepository létrehozásához.

Inicializálja az adattagokat és betölti a kérdéseket a JSON fájlból.

Definíció a(z) [QuestionRepository.java](#) fájl 47. sorában.

### 6.20.3. Tagfüggvények dokumentációja

#### 6.20.3.1. addQuestion()

```
void millionaire.model.QuestionRepository.addQuestion (
    Question question )
```

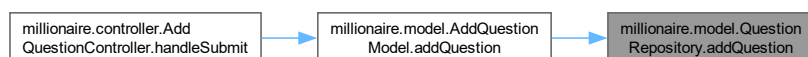
Hozzáad egy új kérdést a gyűjteményhez.

Paraméterek

|                 |                      |
|-----------------|----------------------|
| <i>question</i> | A hozzáadandó kérdés |
|-----------------|----------------------|

Definíció a(z) [QuestionRepository.java](#) fájl 128. sorában.

A függvény hívó gráfja:



### 6.20.3.2. getAllQuestions()

```
List< Question > millionaire.model.QuestionRepository.getAllQuestions ( )
```

Visszaadja az összes tárolt kérdést.

#### Visszatérési érték

Az összes kérdés másolata egy új listában

Definíció a(z) [QuestionRepository.java](#) fájl 160. sorában.

### 6.20.3.3. getQuestionsByCategory()

```
List< Question > millionaire.model.QuestionRepository.getQuestionsByCategory (
    String category )
```

Lekérdezi az adott kategóriába tartozó kérdéseket.

#### Paraméterek

|                 |                         |
|-----------------|-------------------------|
| <i>category</i> | A kívánt kategória neve |
|-----------------|-------------------------|

#### Visszatérési érték

A kategóriához tartozó kérdések listája

Definíció a(z) [QuestionRepository.java](#) fájl 151. sorában.

### 6.20.3.4. getQuestionsByDifficulty()

```
List< Question > millionaire.model.QuestionRepository.getQuestionsByDifficulty (
    int difficulty )
```

Lekérdezi az adott nehézségi szintű kérdéseket.

#### Paraméterek

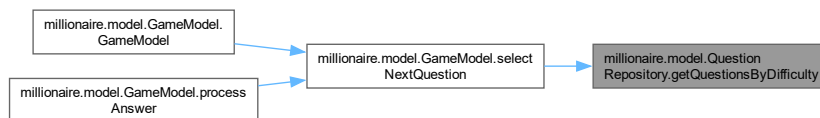
|                   |                                 |
|-------------------|---------------------------------|
| <i>difficulty</i> | A kívánt nehézségi szint (1-15) |
|-------------------|---------------------------------|

#### Visszatérési érték

A megadott nehézségű kérdések listája

Definíció a(z) [QuestionRepository.java](#) fájl 139. sorában.

A függvény hívó gráfja:



## 6.20.4. Adattagok dokumentációja

### 6.20.4.1. CATEGORIES

```
final List<String> millionaire.model.QuestionRepository.CATEGORIES [static]
```

#### Kezdő érték:

```
= Arrays.asList(
    "TÖRTÉNELEM", "SPORT", "FÖLDRAJZ", "IRODALOM", "TUDOMÁNY", "ZENE", "FILM", "KONYHA", "ÁLTALÁNOS"
)
```

Elérhető kategóriák listája.

Definíció a(z) [QuestionRepository.java](#) fájl 31. sorában.

### 6.20.4.2. DIFFICULTIES

```
final List<String> millionaire.model.QuestionRepository.DIFFICULTIES [static]
```

#### Kezdő érték:

```
= Arrays.asList(
    "1", "2", "3", "4", "5", "6", "7", "8", "9", "10", "11", "12", "13", "14", "15"
)
```

Elérhető nehézségi szintek listája.

Definíció a(z) [QuestionRepository.java](#) fájl 37. sorában.

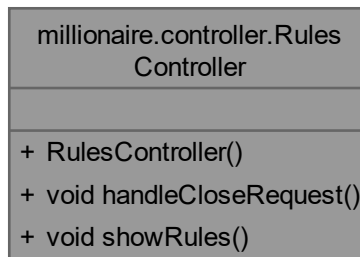
Ez a dokumentáció az osztályról a következő fájl alapján készült:

- [src/main/java/millionaire/model/QuestionRepository.java](#)

## 6.21. millionaire.controller.RulesController osztályreferencia

A játékszabályokat kezelő vezérlő osztály.

A millionaire.controller.RulesController osztály együttműködési diagramja:



### Publikus tagfüggvények

- [RulesController](#) ()  
*A játékszabályok vezérlőjének konstruktora.*
- void [handleCloseRequest](#) ()  
*Kezeli az ablak bezárásának kérését.*
- void [showRules](#) ()  
*Megjeleníti a játékszabályok ablakot.*

### 6.21.1. Részletes leírás

A játékszabályokat kezelő vezérlő osztály.

Ez az osztály felelős:

- A játékszabályok betöltéséért
- A szabályok megjelenítéséért
- A szabályok ablak kezeléséért

Definíció a(z) [RulesController.java](#) fájl 15. sorában.

### 6.21.2. Konstruktorkok és destruktorkok dokumentációja

#### 6.21.2.1. RulesController()

```
millionaire.controller.RulesController.RulesController ( )
```

A játékszabályok vezérlőjének konstruktora.

Létrehozza a játékszabályok modelljét és nézetét, majd inicializálja a nézetet.

Definíció a(z) [RulesController.java](#) fájl 25. sorában.

### 6.21.3. Tagfüggvények dokumentációja

#### 6.21.3.1. handleCloseRequest()

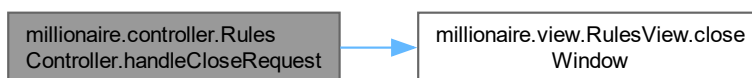
```
void millionaire.controller.RulesController.handleCloseRequest ( )
```

Kezeli az ablak bezárásának kérését.

Meghívja a nézet closeWindow() metódusát az ablak bezárásához és az erőforrások felszabadításához.

Definíció a(z) [RulesController.java](#) fájl 53. sorában.

A függvény hívási gráfja:



#### 6.21.3.2. showRules()

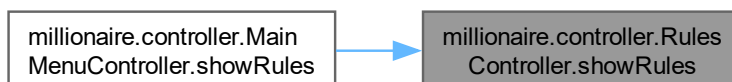
```
void millionaire.controller.RulesController.showRules ( )
```

Megjeleníti a játékszabályok ablakot.

Láthatóvá teszi a játékszabályokat tartalmazó ablakot a felhasználói felületen.

Definíció a(z) [RulesController.java](#) fájl 63. sorában.

A függvény hívó gráfja:



Ez a dokumentáció az osztályról a következő fájl alapján készült:

- [src/main/java/millionaire/controller/RulesController.java](#)

## 6.22. millionaire.model.RulesModel osztályreferencia

Játékszabályok kezelését végző osztály.

A millionaire.model.RulesModel osztály együttműködési diagramja:

| millionaire.model.RulesModel                                                |
|-----------------------------------------------------------------------------|
|                                                                             |
| + RulesModel()<br>+ String getRulesContent()<br>+ boolean isContentLoaded() |

### Publikus tagfüggvények

- [RulesModel](#) ()  
*Konstruktor a RulesModel létrehozásához.*
- String [getRulesContent](#) ()  
*Visszaadja a betöltött játékszabályok szövegét.*
- boolean [isContentLoaded](#) ()  
*Ellenőrzi a játékszabályok betöltésének sikerességét.*

### 6.22.1. Részletes leírás

Játékszabályok kezelését végző osztály.

Ez az osztály felelős:

- A játékszabályok betöltéséért text fájlból
- A szabályok tartalmának tárolásáért
- A betöltés sikerességének ellenőrzéséért

Definíció a(z) [RulesModel.java](#) fájl 18. sorában.

### 6.22.2. Konstruktorok és destruktorok dokumentációja

#### 6.22.2.1. RulesModel()

```
millionaire.model.RulesModel.RulesModel ( )
```

Konstruktor a RulesModel létrehozásához.

Létrehozáskor azonnal megkísérli betölteni a játékszabályokat.

Definíció a(z) [RulesModel.java](#) fájl 30. sorában.



### 6.22.3. Tagfüggvények dokumentációja

#### 6.22.3.1. getRulesContent()

```
String millionaire.model.RulesModel.getRulesContent ( )
```

Visszaadja a betöltött játékszabályok szövegét.

##### Visszatérési érték

A szabályok szövege, vagy hibaüzenet ha a betöltés sikertelen volt

Definíció a(z) [RulesModel.java](#) fájl 53. sorában.

#### 6.22.3.2. isContentLoaded()

```
boolean millionaire.model.RulesModel.isContentLoaded ( )
```

Ellenőrzi a játékszabályok betöltésének sikerességét.

##### Visszatérési érték

true ha a szabályok sikeresen betöltődtek, false ha hiba történt

Definíció a(z) [RulesModel.java](#) fájl 61. sorában.

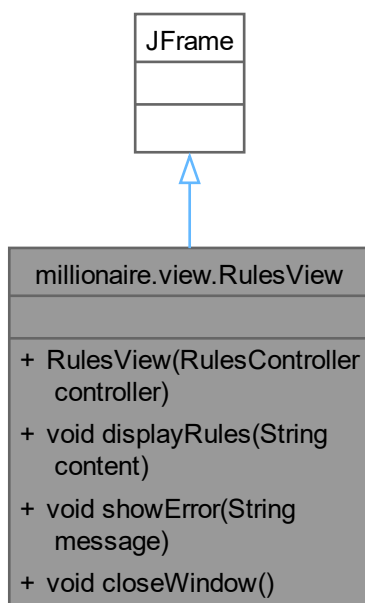
Ez a dokumentáció az osztályról a következő fájl alapján készült:

- [src/main/java/millionaire/model/RulesModel.java](#)

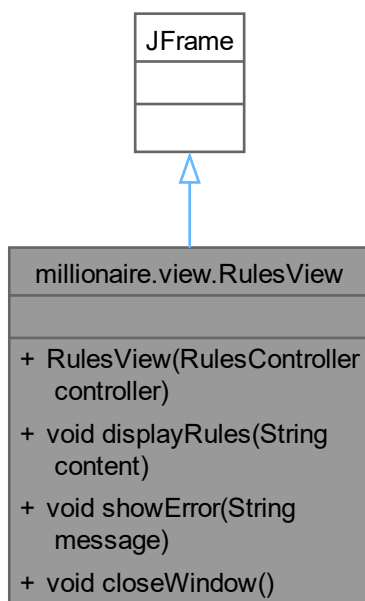
## 6.23. millionaire.view.RulesView osztályreferencia

A játékszabályok megjelenítésére szolgáló ablak.

A millionaire.view.RulesView osztály származási diagramja:



A millionaire.view.RulesView osztály együttműködési diagramja:



## Publikus tagfüggvények

- [RulesView](#) ([RulesController](#) controller)  
*Konstruktor a RulesView létrehozásához.*
- void [displayRules](#) (String content)  
*Beállítja a játékszabályok szövegét.*
- void [showError](#) (String message)  
*Hibaüzenet megjelenítése.*
- void [closeWindow](#) ()  
*Az ablak bezárása után felszabadítja a hozzá tartozó erőforrásokat.*

### 6.23.1. Részletes leírás

A játékszabályok megjelenítésére szolgáló ablak.

Ez az osztály felelős a játékszabályok megjelenítéséért egy görgethető szövegterületen. Az ablak tartalmazza:

- A játékszabályok szövegét
- Görgetősávot a hosszabb tartalom kezeléséhez
- Bezárás gombot

Definíció a(z) [RulesView.java](#) fájl 17. sorában.

### 6.23.2. Konstruktorok és destruktorok dokumentációja

#### 6.23.2.1. RulesView()

```
millionaire.view.RulesView.RulesView (  
    RulesController controller )
```

Konstruktor a RulesView létrehozásához.

#### Paraméterek

|                   |                                 |
|-------------------|---------------------------------|
| <i>controller</i> | A szabályokat kezelő controller |
|-------------------|---------------------------------|

Definíció a(z) [RulesView.java](#) fájl 33. sorában.

### 6.23.3. Tagfüggvények dokumentációja

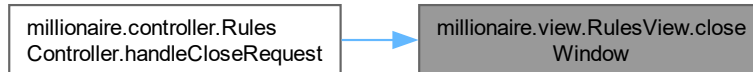
#### 6.23.3.1. closeWindow()

```
void millionaire.view.RulesView.closeWindow ( )
```

Az ablak bezárása után felszabadítja a hozzá tartozó erőforrásokat.

Definíció a(z) [RulesView.java](#) fájl 175. sorában.

A függvény hívó gráfja:



### 6.23.3.2. displayRules()

```
void millionaire.view.RulesView.displayRules (
    String content )
```

Beállítja a játékszabályok szövegét.

#### Paraméterek

|                |                                    |
|----------------|------------------------------------|
| <i>content</i> | A megjelenítendő szabályok szövege |
|----------------|------------------------------------|

Definíció a(z) [RulesView.java](#) fájl 156. sorában.

### 6.23.3.3. showError()

```
void millionaire.view.RulesView.showError (
    String message )
```

Hibaüzenet megjelenítése.

#### Paraméterek

|                |                             |
|----------------|-----------------------------|
| <i>message</i> | A megjelenítendő hibaüzenet |
|----------------|-----------------------------|

Definíció a(z) [RulesView.java](#) fájl 167. sorában.

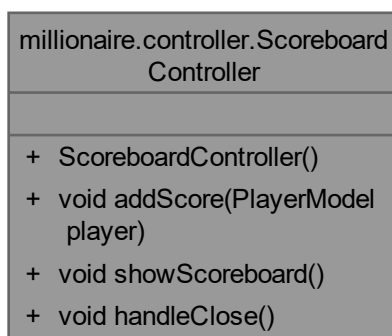
Ez a dokumentáció az osztályról a következő fájl alapján készült:

- `src/main/java/millionaire/view/RulesView.java`

## 6.24. millionaire.controller.ScoreboardController osztályreferencia

A dicsőséglista kezelését végző vezérlő osztály.

A millionaire.controller.ScoreboardController osztály együttműködési diagramja:



#### Publikus tagfüggvények

- [ScoreboardController \(\)](#)  
*Konstruktor a ScoreboardController létrehozásához.*
- void [addScore \(PlayerModel player\)](#)  
*Új eredmény hozzáadása a dicsőséglistához.*
- void [showScoreboard \(\)](#)  
*Megjeleníti a dicsőséglista ablakot.*
- void [handleClose \(\)](#)  
*Kezeli az ablak bezárásának kérését.*

#### 6.24.1. Részletes leírás

A dicsőséglista kezelését végző vezérlő osztály.

Ez az osztály felelős:

- A játékosok eredményeinek kezeléséért és megjelenítéséért
- Új eredmények hozzáadásáért a dicsőséglistához
- A dicsőséglista frissítéséért és megjelenítéséért
- A dicsőséglista ablak életciklusának kezeléséért

Definíció a(z) [ScoreboardController.java](#) fájl 17. sorában.

## 6.24.2. Konstruktorkok és destruktorkok dokumentációja

### 6.24.2.1. ScoreboardController()

```
millionaire.controller.ScoreboardController.ScoreboardController ( )
```

Konstruktork a ScoreboardController létrehozásához.

Inicializálja a modellt és a nézetet, majd meghívja a refreshScoreboard() metódust a kezdeti megjelenítéshez.

Definíció a(z) [ScoreboardController.java](#) fájl 29. sorában.

## 6.24.3. Tagfüggvények dokumentációja

### 6.24.3.1. addScore()

```
void millionaire.controller.ScoreboardController.addScore (
    PlayerModel player )
```

Új eredmény hozzáadása a dicsőséglistához.

A metódus:

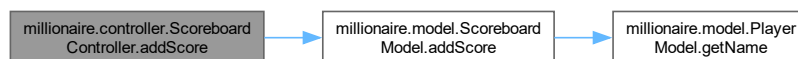
1. Hozzáadja az új játékos eredményét a modellhez
2. Frissíti a dicsőséglista megjelenítését

#### Paraméterek

|               |                                               |
|---------------|-----------------------------------------------|
| <i>player</i> | Az új eredménnyel rendelkező játékos modellje |
|---------------|-----------------------------------------------|

Definíció a(z) [ScoreboardController.java](#) fájl 44. sorában.

A függvény hívási gráfja:



### 6.24.3.2. handleClose()

```
void millionaire.controller.ScoreboardController.handleClose ( )
```

Kezeli az ablak bezárásának kérését.

Meghívja a nézet `closeWindow()` metódusát az ablak bezárásához és az erőforrások felszabadításához.

Definíció a(z) [ScoreboardController.java](#) fájl 78. sorában.

A függvény hívási gráfja:



### 6.24.3.3. showScoreboard()

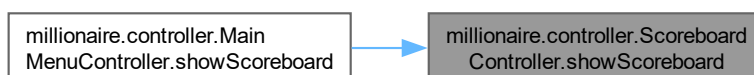
```
void millionaire.controller.ScoreboardController.showScoreboard ( )
```

Megjeleníti a dicsőséglista ablakot.

1. Frissíti a dicsőséglistát a legújabb adatokkal
2. Láthatóvá teszi a dicsőséglistát tartalmazó ablakot

Definíció a(z) [ScoreboardController.java](#) fájl 67. sorában.

A függvény hívó gráfja:



Ez a dokumentáció az osztályról a következő fájl alapján készült:

- `src/main/java/millionaire/controller/ScoreboardController.java`

## 6.25. millionaire.model.ScoreboardModel osztályreferencia

Dicsőségtábla kezelését végző osztály.

A millionaire.model.ScoreboardModel osztály együttműködési diagramja:



### Publikus tagfüggvények

- [ScoreboardModel \(\)](#)  
*Konstruktor a ScoreboardModel létrehozásához.*
- void [addScore \(PlayerModel newPlayer\)](#)  
*Új eredmény hozzáadása vagy meglévő frissítése.*
- List< [PlayerModel](#) > [getPlayers \(\)](#)  
*Játékosok listájának lekérdezése.*

### 6.25.1. Részletes leírás

Dicsőségtábla kezelését végző osztály.

Ez az osztály felelős:

- A játékosok eredményeinek betöltéséért JSON fájlból
- Az eredmények perzisztens tárolásáért
- A játékosok rangsorolásáért
- Új eredmények hozzáadásáért és frissítéséért

Definíció a(z) [ScoreboardModel.java](#) fájl 24. sorában.



## 6.25.2. Konstruktork és destruktork dokumentációja

### 6.25.2.1. ScoreboardModel()

```
millionaire.model.ScoreboardModel.ScoreboardModel ( )
```

Konstruktork a ScoreboardModel létrehozásához.

Inicializálja a listát és betölti a korábbi eredményeket.

Definíció a(z) [ScoreboardModel.java](#) fájl 39. sorában.

## 6.25.3. Tagfüggvények dokumentációja

### 6.25.3.1. addScore()

```
void millionaire.model.ScoreboardModel.addScore (
    PlayerModel newPlayer )
```

Új eredmény hozzáadása vagy meglévő frissítése.

Ha a játékos már szerepel a listában és az új eredménye jobb, akkor frissíti a rekordját. Ha még nem szerepelt, vagy rosszabb az új eredménye, akkor az új eredményt menti el.

Paraméterek

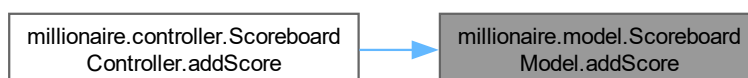
|                  |                         |
|------------------|-------------------------|
| <i>newPlayer</i> | Az új játékos eredménye |
|------------------|-------------------------|

Definíció a(z) [ScoreboardModel.java](#) fájl 109. sorában.

A függvény hívási gráfja:



A függvény hívó gráfja:



### 6.25.3.2. getPlayers()

```
List< PlayerModel > millionaire.model.ScoreboardModel.getPlayers ( )
```

Játékosok listájának lekérdezése.

#### Visszatérési érték

A játékosok listájának másolata

Definíció a(z) [ScoreboardModel.java](#) fájl 160. sorában.

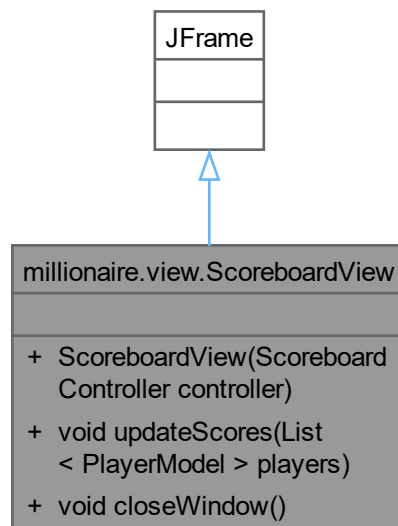
Ez a dokumentáció az osztályról a következő fájl alapján készült:

- [src/main/java/millionaire/model/ScoreboardModel.java](#)

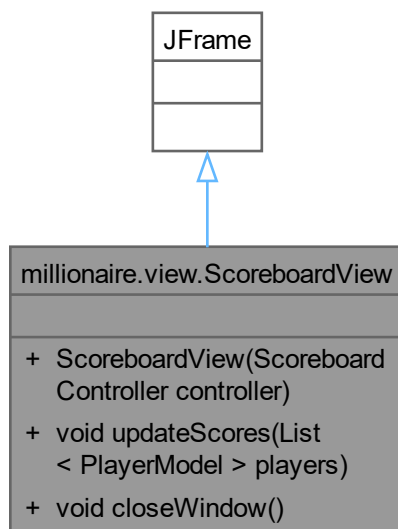
## 6.26. millionaire.view.ScoreboardView osztályreferencia

A dicsőséglista megjelenítésére szolgáló ablak.

A millionaire.view.ScoreboardView osztály származási diagramja:



A millionaire.view.ScoreboardView osztály együttműködési diagramja:



#### Publikus tagfüggvények

- [ScoreboardView](#) ([ScoreboardController](#) controller)  
*Konstruktor a ScoreboardView létrehozásához.*
- void [updateScores](#) (List< [PlayerModel](#) > players)  
*Frissíti a táblázat tartalmát.*
- void [closeWindow](#) ()  
*Az ablak bezárása után felszabadítja a hozzá tartozó erőforrásokat.*

#### 6.26.1. Részletes leírás

A dicsőséglista megjelenítésére szolgáló ablak.

Ez az osztály felelős a játékosok eredményeinek táblázatos megjelenítéséért. Az ablak tartalmazza:

- A játékosok eredményeit tartalmazó táblázatot
- Görgetősávot a hosszabb lista kezeléséhez
- Bezárás gombot

Definíció a(z) [ScoreboardView.java](#) fájl 20. sorában.

## 6.26.2. Konstruktorok és destruktorok dokumentációja

### 6.26.2.1. ScoreboardView()

```
millionaire.view.ScoreboardView.ScoreboardView (  
    ScoreboardController controller )
```

Konstruktor a ScoreboardView létrehozásához.

Inicializálja a táblázatot és beállítja annak modelljét.

## Paraméterek

|                   |                                    |
|-------------------|------------------------------------|
| <i>controller</i> | A dicsőséglistát kezelő controller |
|-------------------|------------------------------------|

Definíció a(z) [ScoreboardView.java](#) fájl 40. sorában.

### 6.26.3. Tagfüggvények dokumentációja

#### 6.26.3.1. closeWindow()

```
void millionaire.view.ScoreboardView.closeWindow ( )
```

Az ablak bezárása után felszabadítja a hozzá tartozó erőforrásokat.

Definíció a(z) [ScoreboardView.java](#) fájl 208. sorában.

A függvény hívó gráfja:



#### 6.26.3.2. updateScores()

```
void millionaire.view.ScoreboardView.updateScores (
    List< PlayerModel > players )
```

Frissíti a táblázat tartalmát.

Törli a korábbi adatokat és megjeleníti az új eredményeket.

## Paraméterek

|                |                                    |
|----------------|------------------------------------|
| <i>players</i> | A megjelenítendő játékosok listája |
|----------------|------------------------------------|

Definíció a(z) [ScoreboardView.java](#) fájl 190. sorában.

Ez a dokumentáció az osztályról a következő fájl alapján készült:

- `src/main/java/millionaire/view/ScoreboardView.java`

## 6.27. millionaire.controller.TimerController osztályreferencia

Az időmérő vezérlője.

A millionaire.controller.TimerController osztály együttműködési diagramja:

| millionaire.controller.TimerController                                                                                   |
|--------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                          |
| + TimerController()<br>+ void startTimer()<br>+ void stopTimer()<br>+ TimerView getView()<br>+ Duration getElapsedTime() |

### Publikus tagfüggvények

- [TimerController \(\)](#)  
*Konstruktor a vezérlő létrehozásához.*
- void [startTimer \(\)](#)  
*Időmérő indítása.*
- void [stopTimer \(\)](#)  
*Időmérő megállítása.*
- [TimerView getView \(\)](#)  
*Visszaadja az időmérő nézetét.*
- Duration [getElapsedTime \(\)](#)  
*Visszaadja az eltelt időt.*

### 6.27.1. Részletes leírás

Az időmérő vezérlője.

Ez az osztály felelős:

- Az időmérő működésének vezérléséért
- A modell és view közötti kommunikációért

Definíció a(z) [TimerController.java](#) fájl 16. sorában.

## 6.27.2. Konstruktorkok és destruktorkok dokumentációja

### 6.27.2.1. TimerController()

```
millionaire.controller.TimerController.TimerController ( )
```

Konstruktork a vezérlő létrehozásához.

Definíció a(z) [TimerController.java](#) fájl 24. sorában.

## 6.27.3. Tagfüggvények dokumentációja

### 6.27.3.1. getElapsedTime()

```
Duration millionaire.controller.TimerController.getElapsedTime ( )
```

Visszaadja az eltelt időt.

#### Visszatérési érték

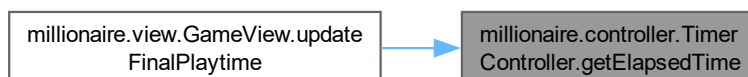
Az eltelt idő Duration objektumként

Definíció a(z) [TimerController.java](#) fájl 68. sorában.

A függvény hívási gráfja:



A függvény hívó gráfja:



### 6.27.3.2. getView()

`TimerView` `millionaire.controller.TimerController.getView ( )`

Visszaadja az időmérő nézetét.

#### Visszatérési érték

Az időmérő nézete

Definíció a(z) `TimerController.java` fájl 60. sorában.

### 6.27.3.3. startTimer()

`void` `millionaire.controller.TimerController.startTimer ( )`

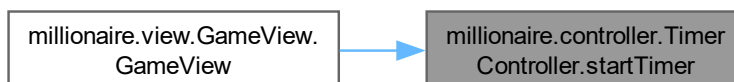
Időmérő indítása.

Definíció a(z) `TimerController.java` fájl 43. sorában.

A függvény hívási gráfja:



A függvény hívó gráfja:





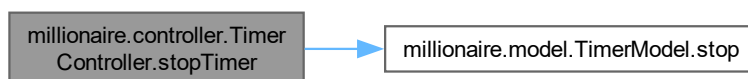
#### 6.27.3.4. stopTimer()

```
void millionaire.controller.TimerController.stopTimer ( )
```

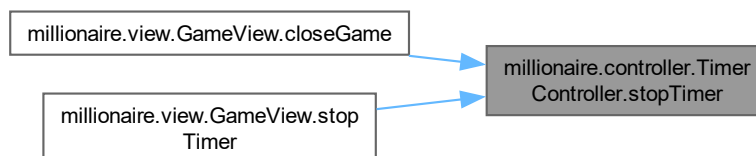
Időmérő megállítása.

Definíció a(z) [TimerController.java](#) fájl 51. sorában.

A függvény hívási gráfja:



A függvény hívó gráfja:



Ez a dokumentáció az osztályról a következő fájl alapján készült:

- `src/main/java/millionaire/controller/TimerController.java`

## 6.28. millionaire.model.TimerModel osztályreferencia

Az időmérő adatmodellje.

A `millionaire.model.TimerModel` osztály együttműködési diagramja:

| millionaire.model.TimerModel                                                                                                               |
|--------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                            |
| + TimerModel()<br>+ void updateElapsedTime()<br>+ void start()<br>+ void stop()<br>+ Duration getElapsedTime()<br>+ Instant getStartTime() |

#### Publikus tagfüggvények

- [TimerModel](#) ()  
*Konstruktor az időmérő létrehozásához.*
- void [updateElapsedTime](#) ()  
*Frissíti az eltelt időt.*
- void [start](#) ()  
*Elindítja az időmérőt.*
- void [stop](#) ()  
*Megállítja az időmérőt.*
- Duration [getElapsedTime](#) ()  
*Visszaadja az eltelt időt.*
- Instant [getStartTime](#) ()  
*Visszaadja a kezdési időpontot.*

#### 6.28.1. Részletes leírás

Az időmérő adatmodellje.

Ez az osztály felelős:

- A játékidő tárolásáért
- Az időmérés állapotának kezeléséért

Definíció a(z) [TimerModel.java](#) fájl 14. sorában.

## 6.28.2. Konstruktorkok és destruktorkok dokumentációja

### 6.28.2.1. TimerModel()

```
millionaire.model.TimerModel.TimerModel ( )
```

Konstruktork az időmérő létrehozásához.

Definíció a(z) [TimerModel.java](#) fájl 22. sorában.

## 6.28.3. Tagfüggvények dokumentációja

### 6.28.3.1. getElapsedTime()

```
Duration millionaire.model.TimerModel.getElapsedTime ( )
```

Visszaadja az eltelt időt.

#### Visszatérési érték

Az eltelt idő Duration objektumként

Definíció a(z) [TimerModel.java](#) fájl 55. sorában.

A függvény hívó gráfja:



### 6.28.3.2. getStartTime()

```
Instant millionaire.model.TimerModel.getStartTime ( )
```

Visszaadja a kezdési időpontot.

#### Visszatérési érték

A kezdési időpont Instant objektumként

Definíció a(z) [TimerModel.java](#) fájl 63. sorában.

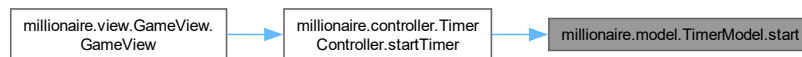
### 6.28.3.3. start()

```
void millionaire.model.TimerModel.start ( )
```

Elindítja az időmérőt.

Definíció a(z) [TimerModel.java](#) fájl 40. sorában.

A függvény hívó gráfja:



### 6.28.3.4. stop()

```
void millionaire.model.TimerModel.stop ( )
```

Megállítja az időmérőt.

Definíció a(z) [TimerModel.java](#) fájl 47. sorában.

A függvény hívó gráfja:



### 6.28.3.5. updateElapsedTime()

```
void millionaire.model.TimerModel.updateElapsedTime ( )
```

Frissíti az eltelt időt.

Definíció a(z) [TimerModel.java](#) fájl 31. sorában.

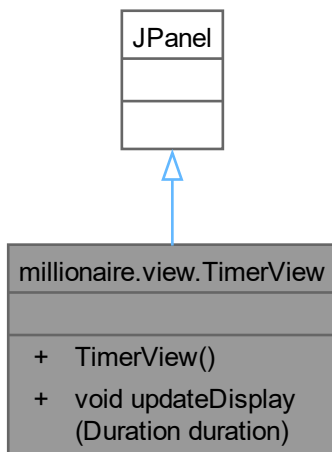
Ez a dokumentáció az osztályról a következő fájl alapján készült:

- `src/main/java/millionaire/model/TimerModel.java`

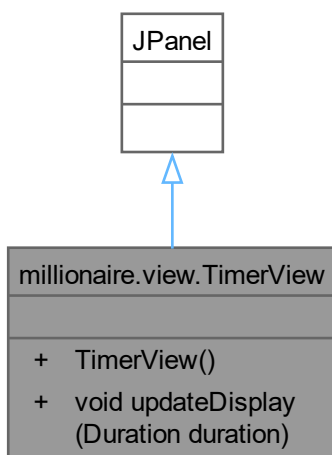
## 6.29. millionaire.view.TimerView osztályreferencia

Az időmérő megjelenítése.

A millionaire.view.TimerView osztály származási diagramja:



A millionaire.view.TimerView osztály együttműködési diagramja:



### Publikus tagfüggvények

- [TimerView](#) ()

*Konstruktor a nézet létrehozásához.*

- void [updateDisplay](#) (Duration duration)

*Az idő kijelzésének frissítése.*

### 6.29.1. Részletes leírás

Az időmérő megjelenítése.

Ez az osztály felelős:

- Az időmérő grafikus megjelenítéséért
- Az idő formázott kijelzéséért

Definíció a(z) [TimerView.java](#) fájl 15. sorában.

### 6.29.2. Konstruktorok és destruktorok dokumentációja

#### 6.29.2.1. TimerView()

```
millionaire.view.TimerView.TimerView ( )
```

Konstruktor a nézet létrehozásához.

Definíció a(z) [TimerView.java](#) fájl 24. sorában.

### 6.29.3. Tagfüggvények dokumentációja

#### 6.29.3.1. updateDisplay()

```
void millionaire.view.TimerView.updateDisplay (
    Duration duration )
```

Az idő kijelzésének frissítése.

Paraméterek

|                 |               |
|-----------------|---------------|
| <i>duration</i> | Az eltelt idő |
|-----------------|---------------|

Definíció a(z) [TimerView.java](#) fájl 41. sorában.

Ez a dokumentáció az osztályról a következő fájl alapján készült:

- src/main/java/millionaire/view/[TimerView.java](#)

## 7. fejezet

# Fájlok dokumentációja

### 7.1. src/main/java/millionaire/controller/AddQuestionController.java fájlreferencia

#### Osztályok

- class `millionaire.controller.AddQuestionController`  
*Új kérdések hozzáadását vezérlő osztály.*

#### Csomagok

- package `millionaire.controller`

### 7.2. AddQuestionController.java

[Ugrás a fájl dokumentációjához.](#)

```
00001
00002 package millionaire.controller;
00003
00004 import millionaire.model.AddQuestionModel;
00005 import millionaire.model.QuestionRepository;
00006 import millionaire.view.AddQuestionView;
00007 import java.util.List;
00008
00018 public class AddQuestionController {
00022     private final AddQuestionModel model;
00023
00027     private final AddQuestionView view;
00028
00029
00034     public AddQuestionController(QuestionRepository repository) {
00035         this.model = new AddQuestionModel(repository);
00036         this.view = new AddQuestionView(this);
00037     }
00038
00039
00056     public void handleSubmit(String question, String optionA, String optionB, String optionC, String
optionD,
00057                             String correctAnswer, String difficulty, String category) {
00058         if (!model.validateInputs(question, optionA, optionB, optionC, optionD)) {
00059             view.showError("Minden mezőt ki kell tölteni!");
00060             return;
00061         }
00062
00063         boolean success = model.addQuestion(question, optionA, optionB, optionC, optionD,
correctAnswer, difficulty, category);
```

```

00064
00065         if (success) {
00066             view.showSuccess("A kérdés sikeresen hozzáadva!");
00067             view.clearFields();
00068         } else {
00069             view.showError("Hiba történt a kérdés mentése közben!");
00070         }
00071     }
00072
00073
00074
00075
00076
00077     public void handleCancel() {
00078         view.closeWindow();
00079     }
00080
00081
00082
00083
00084
00085     public List<String> getDifficulties() {
00086         return model.getDifficulties();
00087     }
00088
00089
00090
00091
00092
00093     public List<String> getCategories() {
00094         return model.getCategories();
00095     }
00096
00097
00098
00099
00100
00101     public void showDialog() {
00102         view.setVisible(true);
00103     }
00104
00105
00106 }

```

### 7.3. src/main/java/millionaire/controller/GameController.java fájlreferencia

#### Osztályok

- class [millionaire.controller.GameController](#)

*A játék fő vezérlő osztálya.*

#### Csomagok

- package [millionaire.controller](#)

### 7.4. GameController.java

[Ugrás a fájl dokumentációjához.](#)

```

00001
00002 package millionaire.controller;
00003
00004 import millionaire.model.*;
00005 import millionaire.view.GameView;
00006 import javax.swing.*;
00007 import java.util.Timer;
00008 import java.util.TimerTask;
00009 import java.time.Duration;
00010
00011
00012
00013
00014
00015
00016
00017
00018
00019
00020
00021 public class GameController {
00022     private final GameModel model;
00023     private final GameView view;
00024     private final LifelineController lifelineController;
00025     private boolean isProcessingAnswer;
00026     public GameController(PlayerModel player, QuestionRepository questionRepository) {
00027         this.model = new GameModel(questionRepository, player);
00028         this.lifelineController = new LifelineController(this);
00029         this.view = new GameView(this, lifelineController);
00030         this.isProcessingAnswer = false;
00031
00032         initializeGame();
00033     }
00034
00035
00036
00037
00038
00039     private void initializeGame() {
00040
00041
00042
00043
00044
00045
00046

```



```

00047         updateGameState();
00048         view.setVisible(true);
00049     }
00050
00051
00061     public void handleAnswer(String selectedAnswer) {
00062         if (isProcessingAnswer) return;
00063         isProcessingAnswer = true;
00064
00065         Question currentQuestion = model.getCurrentQuestion();
00066         boolean isCorrect = currentQuestion.getCorrectAnswer().equals(selectedAnswer);
00067
00068         //
00069         view.showAnswerResult(selectedAnswer, currentQuestion.getCorrectAnswer(), isCorrect);
00070
00071         // Process the answer after a delay
00072         Timer timer = new Timer();
00073         timer.schedule(new TimerTask() {
00074             @Override
00075             public void run() {
00076                 SwingUtilities.invokeLater(() -> processAnswer(selectedAnswer));
00077                 timer.cancel();
00078             }
00079         }, 1500);
00080     }
00081
00082
00093     private void processAnswer(String selectedAnswer) {
00094         GameState result = model.processAnswer(selectedAnswer);
00095
00096         switch (result) {
00097             case CORRECT:
00098                 updateGameState();
00099                 break;
00100
00101             case WON:
00102             case LOST:
00103                 handleGameEnd(result);
00104                 break;
00105         }
00106
00107         isProcessingAnswer = false;
00108     }
00109
00110
00119     private void updateGameState() {
00120         Question currentQuestion = model.getCurrentQuestion();
00121         int currentLevel = model.getPlayer().getCurrentLevel();
00122
00123         view.updateQuestion(currentQuestion);
00124         view.updatePrizeLadder(currentLevel);
00125
00126         lifelineController.setGameContext(currentQuestion, currentLevel, view.getAnswerButtons());
00127     }
00128
00129
00140     private void handleGameEnd(GameState result) {
00141         PlayerModel player = model.getPlayer();
00142
00143         view.stopTimer();
00144
00145         long finalPrize = result == GameState.WON ?
00146             model.getCurrentPrize() :
00147             model.getLastSafePrize();
00148
00149         player.setPrize(finalPrize);
00150
00151         view.updateFinalPlaytime();
00152
00153         view.showGameEndDialog(result, player.getFormattedPrize());
00154
00155         saveScoreAndClose(player);
00156     }
00157
00158
00168     private void saveScoreAndClose(PlayerModel player) {
00169         ScoreboardController scoreboardController = new ScoreboardController();
00170         scoreboardController.addScore(player);
00171
00172         view.closeGame();
00173         new MainMenuController().showMenu();
00174     }
00175
00176
00181     public PlayerModel getPlayer() {
00182         return model.getPlayer();
00183     }

```

```

00184
00185
00190     public void updateGameTime(Duration elapsedTime) {
00191         model.updatePlayTime(elapsedTime);
00192     }
00193
00194
00199     private void checkTimeLimit(Duration currentTime) {
00200         if (model.isTimeUp(currentTime)) {
00201             handleGameEnd(GameState.LOST);
00202         }
00203     }
00204
00205
00210     public Question getCurrentQuestion() {
00211         return model.getCurrentQuestion();
00212     }
00213
00214
00219     public int getCurrentLevel() {
00220         return model.getPlayer().getCurrentLevel();
00221     }
00222
00223
00228     public void setAnswerButtonsEnabled(boolean enabled) {
00229         view.setAnswerButtonsEnabled(enabled);
00230     }
00231
00232
00237     public GameView getGameView() {
00238         return view;
00239     }
00240 }

```

## 7.5. src/main/java/millionaire/controller/LifelineController.java fájlreferencia

### Osztályok

- class [millionaire.controller.LifelineController](#)  
A játék segítségét kezelő vezérlő osztály.

### Csomagok

- package [millionaire.controller](#)

## 7.6. LifelineController.java

[Ugrás a fájl dokumentációjához.](#)

```

00001
00002 package millionaire.controller;
00003
00004 import millionaire.model.LifelineModel;
00005 import millionaire.model.Question;
00006 import millionaire.view.LifelineView;
00007 import javax.swing.JButton;
00008 import javax.swing.JPanel;
00009 import javax.swing.SwingUtilities;
00010
00011 import java.util.Map;
00012
00013
00023 public class LifelineController {
00024     private final LifelineModel model;
00025     private final LifelineView view;
00026     private final GameController gameController;
00027     private Question currentQuestion;
00028     private int currentLevel;

```

```

00029     private JButton[] answerButtons;
00036     public LifelineController(GameController gameController) {
00037         this.gameController = gameController;
00038         this.model = new LifelineModel();
00039         this.view = new LifelineView(gameController.getGameView(), this);
00040     }
00041
00042
00056     public void setGameContext(Question question, int level, JButton[] buttons) {
00057         this.currentQuestion = question;
00058         this.currentLevel = level;
00059         this.answerButtons = buttons;
00060         updateLifelineAvailability();
00061     }
00062
00063
00069     public void useFiftyFifty() {
00070         if (!model.isFiftyFiftyAvailable()) {
00071             return;
00072         }
00073         try {
00074             Map<String, Boolean> result = model.getFiftyFiftyResult(currentQuestion);
00075             view.updateAnswerButtons(answerButtons, result);
00076             updateLifelineAvailability();
00077         } catch (Exception e) {
00078             System.err.println("Error using fifty-fifty: " + e.getMessage());
00079         }
00080     }
00081
00082
00088     public void useAudienceHelp() {
00089         if (!model.isAudienceHelpAvailable()) {
00090             return;
00091         }
00092         try {
00093             Map<String, Integer> result = model.getAudienceResult(currentQuestion, currentLevel);
00094             view.showAudienceResults(result);
00095             updateLifelineAvailability();
00096         } catch (Exception e) {
00097             System.err.println("Error using audience help: " + e.getMessage());
00098         }
00099     }
00100
00101
00105     private void updateLifelineAvailability() {
00106         SwingUtilities.invokeLater(() -> {
00107             view.updateButtonStates(
00108                 model.isFiftyFiftyAvailable(),
00109                 model.isAudienceHelpAvailable()
00110             );
00111         });
00112     }
00113
00114
00120     public JPanel getLifelinePanel() {
00121         return view.createLifelinePanel();
00122     }
00123 }

```

## 7.7. src/main/java/millionaire/controller/MainMenuController.java fájlreferencia

### Osztályok

- class [millionaire.controller.MainMenuController](#)

A főmenü vezérlő osztálya.

### Csomagok

- package [millionaire.controller](#)

## 7.8. MainMenuController.java

[Ugrás a fájl dokumentációjához.](#)

```

00001
00002 package millionaire.controller;
00003
00004 import millionaire.model.*;
00005 import millionaire.view.MainMenuView;
00006
00007
00018 public class MainMenuController {
00019     private final MainMenuModel model;
00020     private final MainMenuView view;
00025     public MainMenuController() {
00026         this.model = new MainMenuModel();
00027         this.view = new MainMenuView(this);
00028     }
00029
00030
00037     public void startNewGame() {
00038         PlayerController playerController = new PlayerController(view);
00039         PlayerModel player = playerController.showPlayerCreationDialog();
00040
00041         if (player != null) {
00042             view.hideWindow();
00043             GameController gameController = new GameController(player, model.getQuestionRepository());
00044         }
00045     }
00046
00047
00051     public void showAddQuestionDialog() {
00052         AddQuestionController addQuestionController =
00053             new AddQuestionController(model.getQuestionRepository());
00054         addQuestionController.showDialog();
00055     }
00056
00057
00061     public void showScoreboard() {
00062         ScoreboardController scoreboardController = new ScoreboardController();
00063         scoreboardController.showScoreboard();
00064     }
00065
00066
00070     public void showRules() {
00071         RulesController rulesController = new RulesController();
00072         rulesController.showRules();
00073     }
00074
00075
00079     public void exitGame() {
00080         System.exit(0);
00081     }
00082
00083
00087     public void showMenu() {
00088         view.showWindow();
00089     }
00090 }

```

## 7.9. src/main/java/millionaire/controller/PlayerController.java fájlreferencia

### Osztályok

- class [millionaire.controller.PlayerController](#)  
*A játékos létrehozását kezelő vezérlő osztály.*

### Csomagok

- package [millionaire.controller](#)

## 7.10. PlayerController.java

[Ugrás a fájl dokumentációjához.](#)

```
00001
00002 package millionaire.controller;
00003
00004 import millionaire.model.PlayerModel;
00005 import millionaire.view.PlayerView;
00006 import javax.swing.JFrame;
00007
00008
00017 public class PlayerController {
00018     private PlayerView view;
00019     private PlayerModel player;
00020     private boolean wasSuccessful;
00027     public PlayerController(JFrame parentFrame) {
00028         this.view = new PlayerView(parentFrame, this);
00029         this.wasSuccessful = false;
00030     }
00031
00032
00040     public void createPlayer(String name) {
00041         if (name.isEmpty()) {
00042             view.showError("A név nem lehet üres!");
00043             return;
00044         }
00045
00046         player = new PlayerModel(name);
00047         wasSuccessful = true;
00048         view.closeDialog();
00049     }
00050
00051
00055     public void cancelPlayerCreation() {
00056         wasSuccessful = false;
00057         view.closeDialog();
00058     }
00059
00060
00065     public PlayerModel showPlayerCreationDialog() {
00066         view.setVisible(true);
00067         return wasSuccessful ? player : null;
00068     }
00069 }
```

## 7.11. src/main/java/millionaire/controller/RulesController.java fájlreferencia

### Osztályok

- class [millionaire.controller.RulesController](#)  
*A játékszabályokat kezelő vezérlő osztály.*

### Csomagok

- package [millionaire.controller](#)

## 7.12. RulesController.java

[Ugrás a fájl dokumentációjához.](#)

```
00001
00002 package millionaire.controller;
00003
00004 import millionaire.model.RulesModel;
00005 import millionaire.view.RulesView;
00006
```

```

00015 public class RulesController {
00016     private final RulesModel model;
00017     private final RulesView view;
00025     public RulesController() {
00026         this.model = new RulesModel();
00027         this.view = new RulesView(this);
00028         initializeView();
00029     }
00030
00031
00039     private void initializeView() {
00040         if (model.isContentLoaded()) {
00041             view.displayRules(model.getRulesContent());
00042         } else {
00043             view.showError("Nem sikerült betölteni a játékszabályokat!");
00044         }
00045     }
00046
00047
00053     public void handleCloseRequest() {
00054         view.closeWindow();
00055     }
00056
00057
00063     public void showRules() {
00064         view.setVisible(true);
00065     }
00066 }

```

## 7.13. src/main/java/millionaire/controller/ScoreboardController.java fájlreferencia

### Osztályok

- class [millionaire.controller.ScoreboardController](#)  
A dícsőséglista kezelését végző vezérlő osztály.

### Csomagok

- package [millionaire.controller](#)

## 7.14. ScoreboardController.java

[Ugrás a fájl dokumentációjához.](#)

```

00001
00002 package millionaire.controller;
00003
00004 import millionaire.model.ScoreboardModel;
00005 import millionaire.model.PlayerModel;
00006 import millionaire.view.ScoreboardView;
00007
00017 public class ScoreboardController {
00018     private final ScoreboardModel model;
00019     private final ScoreboardView view;
00029     public ScoreboardController() {
00030         this.model = new ScoreboardModel();
00031         this.view = new ScoreboardView(this);
00032         refreshScoreboard();
00033     }
00034
00044     public void addScore(PlayerModel player) {
00045         model.addScore(player);
00046         refreshScoreboard();
00047     }
00048
00049
00056     private void refreshScoreboard() {
00057         view.updateScores(model.getPlayers());

```

```

00058     }
00059
00060
00067     public void showScoreboard() {
00068         refreshScoreboard();
00069         view.setVisible(true);
00070     }
00071
00072
00078     public void handleClose() {
00079         view.closeWindow();
00080     }
00081 }

```

## 7.15. src/main/java/millionaire/controller/TimerController.java fájlreferencia

### Osztályok

- class `millionaire.controller.TimerController`  
Az időmérő vezérlője.

### Csomagok

- package `millionaire.controller`

## 7.16. TimerController.java

[Ugrás a fájl dokumentációjához.](#)

```

00001
00002 package millionaire.controller;
00003
00004 import millionaire.model.TimerModel;
00005 import millionaire.view.TimerView;
00006 import java.time.Duration;
00007 import javax.swing.Timer;
00008
00016 public class TimerController {
00017     private final TimerModel model;
00018     private final TimerView view;
00019     private final Timer updateTimer;
00024     public TimerController() {
00025         this.model = new TimerModel();
00026         this.view = new TimerView();
00027
00028         // Timer létrehozása 1 másodperces frissítéssel
00029         this.updateTimer = new Timer(1000, e -> updateTime());
00030     }
00031
00035     private void updateTime() {
00036         model.updateElapsedTime();
00037         view.updateDisplay(model.getElapsedTime());
00038     }
00039
00043     public void startTimer() {
00044         model.start();
00045         updateTimer.start();
00046     }
00047
00051     public void stopTimer() {
00052         model.stop();
00053         updateTimer.stop();
00054     }
00055
00060     public TimerView getView() {
00061         return view;
00062     }
00063
00068     public Duration getElapsedTime() {
00069         return model.getElapsedTime();
00070     }
00071 }

```

## 7.17. src/main/java/millionaire/Main.java fájlreferencia

### Osztályok

- class [millionaire.Main](#)  
*A "Legyen Ön is Milliomos!" kvízzjáték főprogramja.*

### Csomagok

- package [millionaire](#)

## 7.18. Main.java

[Ugrás a fájl dokumentációjához.](#)

```
00001
00002 package millionaire;
00003
00004 import millionaire.controller.MainMenuController;
00005
00015 public class Main {
00016     private static boolean developerMode = false;
00022     public static boolean isDeveloperMode() {
00023         return developerMode;
00024     }
00025
00031     public static void main(String[] args) {
00032         if (args.length > 0 && args[0].equals("dev")) {
00033             developerMode = true;
00034             System.out.println("Developer mode activated");
00035         }
00036
00037         javax.swing.SwingUtilities.invokeLater(() -> {
00038             MainMenuController controller = new MainMenuController();
00039             controller.showMenu();
00040         });
00041     }
00042 }
```

## 7.19. src/main/java/millionaire/model/AddQuestionModel.java fájlreferencia

### Osztályok

- class [millionaire.model.AddQuestionModel](#)  
*Új kérdések hozzáadását kezelő modell osztály.*

### Csomagok

- package [millionaire.model](#)



## 7.20. AddQuestionModel.java

[Ugrás a fájl dokumentációjához.](#)

```

00001
00002 package millionaire.model;
00003
00004 import java.util.List;
00005
00012 public class AddQuestionModel {
00013     private final QuestionRepository repository;
00020     public AddQuestionModel(QuestionRepository repository) {
00021         this.repository = repository;
00022     }
00023
00024
00029     public List<String> getDifficulties() {
00030         return QuestionRepository.DIFFICULTIES;
00031     }
00032
00033
00038     public List<String> getCategories() {
00039         return QuestionRepository.CATEGORIES;
00040     }
00041
00042
00057     public boolean addQuestion(String questionText, String optionA, String optionB, String optionC,
String optionD,
                                String correctAnswer, String difficulty, String category) {
00058
00059         try {
00060             Question question = new Question.QuestionBuilder()
00061                 .difficulty(Integer.parseInt(difficulty))
00062                 .question(questionText)
00063                 .optionA(optionA)
00064                 .optionB(optionB)
00065                 .optionC(optionC)
00066                 .optionD(optionD)
00067                 .correctAnswer(correctAnswer)
00068                 .category(category)
00069                 .build();
00070
00071             repository.addQuestion(question);
00072             return true;
00073         } catch (Exception e) {
00074             return false;
00075         }
00076     }
00077
00078
00092     public boolean validateInputs(String question, String optionA, String optionB, String optionC,
String optionD) {
00093         return !question.trim().isEmpty() &&
00094             !optionA.trim().isEmpty() &&
00095             !optionB.trim().isEmpty() &&
00096             !optionC.trim().isEmpty() &&
00097             !optionD.trim().isEmpty();
00098     }
00099 }

```

## 7.21. src/main/java/millionaire/model/GameModel.java fájlreferencia

### Osztályok

- class [millionaire.model.GameModel](#)  
A játék logikáját és állapotát kezelő osztály.

### Csomagok

- package [millionaire.model](#)

## 7.22. GameModel.java

[Ugrás a fájl dokumentációjához.](#)

```

00001
00002 package millionaire.model;
00003
00004 import java.util.*;
00005 import java.time.Duration;
00006
00007
00017 public class GameModel {
00018
00020     private static final long[] PRIZES = {
00021         40_000_000L, 20_000_000L, 10_000_000L, 5_000_000L, 3_000_000L,
00022         1_500_000L, 800_000L, 500_000L, 300_000L, 200_000L,
00023         100_000L, 50_000L, 25_000L, 10_000L, 5_000L
00024     };
00025
00027     private static final int[] SAFE_SPOTS = {5, 8, 11, 13, 15};
00028
00029     private final QuestionRepository questionRepository;
00030     private final PlayerModel player;
00031     private Question currentQuestion;
00032     private final Random random;
00033     private boolean isGameOver;
00034     private final Duration timeLimit;
00042     public GameModel(QuestionRepository questionRepository, PlayerModel player) {
00043         this.questionRepository = questionRepository;
00044         this.player = player;
00045         this.random = new Random();
00046         this.isGameOver = false;
00047         this.timeLimit = null; // Duration.ofMinutes(X)
00048         this.currentQuestion = selectNextQuestion();
00049     }
00050
00051
00057     public Question selectNextQuestion() {
00058         List<Question> questions =
00059             questionRepository.getQuestionsByDifficulty(player.getCurrentLevel());
00060         if (questions.isEmpty()) {
00061             throw new IllegalStateException("No questions available for level " +
00062                 player.getCurrentLevel());
00063         }
00064         currentQuestion = questions.get(random.nextInt(questions.size()));
00065         return currentQuestion;
00066     }
00072     public GameState processAnswer(String selectedAnswer) {
00073         if (isGameOver || currentQuestion == null) {
00074             return GameState.LOST;
00075         }
00076
00077         boolean isCorrect = currentQuestion.getCorrectAnswer().equals(selectedAnswer);
00078
00079         if (!isCorrect) {
00080             isGameOver = true;
00081             return GameState.LOST;
00082         }
00083
00084         if (player.getCurrentLevel() == 15) {
00085             isGameOver = true;
00086             return GameState.WON;
00087         }
00088
00089         player.incrementLevel();
00090         currentQuestion = selectNextQuestion();
00091         return GameState.CORRECT;
00092     }
00093
00094
00099     public long getLastSafePrize() {
00100         for (int i = SAFE_SPOTS.length - 1; i >= 0; i--) {
00101             if (player.getCurrentLevel() >= SAFE_SPOTS[i]) {
00102                 return PRIZES[15 - SAFE_SPOTS[i]];
00103             }
00104         }
00105         return 0L;
00106     }
00107
00108
00113     public long getCurrentPrize() {
00114         return PRIZES[15 - player.getCurrentLevel()];
00115     }
00116

```

```

00117
00119     public PlayerModel getPlayer() { return player; }
00120
00121
00123     public Question getCurrentQuestion() { return currentQuestion; }
00124
00125
00127     public boolean isGameOver() { return isGameOver; }
00128
00129
00134     public static List<PrizeLevel> getPrizeLevels() {
00135         List<PrizeLevel> levels = new ArrayList<>();
00136         for (int i = 0; i < PRIZES.length; i++) {
00137             int level = 15 - i;
00138             long prize = PRIZES[i];
00139             boolean isSafeSpot = Arrays.stream(SAFE_SPOTS).anyMatch(spot -> spot == level);
00140             levels.add(new PrizeLevel(level, prize, isSafeSpot));
00141         }
00142         return levels;
00143     }
00144
00145
00150     public void updatePlayTime(Duration elapsedTime) {
00151         player.setPlaytime(elapsedTime);
00152     }
00153
00154
00160     public boolean isTimeUp(Duration currentTime) {
00161         return timeLimit != null && currentTime.compareTo(timeLimit) > 0;
00162     }
00163 }

```

## 7.23. src/main/java/millionaire/model/GameState.java fájlreferencia

### Osztályok

- enum [millionaire.model.GameState](#)  
*Enum osztály, amely a játék állapotait reprezentálja.*

### Csomagok

- package [millionaire.model](#)

## 7.24. GameState.java

[Ugrás a fájl dokumentációjához.](#)

```

00001
00002 package millionaire.model;
00003
00007 public enum GameState {
00012     CORRECT,
00013
00018     LOST,
00019
00024     WON;
00025
00030     public boolean isGameEnding() {
00031         return this == LOST || this == WON;
00032     }
00033 }

```

## 7.25. src/main/java/millionaire/model/LifelineModel.java fájlreferencia

### Osztályok

- class [millionaire.model.LifelineModel](#)  
*Segítségek (lifeline) logikáját kezelő osztály.*

## Csomagok

- package [millionaire.model](#)

## 7.26. LifelineModel.java

[Ugrás a fájl dokumentációjához.](#)

```

00001
00002 package millionaire.model;
00003
00004 import java.util.*;
00005
00012 public class LifelineModel {
00013     private final Random random = new Random();
00014     private boolean fiftyFiftyUsed = false;
00015     private boolean audienceHelpUsed = false;
00028     public Map<String, Boolean> getFiftyFiftyResult(Question question) {
00029         if (fiftyFiftyUsed) {
00030             throw new IllegalStateException("Fifty-fifty lifeline already used");
00031         }
00032
00033         String correctAnswer = question.getCorrectAnswer();
00034         Map<String, Boolean> visibilityMap = new HashMap<>();
00035         List<String> options = new ArrayList<>(Arrays.asList("A", "B", "C", "D"));
00036
00037         // Kivesszük a jó választ a lehetőségek közül..
00038         options.remove(correctAnswer);
00039         // Random választunk egy rossz választ amit megtartunk..
00040         String keepWrongAnswer = options.remove(random.nextInt(options.size()));
00041
00042         // Beállítjuk a válaszlehetőségek láthatóságát..
00043         for (String option : Arrays.asList("A", "B", "C", "D")) {
00044             visibilityMap.put(option, option.equals(correctAnswer) || option.equals(keepWrongAnswer));
00045         }
00046
00047         fiftyFiftyUsed = true;
00048         return visibilityMap;
00049     }
00050
00051
00063     public Map<String, Integer> getAudienceResult(Question question, int currentLevel) {
00064         if (audienceHelpUsed) {
00065             throw new IllegalStateException("Audience help already used");
00066         }
00067
00068         int correctPercentage = calculateCorrectPercentage(currentLevel);
00069         Map<String, Integer> percentages = distributePercentages(question.getCorrectAnswer(),
correctPercentage);
00070
00071         audienceHelpUsed = true;
00072         return percentages;
00073     }
00074
00075
00087     private int calculateCorrectPercentage(int currentLevel) {
00088         if (currentLevel <= 5) {
00089             return random.nextInt(71, 81);
00090         } else if (currentLevel <= 10) {
00091             return random.nextInt(61, 71);
00092         } else {
00093             return random.nextInt(51, 61);
00094         }
00095     }
00096
00097
00105     private Map<String, Integer> distributePercentages(String correctAnswer, int correctPercentage) {
00106         Map<String, Integer> percentages = new LinkedHashMap<>();
00107         List<String> wrongAnswers = new ArrayList<>(Arrays.asList("A", "B", "C", "D"));
00108         wrongAnswers.remove(correctAnswer);
00109
00110         percentages.put(correctAnswer, correctPercentage);
00111
00112         int remaining = 100 - correctPercentage;
00113         for (int i = 0; i < wrongAnswers.size(); i++) {
00114             int percentage;
00115             if (i == wrongAnswers.size() - 1) {
00116                 percentage = remaining;
00117             } else {
00118                 percentage = Math.min(remaining - (wrongAnswers.size() - i - 1),
00119                     random.nextInt(5, 26));

```

```
00120         }
00121         percentages.put(wrongAnswers.get(i), percentage);
00122         remaining -= percentage;
00123     }
00124
00125     return new TreeMap<>(percentages);
00126 }
00127
00128
00133 public boolean isFiftyFiftyAvailable() {
00134     return !fiftyFiftyUsed;
00135 }
00136
00137
00142 public boolean isAudienceHelpAvailable() {
00143     return !audienceHelpUsed;
00144 }
00145 }
```

## 7.27. src/main/java/millionaire/model/MainMenuModel.java fájlreferencia

### Osztályok

- class [millionaire.model.MainMenuModel](#)

*A főmenü modell osztálya.*

### Csomagok

- package [millionaire.model](#)

## 7.28. MainMenuModel.java

[Ugrás a fájl dokumentációjához.](#)

```
00001
00002 package millionaire.model;
00003
00010 public class MainMenuModel {
00011     private final QuestionRepository questionRepository;
00018     public MainMenuModel() {
00019         this.questionRepository = new QuestionRepository();
00020     }
00021
00026     public QuestionRepository getQuestionRepository() {
00027         return questionRepository;
00028     }
00029 }
```

## 7.29. src/main/java/millionaire/model/PlayerModel.java fájlreferencia

### Osztályok

- class [millionaire.model.PlayerModel](#)

### Csomagok

- package [millionaire.model](#)

## 7.30. PlayerModel.java

[Ugrás a fájl dokumentációjához.](#)

```

00001
00002 package millionaire.model;
00003
00004 import java.text.NumberFormat;
00005 import java.time.Duration;
00006 import java.time.Instant;
00007 import java.util.Locale;
00008
00009 public class PlayerModel implements Comparable<PlayerModel> {
00010     private String name;
00011     private int currentLevel;
00012     private Duration playtime;
00013     private Instant startTime;
00014     private long prize;
00021     public PlayerModel(String name) {
00022         this.name = name;
00023         this.currentLevel = 1;
00024         this.playtime = Duration.ZERO;
00025         this.startTime = Instant.now();
00026         this.prize = 0;
00027     }
00028
00029
00037     public PlayerModel(String name, int level, long prize, Duration playtime) {
00038         this.name = name;
00039         this.currentLevel = level;
00040         this.prize = prize;
00041         this.playtime = playtime;
00042         this.startTime = Instant.now();
00043     }
00044
00045
00050     public String getName() { return name; }
00051
00052
00057     public int getCurrentLevel() { return currentLevel; }
00058
00059
00064     public Duration getPlaytime() { return playtime; }
00065
00066
00071     public Instant getStartTime() { return startTime; }
00072
00073
00078     public long getPrize() { return prize; }
00079
00080
00085     public void setName(String name) { this.name = name; }
00086
00087
00092     public void setCurrentLevel(int currentLevel) { this.currentLevel = currentLevel; }
00093
00094
00099     public void setPlaytime(Duration playtime) { this.playtime = playtime; }
00100
00105     public void setPrize(long prize) { this.prize = prize; }
00106
00107
00111     public void updatePlaytime() {
00112         if (this.playtime == null || this.playtime.equals(Duration.ZERO)) {
00113             this.playtime = Duration.between(startTime, Instant.now());
00114         }
00115         // Ha a Timer már beállította a játékidjét, akkor ne frissítsük
00116     }
00117
00118
00122     public void incrementLevel() { currentLevel++; }
00123
00124
00129     public String getFormattedPrize() {
00130         return NumberFormat.getNumberInstance(new Locale("hu", "HU")).format(prize) + " Ft";
00131     }
00132
00133
00138     public String getFormattedPlaytime() {
00139         if (playtime == null) {
00140             return "00:00";
00141         }
00142         long totalSeconds = playtime.getSeconds();
00143         long minutes = totalSeconds / 60;
00144         long seconds = totalSeconds % 60;
00145         return String.format("%02d:%02d", minutes, seconds);

```

```

00146     }
00147
00148
00154     @Override
00155     public int compareTo(PlayerModel other) {
00156         int levelComparison = Integer.compare(other.currentLevel, this.currentLevel);
00157         if (levelComparison != 0) {
00158             return levelComparison;
00159         }
00160
00161         return this.playtime.compareTo(other.playtime);
00162     }
00163
00164
00170     @Override
00171     public boolean equals(Object obj) {
00172         if (!(obj instanceof PlayerModel)) return false;
00173         PlayerModel other = (PlayerModel) obj;
00174         return this.name.equals(other.name);
00175     }
00176 }

```

## 7.31. src/main/java/millionaire/model/PrizeLevel.java fájlreferencia

### Osztályok

- class [millionaire.model.PrizeLevel](#)  
*Nyereményszintet reprezentáló osztály.*

### Csomagok

- package [millionaire.model](#)

## 7.32. PrizeLevel.java

[Ugrás a fájl dokumentációjához.](#)

```

00001
00002 package millionaire.model;
00003
00013 public class PrizeLevel {
00014     private final int level;
00015     private final long prize;
00016     private final boolean isSafeSpot;
00025     public PrizeLevel(int level, long prize, boolean isSafeSpot) {
00026         this.level = level;
00027         this.prize = prize;
00028         this.isSafeSpot = isSafeSpot;
00029     }
00030
00035     public int getLevel() { return level; }
00036
00041     public long getPrize() { return prize; }
00042
00047     public boolean isSafeSpot() { return isSafeSpot; }
00048 }

```

## 7.33. src/main/java/millionaire/model/Question.java fájlreferencia

### Osztályok

- class [millionaire.model.Question](#)  
*Egy kérdést reprezentáló osztály.*
- class [millionaire.model.Question.QuestionBuilder](#)  
*Builder osztály a Question példányok létrehozásához.*

## Csomagok

- package [millionaire.model](#)

## 7.34. Question.java

[Ugrás a fájl dokumentációjához.](#)

```
00001
00002 package millionaire.model;
00003
00016 public class Question {
00017     private final int difficulty;
00018     private final String question;
00019     private final String optionA;
00020     private final String optionB;
00021     private final String optionC;
00022     private final String optionD;
00023     private final String correctAnswer;
00024     private final String category;
00031     private Question(QuestionBuilder builder) {
00032         this.difficulty = builder.difficulty;
00033         this.question = builder.question;
00034         this.optionA = builder.optionA;
00035         this.optionB = builder.optionB;
00036         this.optionC = builder.optionC;
00037         this.optionD = builder.optionD;
00038         this.correctAnswer = builder.correctAnswer;
00039         this.category = builder.category;
00040     }
00041
00042
00047     public int getDifficulty() { return difficulty; }
00048
00049
00054     public String getQuestion() { return question; }
00055
00056
00061     public String getOptionA() { return optionA; }
00062
00063
00068     public String getOptionB() { return optionB; }
00069
00070
00075     public String getOptionC() { return optionC; }
00076
00077
00082     public String getOptionD() { return optionD; }
00083
00084
00089     public String getCorrectAnswer() { return correctAnswer; }
00090
00091
00096     public String getCategory() { return category; }
00097
00098
00099
00100
00101
00108     public static class QuestionBuilder {
00109         private int difficulty;
00110         private String question;
00111         private String optionA;
00112         private String optionB;
00113         private String optionC;
00114         private String optionD;
00115         private String correctAnswer;
00116         private String category;
00124         public QuestionBuilder difficulty(int difficulty) {
00125             this.difficulty = difficulty;
00126             return this;
00127         }
00128
00129
00135         public QuestionBuilder question(String question) {
00136             this.question = question;
00137             return this;
00138         }
00139
00140
00146         public QuestionBuilder optionA(String optionA) {
```



```

00147         this.optionA = optionA;
00148         return this;
00149     }
00150
00151
00157     public QuestionBuilder optionB(String optionB) {
00158         this.optionB = optionB;
00159         return this;
00160     }
00161
00162
00168     public QuestionBuilder optionC(String optionC) {
00169         this.optionC = optionC;
00170         return this;
00171     }
00172
00173
00179     public QuestionBuilder optionD(String optionD) {
00180         this.optionD = optionD;
00181         return this;
00182     }
00183
00184
00190     public QuestionBuilder correctAnswer(String correctAnswer) {
00191         this.correctAnswer = correctAnswer;
00192         return this;
00193     }
00194
00195
00201     public QuestionBuilder category(String category) {
00202         this.category = category;
00203         return this;
00204     }
00205
00206
00212     public Question build() {
00213         validateQuestionData();
00214         return new Question(this);
00215     }
00216
00217
00222     private void validateQuestionData() {
00223         if (question == null || question.trim().isEmpty()) {
00224             throw new IllegalStateException("Question cannot be empty");
00225         }
00226         if (optionA == null || optionA.trim().isEmpty() ||
00227             optionB == null || optionB.trim().isEmpty() ||
00228             optionC == null || optionC.trim().isEmpty() ||
00229             optionD == null || optionD.trim().isEmpty()) {
00230             throw new IllegalStateException("All options must be provided");
00231         }
00232         if (correctAnswer == null || !correctAnswer.matches("[A-D]")) {
00233             throw new IllegalStateException("Correct answer must be A, B, C, or D");
00234         }
00235         if (difficulty < 1 || difficulty > 15) {
00236             throw new IllegalStateException("Difficulty must be between 1 and 15");
00237         }
00238         if (category == null || category.trim().isEmpty()) {
00239             throw new IllegalStateException("Category must be provided");
00240         }
00241     }
00242 }
00243 }

```

## 7.35. src/main/java/millionaire/model/QuestionRepository.java fájlreferencia

### Osztályok

- class [millionaire.model.QuestionRepository](#)  
*Kérdések perzisztens tárolását végző osztály.*

### Csomagok

- package [millionaire.model](#)

## 7.36. QuestionRepository.java

[Ugrás a fájl dokumentációjához.](#)

```

00001
00002 package millionaire.model;
00003
00004 import com.google.gson.Gson;
00005 import com.google.gson.GsonBuilder;
00006 import com.google.gson.JsonArray;
00007 import com.google.gson.JsonObject;
00008 import com.google.gson.JsonElement;
00009 import com.google.gson.JsonParser;
00010 import java.io.*;
00011 import java.nio.file.*;
00012 import java.util.*;
00013 import java.util.stream.Collectors;
00014
00024 public class QuestionRepository {
00025     private static final String JSON_FILE_PATH = "src/main/resources/loim.json";
00026     private final List<Question> questions;
00027     private final Gson gson;
00031     public static final List<String> CATEGORIES = Arrays.asList(
00032         "TÖRTÉNELEM", "SPORT", "FÖLDRAJZ", "IRODALOM", "TUDOMÁNY", "ZENE", "FILM", "KONYHA",
"ÁLTALÁNOS"
00033     );
00034
00035
00037     public static final List<String> DIFFICULTIES = Arrays.asList(
00038         "1", "2", "3", "4", "5", "6", "7", "8", "9", "10", "11", "12", "13", "14", "15"
00039     );
00040
00041
00047     public QuestionRepository() {
00048         this.questions = new ArrayList<>();
00049         this.gson = new GsonBuilder().setPrettyPrinting().create();
00050         loadQuestions();
00051     }
00052
00053
00063     private void loadQuestions() {
00064         try (Reader reader = Files.newBufferedReader(Paths.get(JSON_FILE_PATH))) {
00065             JsonArray jsonArray = JsonParser.parseReader(reader).getAsJsonArray();
00066
00067             for (JsonElement element : jsonArray) {
00068                 JsonObject questionObj = element.getAsJsonObject();
00069                 Question question = new Question.QuestionBuilder()
00070                     .difficulty(questionObj.get("Nehézség").getAsInt())
00071                     .question(questionObj.get("Kérdés").getAsString())
00072                     .optionA(questionObj.get("A").getAsString())
00073                     .optionB(questionObj.get("B").getAsString())
00074                     .optionC(questionObj.get("C").getAsString())
00075                     .optionD(questionObj.get("D").getAsString())
00076                     .correctAnswer(questionObj.get("Válasz").getAsString())
00077                     .category(questionObj.get("Kategória").getAsString())
00078                     .build();
00079                 questions.add(question);
00080             }
00081         } catch (IOException e) {
00082             System.err.println("Error loading questions: " + e.getMessage());
00083         }
00084     }
00085
00086
00097     private void saveQuestions() {
00098         try {
00099             JsonArray jsonArray = new JsonArray();
00100             for (Question question : questions) {
00101                 JsonObject questionObj = new JsonObject();
00102                 questionObj.addProperty("Nehézség", question.getDifficulty());
00103                 questionObj.addProperty("Kérdés", question.getQuestion());
00104                 questionObj.addProperty("A", question.getOptionA());
00105                 questionObj.addProperty("B", question.getOptionB());
00106                 questionObj.addProperty("C", question.getOptionC());
00107                 questionObj.addProperty("D", question.getOptionD());
00108                 questionObj.addProperty("Válasz", question.getCorrectAnswer());
00109                 questionObj.addProperty("Kategória", question.getCategory());
00110                 jsonArray.add(questionObj);
00111             }
00112
00113             Files.createDirectories(Paths.get(JSON_FILE_PATH).getParent());
00114             Files.write(Paths.get(JSON_FILE_PATH),
00115                 gson.toJson(jsonArray).getBytes(),
00116                 StandardOpenOption.CREATE,
00117                 StandardOpenOption.TRUNCATE_EXISTING);
00118         } catch (IOException e) {

```

```

00119         throw new RuntimeException("Error saving questions: " + e.getMessage());
00120     }
00121 }
00122
00123
00128 public void addQuestion(Question question) {
00129     questions.add(question);
00130     saveQuestions();
00131 }
00132
00133
00139 public List<Question> getQuestionsByDifficulty(int difficulty) {
00140     return questions.stream()
00141         .filter(q -> q.getDifficulty() == difficulty)
00142         .collect(Collectors.toList());
00143 }
00144
00145
00151 public List<Question> getQuestionsByCategory(String category) {
00152     return questions.stream().filter(q ->
00153         q.getCategory().equals(category)).collect(Collectors.toList());
00154 }
00155
00160 public List<Question> getAllQuestions() {
00161     return new ArrayList<>(questions);
00162 }
00163 }

```

## 7.37. src/main/java/millionaire/model/RulesModel.java fájlreferencia

### Osztályok

- class [millionaire.model.RulesModel](#)  
*Játékszabályok kezelését végző osztály.*

### Csomagok

- package [millionaire.model](#)

## 7.38. RulesModel.java

[Ugrás a fájl dokumentációjához.](#)

```

00001
00002 package millionaire.model;
00003
00004 import java.io.IOException;
00005 import java.nio.charset.StandardCharsets;
00006 import java.nio.file.Files;
00007 import java.nio.file.Path;
00008 import java.nio.file.Paths;
00009
00018 public class RulesModel {
00020     private static final String RULES_FILE_PATH = "src/main/resources/rules.txt";
00021
00022     private String rulesContent;
00023
00024     public RulesModel() {
00030         loadRulesContent();
00031     }
00032
00033     private void loadRulesContent() {
00040         try {
00041             Path path = Paths.get(RULES_FILE_PATH);
00042             rulesContent = Files.readString(path, StandardCharsets.UTF_8);
00043         } catch (IOException e) {
00044             rulesContent = "Failed to load rules: " + e.getMessage();
00045         }
00046     }
00047 }
00048

```

```

00053     public String getRulesContent() {
00054         return rulesContent;
00055     }
00056
00061     public boolean isContentLoaded() {
00062         return rulesContent != null && !rulesContent.startsWith("Failed to load rules");
00063     }
00064 }

```

## 7.39. src/main/java/millionaire/model/ScoreboardModel.java fájlreferencia

### Osztályok

- class [millionaire.model.ScoreboardModel](#)  
*Dicsőségtábla kezelését végző osztály.*

### Csomagok

- package [millionaire.model](#)

## 7.40. ScoreboardModel.java

[Ugrás a fájl dokumentációjához.](#)

```

00001
00002 package millionaire.model;
00003
00004 import com.google.gson.Gson;
00005 import com.google.gson.GsonBuilder;
00006 import com.google.gson.JsonArray;
00007 import com.google.gson.JsonObject;
00008 import com.google.gson.JsonElement;
00009 import com.google.gson.JsonParser;
00010 import java.io.*;
00011 import java.nio.file.*;
00012 import java.time.Duration;
00013 import java.util.*;
00014
00024 public class ScoreboardModel {
00026     private static final String JSON_FILE_PATH = "src/main/resources/scoreboard.json";
00027
00029     private List<PlayerModel> players;
00030
00032     private final Gson gson;
00033
00039     public ScoreboardModel() {
00040         this.players = new ArrayList<>();
00041         this.gson = new GsonBuilder().setPrettyPrinting().create();
00042         initializeAndLoadScores();
00043     }
00044
00054     private void initializeAndLoadScores() {
00055         try {
00056             Files.createDirectories(Paths.get("src/main/resources"));
00057
00058             Path filePath = Paths.get(JSON_FILE_PATH);
00059             if (!Files.exists(filePath)) {
00060                 Files.write(filePath, [].getBytes());
00061                 return;
00062             }
00063
00064             String jsonContent = Files.readString(filePath);
00065             if (jsonContent.trim().isEmpty()) {
00066                 Files.write(filePath, [].getBytes());
00067                 return;
00068             }
00069
00070             JsonElement jsonElement = JsonParser.parseString(jsonContent);
00071             if (!jsonElement.isJsonArray()) {
00072                 Files.write(filePath, [].getBytes());
00073                 return;

```

```

00074         }
00075
00076         JSONArray jsonArray = jsonElement.getAsJsonArray();
00077         for (JsonElement element : jsonArray) {
00078             JsonObject playerObj = element.getAsJsonObject();
00079             PlayerModel player = new PlayerModel(
00080                 playerObj.get("name").getString(),
00081                 playerObj.get("level").getAsInt(),
00082                 playerObj.get("prize").getAsLong(),
00083                 Duration.ofSeconds(playerObj.get("playtime").getAsLong())
00084             );
00085             players.add(player);
00086         }
00087
00088         sortPlayers();
00089     } catch (Exception e) {
00090         System.err.println("Error initializing scoreboard: " + e.getMessage());
00091         players = new ArrayList<>();
00092         try {
00093             Files.write(Paths.get(JSON_FILE_PATH), "".getBytes());
00094         } catch (IOException ioe) {
00095             System.err.println("Could not create scoreboard file: " + ioe.getMessage());
00096         }
00097     }
00098 }
00099
00100 public void addScore(PlayerModel newPlayer) {
00101     Optional<PlayerModel> existingPlayer = players.stream()
00102         .filter(p -> p.getName().equals(newPlayer.getName()))
00103         .findFirst();
00104
00105     if (existingPlayer.isPresent()) {
00106         PlayerModel old = existingPlayer.get();
00107         if (shouldReplaceScore(newPlayer, old)) {
00108             players.remove(old);
00109             players.add(newPlayer);
00110         }
00111     } else {
00112         players.add(newPlayer);
00113     }
00114
00115     sortPlayers();
00116     saveScores();
00117 }
00118
00119 private boolean shouldReplaceScore(PlayerModel newPlayer, PlayerModel oldPlayer) {
00120     return newPlayer.getCurrentLevel() > oldPlayer.getCurrentLevel() ||
00121         (newPlayer.getCurrentLevel() == oldPlayer.getCurrentLevel() &&
00122             newPlayer.getPlaytime().compareTo(oldPlayer.getPlaytime()) < 0);
00123 }
00124
00125 private void sortPlayers() {
00126     Collections.sort(players);
00127 }
00128
00129 public List<PlayerModel> getPlayers() {
00130     return new ArrayList<>(players);
00131 }
00132
00133 private void saveScores() {
00134     try {
00135         JSONArray jsonArray = new JSONArray();
00136         for (PlayerModel player : players) {
00137             JsonObject playerObj = new JsonObject();
00138             playerObj.addProperty("name", player.getName());
00139             playerObj.addProperty("level", player.getCurrentLevel());
00140             playerObj.addProperty("prize", player.getPrize());
00141             // Store the actual seconds value
00142             playerObj.addProperty("playtime", player.getPlaytime().getSeconds());
00143             jsonArray.add(playerObj);
00144         }
00145
00146         Files.createDirectories(Paths.get("src/main/resources"));
00147
00148         Files.write(
00149             Paths.get(JSON_FILE_PATH),
00150             gson.toJson(jsonArray).getBytes(),
00151             StandardOpenOption.CREATE,
00152             StandardOpenOption.TRUNCATE_EXISTING,
00153             StandardOpenOption.WRITE
00154         );
00155     } catch (IOException e) {
00156         System.err.println("Error saving scoreboard: " + e.getMessage());
00157     }
00158 }
00159
00160 }
00161
00162 }

```

## 7.41. src/main/java/millionaire/model/TimerModel.java fájlreferencia

### Osztályok

- class [millionaire.model.TimerModel](#)  
*Az időmérő adatmodellje.*

### Csomagok

- package [millionaire.model](#)

## 7.42. TimerModel.java

[Ugrás a fájl dokumentációjához.](#)

```
00001
00002 package millionaire.model;
00003
00004 import java.time.Duration;
00005 import java.time.Instant;
00006
00014 public class TimerModel {
00015     private final Instant startTime;
00016     private Duration elapsedTime;
00017     private boolean isRunning;
00022     public TimerModel() {
00023         this.startTime = Instant.now();
00024         this.elapsedTime = Duration.ZERO;
00025         this.isRunning = false;
00026     }
00027
00031     public void updateElapsedTime() {
00032         if (isRunning) {
00033             elapsedTime = Duration.between(startTime, Instant.now());
00034         }
00035     }
00036
00040     public void start() {
00041         isRunning = true;
00042     }
00043
00047     public void stop() {
00048         isRunning = false;
00049     }
00050
00055     public Duration getElapsedTime() {
00056         return elapsedTime;
00057     }
00058
00063     public Instant getStartTime() {
00064         return startTime;
00065     }
00066 }
```

## 7.43. src/main/java/millionaire/view/AddQuestionView.java fájlreferencia

### Osztályok

- class [millionaire.view.AddQuestionView](#)  
*Új kérdések hozzáadására szolgáló grafikus felület.*

### Csomagok

- package [millionaire.view](#)

## 7.44. AddQuestionView.java

[Ugrás a fájl dokumentációjához.](#)

```

00001
00002 package millionaire.view;
00003
00004 import millionaire.controller.AddQuestionController;
00005 import javax.swing.*;
00006 import java.awt.*;
00007 import java.util.List;
00008
00018 public class AddQuestionView extends JFrame {
00019     private static final int WINDOW_WIDTH = 800;
00020     private static final int WINDOW_HEIGHT = 600;
00021     private static final Color BACKGROUND_COLOR = new Color(90, 90, 90);
00022     private static final Color BUTTON_COLOR = new Color(70, 130, 180);
00023     private static final Color BUTTON_HOVER_COLOR = new Color(100, 149, 237);
00024     private static final Color TEXT_COLOR = Color.WHITE;
00026     private final AddQuestionController controller;
00028     private JTextField questionField;
00029     private JTextField optionAField;
00030     private JTextField optionBField;
00031     private JTextField optionCField;
00032     private JTextField optionDField;
00033     private JComboBox<String> correctAnswerCombo;
00034     private JComboBox<String> difficultyCombo;
00035     private JComboBox<String> categoryCombo;
00042     public AddQuestionView(AddQuestionController controller) {
00043         this.controller = controller;
00044         setupWindow();
00045         setupComponents();
00046         pack();
00047         setLocationRelativeTo(null);
00048     }
00049
00050
00061     private void setupWindow() {
00062         setTitle("Új kérdés hozzáadása");
00063         setSize(WINDOW_WIDTH, WINDOW_HEIGHT);
00064         setDefaultCloseOperation(DISPOSE_ON_CLOSE);
00065         setResizable(false);
00066     }
00067
00068
00077     private void setupComponents() {
00078         JPanel mainPanel = new JPanel();
00079         mainPanel.setLayout(new BorderLayout(mainPanel, BorderLayout.Y_AXIS));
00080         mainPanel.setBorder(BorderFactory.createEmptyBorder(20, 20, 20, 20));
00081         mainPanel.setBackground(BACKGROUND_COLOR);
00082
00083         addTitle(mainPanel);
00084         addFormPanel(mainPanel);
00085         addButtonPanel(mainPanel);
00086
00087         add(mainPanel);
00088     }
00089
00090
00099     private void addTitle(JPanel mainPanel) {
00100         JLabel titleLabel = new JLabel("Új kérdés hozzáadása");
00101         titleLabel.setFont(new Font("Arial", Font.BOLD, 24));
00102         titleLabel.setForeground(TEXT_COLOR);
00103         titleLabel.setAlignmentX(Component.CENTER_ALIGNMENT);
00104         mainPanel.add(titleLabel);
00105         mainPanel.add(Box.createRigidArea(new Dimension(0, 20)));
00106     }
00107
00108
00118     private void addFormPanel(JPanel mainPanel) {
00119         JPanel formPanel = new JPanel(new GridBagLayout());
00120         formPanel.setBackground(BACKGROUND_COLOR);
00121         GridBagConstraints gbc = new GridBagConstraints();
00122         gbc.fill = GridBagConstraints.HORIZONTAL;
00123         gbc.insets = new Insets(5, 5, 5, 5);
00124
00125         questionField = new JTextField(40);
00126         optionAField = new JTextField(40);
00127         optionBField = new JTextField(40);
00128         optionCField = new JTextField(40);
00129         optionDField = new JTextField(40);
00130         correctAnswerCombo = new JComboBox<>(new String[]{"A", "B", "C", "D"});
00131         difficultyCombo = new JComboBox<>(controller.getDifficulties().toArray(new String[0]));
00132         categoryCombo = new JComboBox<>(controller.getCategories().toArray(new String[0]));
00133
00134         addFormRow(formPanel, gbc, 0, "Kérdés:", questionField);

```

```

00135         addFormRow(formPanel, gbc, 1, "A válasz:", optionAField);
00136         addFormRow(formPanel, gbc, 2, "B válasz:", optionBField);
00137         addFormRow(formPanel, gbc, 3, "C válasz:", optionCField);
00138         addFormRow(formPanel, gbc, 4, "D válasz:", optionDField);
00139         addFormRow(formPanel, gbc, 5, "Helyes válasz:", correctAnswerCombo);
00140         addFormRow(formPanel, gbc, 6, "Nehézség:", difficultyCombo);
00141         addFormRow(formPanel, gbc, 7, "Kategória:", categoryCombo);
00142
00143         mainPanel.add(formPanel);
00144         mainPanel.add(Box.createRigidArea(new Dimension(0, 20)));
00145     }
00146
00147
00148
00149
00150     private void addFormRow(JPanel panel, GridBagConstraints gbc, int row, String labelText,
00151                             JComponent component) {
00152         gbc.gridx = 0;
00153         gbc.gridy = row;
00154         gbc.gridwidth = 1;
00155
00156         JLabel label = new JLabel(labelText);
00157         label.setForeground(TEXT_COLOR);
00158         label.setPreferredSize(new Dimension(100, 25));
00159         panel.add(label, gbc);
00160
00161         gbc.gridx = 1;
00162         gbc.gridwidth = 2;
00163         panel.add(component, gbc);
00164     }
00165
00166
00167
00168     private void addButtonPanel(JPanel mainPanel) {
00169         JPanel buttonPanel = new JPanel();
00170         buttonPanel.setBackground(BACKGROUND_COLOR);
00171
00172         JButton submitButton = createStyledButton("Mentés");
00173         JButton cancelButton = createStyledButton("Mégse");
00174
00175         submitButton.addActionListener(e -> handleSubmit());
00176         cancelButton.addActionListener(e -> handleCancel());
00177
00178         buttonPanel.add(submitButton);
00179         buttonPanel.add(Box.createRigidArea(new Dimension(10, 0)));
00180         buttonPanel.add(cancelButton);
00181
00182         mainPanel.add(buttonPanel);
00183     }
00184
00185
00186
00187
00188     private JButton createStyledButton(String text) {
00189         JButton button = new JButton(text);
00190         button.setPreferredSize(new Dimension(120, 40));
00191         button.setFont(new Font("Arial", Font.PLAIN, 14));
00192         button.setFocusPainted(false);
00193         button.setBackground(BUTTON_COLOR);
00194         button.setForeground(TEXT_COLOR);
00195         button.setBorder(BorderFactory.createRaisedBevelBorder());
00196
00197         button.addMouseListener(new java.awt.event.MouseAdapter() {
00198             @Override
00199             public void mouseEntered(java.awt.event.MouseEvent evt) {
00200                 button.setBackground(BUTTON_HOVER_COLOR);
00201             }
00202
00203             @Override
00204             public void mouseExited(java.awt.event.MouseEvent evt) {
00205                 button.setBackground(BUTTON_COLOR);
00206             }
00207         });
00208
00209         return button;
00210     }
00211
00212
00213
00214     private void handleSubmit() {
00215         controller.handleSubmit(
00216             questionField.getText(),
00217             optionAField.getText(),
00218             optionBField.getText(),
00219             optionCField.getText(),
00220             optionDField.getText(),
00221             (String) correctAnswerCombo.getSelectedItem(),
00222             (String) difficultyCombo.getSelectedItem(),
00223             (String) categoryCombo.getSelectedItem()
00224         );
00225     }
00226
00227
00228
00229
00230
00231
00232
00233
00234
00235
00236
00237
00238
00239
00240
00241
00242
00243
00244
00245
00246
00247
00248
00249
00250
00251
00252
00253
00254
00255
00256
00257
00258

```



```

00264     private void handleCancel() {
00265         controller.handleCancel();
00266     }
00267
00268
00276     public void showError(String message) {
00277         JOptionPane.showMessageDialog(this, message, "Hiba", JOptionPane.ERROR_MESSAGE);
00278     }
00279
00280
00288     public void showSuccess(String message) {
00289         JOptionPane.showMessageDialog(this, message, "Siker", JOptionPane.INFORMATION_MESSAGE);
00290     }
00291
00292
00298     public void clearFields() {
00299         questionField.setText("");
00300         optionAField.setText("");
00301         optionBField.setText("");
00302         optionCField.setText("");
00303         optionDField.setText("");
00304         correctAnswerCombo.setSelectedIndex(0);
00305         difficultyCombo.setSelectedIndex(0);
00306         categoryCombo.setSelectedIndex(0);
00307     }
00308
00309
00315     public void closeWindow() {
00316         dispose();
00317     }
00318 }

```

## 7.45. src/main/java/millionaire/view/MapView.java fájlreferencia

### Osztályok

- class [millionaire.view.MapView](#)

*A játék fő grafikus felülete.*

### Csomagok

- package [millionaire.view](#)

## 7.46. MapView.java

[Ugrás a fájl dokumentációjához.](#)

```

00001
00002 package millionaire.view;
00003
00004 import millionaire.Main;
00005 import millionaire.controller.GameController;
00006 import millionaire.controller.LifelineController;
00007 import millionaire.controller.TimerController;
00008 import millionaire.model.*;
00009 import javax.swing.*;
00010 import java.awt.*;
00011 import java.util.*;
00012 import java.util.List;
00013 import java.util.Timer;
00014
00026 public class MapView extends JFrame {
00027     private static final int WINDOW_WIDTH = 1440;
00028     private static final int WINDOW_HEIGHT = 720;
00029     private static final Color BACKGROUND_COLOR = new Color(90, 90, 90);
00030     private static final Color BUTTON_COLOR = new Color(70, 130, 180);
00031     private static final Color BUTTON_HOVER_COLOR = new Color(100, 149, 237);
00032     private static final Color TEXT_COLOR = Color.WHITE;
00033     private static final Color HIGHLIGHT_COLOR = new Color(255, 215, 0);
00034     private static final Color CORRECT_ANSWER_COLOR = new Color(50, 205, 50);
00035     private static final Color WRONG_ANSWER_COLOR = new Color(220, 20, 20);

```

```

00037     private final GameController controller;
00038     private final TimerController timerController;
00039     private JLabel questionLabel;
00040     private JButton[] answerButtons;
00041     private final List<JPanel> prizeLadderRows;
00042     private final LifelineView lifelineView;
00043     private JLabel devHintLabel;
00052     public GameView(GameController controller, LifelineController lifelineController) {
00053         this.controller = controller;
00054         this.timerController = new TimerController();
00055         this.prizeLadderRows = new ArrayList<>();
00056         this.lifelineView = new LifelineView(this, lifelineController);
00057
00058         setupWindow();
00059         setupComponents();
00060         timerController.startTimer();
00061     }
00062
00063
00075     private void setupWindow() {
00076         setTitle("Legyen Ön is Milliomos!");
00077         setSize(WINDOW_WIDTH, WINDOW_HEIGHT);
00078         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
00079         setResizable(false);
00080         setLocationRelativeTo(null);
00081
00082         setLayout(new BorderLayout());
00083         getContentPane().setBackground(BACKGROUND_COLOR);
00084     }
00085
00086
00096     private void setupComponents() {
00097         JPanel gamePanel = new JPanel(new BorderLayout(20, 20));
00098         gamePanel.setBackground(BACKGROUND_COLOR);
00099         gamePanel.setBorder(BorderFactory.createEmptyBorder(20, 20, 20, 20));
00100
00101         JPanel questionPanel = createQuestionPanel();
00102         gamePanel.add(questionPanel, BorderLayout.NORTH);
00103
00104         JPanel answersPanel = createAnswersPanel();
00105         gamePanel.add(answersPanel, BorderLayout.CENTER);
00106
00107         JPanel southPanel = new JPanel(new BorderLayout());
00108         southPanel.setBackground(BACKGROUND_COLOR);
00109
00110         southPanel.add(lifelineView.createLifelinePanel(), BorderLayout.CENTER);
00111
00112         if (Main.isDeveloperMode()) {
00113             devHintLabel = new JLabel();
00114             devHintLabel.setForeground(TEXT_COLOR);
00115             devHintLabel.setHorizontalAlignment(SwingConstants.CENTER);
00116             devHintLabel.setFont(new Font("Arial", Font.PLAIN, 12));
00117             southPanel.add(devHintLabel, BorderLayout.SOUTH);
00118         }
00119
00120         gamePanel.add(southPanel, BorderLayout.SOUTH);
00121
00122         JPanel rightPanel = new JPanel(new BorderLayout());
00123         rightPanel.setBackground(BACKGROUND_COLOR);
00124
00125         rightPanel.add(timerController.getView(), BorderLayout.NORTH);
00126
00127         rightPanel.add(createPrizeLadderPanel(), BorderLayout.CENTER);
00128
00129         add(gamePanel, BorderLayout.CENTER);
00130         add(rightPanel, BorderLayout.EAST);
00131     }
00132
00133
00142     private JPanel createQuestionPanel() {
00143         JPanel panel = new JPanel();
00144         panel.setBackground(BACKGROUND_COLOR);
00145         panel.setBorder(BorderFactory.createCompoundBorder(
00146             BorderFactory.createLineBorder(BUTTON_COLOR, 2),
00147             BorderFactory.createEmptyBorder(10, 10, 10, 10)
00148         ));
00149         panel.setPreferredSize(new Dimension(800, 100));
00150
00151         questionLabel = new JLabel();
00152         questionLabel.setForeground(TEXT_COLOR);
00153         questionLabel.setFont(new Font("Arial", Font.BOLD, 18));
00154         panel.add(questionLabel);
00155
00156         return panel;
00157     }
00158
00159

```

```

00170     private JPanel createAnswersPanel() {
00171         JPanel panel = new JPanel(new GridLayout(2, 2, 20, 20));
00172         panel.setBackground(BACKGROUND_COLOR);
00173
00174         answerButtons = new JButton[4];
00175         String[] letters = {"A", "B", "C", "D"};
00176
00177         for (int i = 0; i < 4; i++) {
00178             final String letter = letters[i];
00179             answerButtons[i] = createAnswerButton(letter);
00180             answerButtons[i].addActionListener(e -> controller.handleAnswer(letter));
00181             panel.add(answerButtons[i]);
00182         }
00183
00184         return panel;
00185     }
00186
00187     private JButton createAnswerButton(String letter) {
00188         JButton button = new JButton(letter);
00189         styleButton(button);
00190         return button;
00191     }
00192
00193     private JPanel createPrizeLadderPanel() {
00194         JPanel panel = new JPanel();
00195         panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));
00196         panel.setBackground(BACKGROUND_COLOR);
00197         panel.setBorder(BorderFactory.createEmptyBorder(20, 10, 20, 20));
00198
00199         List<PrizeLevel> prizeLevels = GameModel.getPrizeLevels();
00200
00201         for (PrizeLevel level : prizeLevels) {
00202             JPanel prizeRow = createPrizeRow(level);
00203             prizeLadderRows.add(prizeRow);
00204             panel.add(prizeRow);
00205             if (prizeLevels.indexOf(level) < prizeLevels.size() - 1) {
00206                 panel.add(Box.createRigidArea(new Dimension(0, 5)));
00207             }
00208         }
00209
00210         return panel;
00211     }
00212
00213     private JPanel createPrizeRow(PrizeLevel level) {
00214         JPanel panel = new JPanel();
00215         panel.setLayout(new BoxLayout(panel, BoxLayout.X_AXIS));
00216         panel.setBackground(BACKGROUND_COLOR);
00217
00218         JLabel levelLabel = new JLabel(String.valueOf(level.getLevel()));
00219         JLabel prizeLabel = new JLabel(formatPrize(level.getPrize()));
00220
00221         levelLabel.setForeground(TEXT_COLOR);
00222         prizeLabel.setForeground(TEXT_COLOR);
00223
00224         levelLabel.setFont(new Font("Arial", Font.BOLD, 14));
00225         prizeLabel.setFont(new Font("Arial", Font.BOLD, 14));
00226
00227         panel.add(levelLabel);
00228         panel.add(Box.createHorizontalStrut(10));
00229         panel.add(prizeLabel);
00230
00231         return panel;
00232     }
00233
00234     private String formatPrize(long prize) {
00235         return String.format("%,d Ft", prize).replace(",", ".");
00236     }
00237
00238     private void styleButton(JButton button) {
00239         button.setFont(new Font("Arial", Font.BOLD, 16));
00240         button.setForeground(TEXT_COLOR);
00241         button.setBackground(BUTTON_COLOR);
00242         button.setBorder(BorderFactory.createCompoundBorder(
00243             BorderFactory.createLineBorder(BUTTON_COLOR, 2),
00244             BorderFactory.createEmptyBorder(10, 10, 10, 10)
00245         ));
00246         button.setFocusPainted(false);
00247
00248         button.addMouseListener(new java.awt.event.MouseAdapter() {
00249             @Override
00250             public void mouseEntered(java.awt.event.MouseEvent evt) {
00251                 if (button.isEnabled()) {

```

```

00298         button.setBackground(BUTTON_HOVER_COLOR);
00299     }
00300 }
00301 @Override
00302 public void mouseExited(java.awt.event.MouseEvent evt) {
00303     if (button.isEnabled()) {
00304         button.setBackground(BUTTON_COLOR);
00305     }
00306 }
00307 });
00308 }
00309
00310
00321 public void updateQuestion(Question question) {
00322     questionLabel.setText(question.getQuestion());
00323
00324     answerButtons[0].setText("A: " + question.getOptionA());
00325     answerButtons[1].setText("B: " + question.getOptionB());
00326     answerButtons[2].setText("C: " + question.getOptionC());
00327     answerButtons[3].setText("D: " + question.getOptionD());
00328
00329     if (Main.isDeveloperMode()) {
00330         devHintLabel.setText("Válasz:" + question.getCorrectAnswer());
00331     }
00332
00333     resetAnswerButtons();
00334 }
00335
00336
00348 public void showAnswerResult(String selectedAnswer, String correctAnswer, boolean isCorrect) {
00349     setAnswerButtonsEnabled(false);
00350
00351     for (JButton button : answerButtons) {
00352         String buttonLetter = button.getText().substring(0, 1);
00353         if (buttonLetter.equals(selectedAnswer)) {
00354             button.setBackground(isCorrect ? CORRECT_ANSWER_COLOR : WRONG_ANSWER_COLOR);
00355         } else if (buttonLetter.equals(correctAnswer) && !isCorrect) {
00356             button.setBackground(CORRECT_ANSWER_COLOR);
00357         }
00358     }
00359 }
00360
00361
00369 public void updatePrizeLadder(int currentLevel) {
00370     for (int i = 0; i < prizeLadderRows.size(); i++) {
00371         JPanel row = prizeLadderRows.get(i);
00372         int rowLevel = 15 - i;
00373
00374         if (rowLevel == currentLevel) {
00375             row.setBackground(HIGHLIGHT_COLOR);
00376             for (Component comp : row.getComponents()) {
00377                 if (comp instanceof JLabel) {
00378                     ((JLabel) comp).setForeground(Color.BLACK);
00379                 }
00380             }
00381         } else {
00382             row.setBackground(BACKGROUND_COLOR);
00383             for (Component comp : row.getComponents()) {
00384                 if (comp instanceof JLabel) {
00385                     ((JLabel) comp).setForeground(TEXT_COLOR);
00386                 }
00387             }
00388         }
00389     }
00390 }
00391
00392
0400 private void resetAnswerButtons() {
0401     for (JButton button : answerButtons) {
0402         button.setBackground(BUTTON_COLOR);
0403         button.setEnabled(true);
0404     }
0405 }
0406
0407
0413 public void setAnswerButtonsEnabled(boolean enabled) {
0414     for (JButton button : answerButtons) {
0415         button.setEnabled(enabled);
0416     }
0417 }
0418
0419
0425 public JButton[] getAnswerButtons() {
0426     return answerButtons;
0427 }
0428
0429

```

```

00433     public void stopTimer() {
00434         timerController.stopTimer();
00435     }
00436
00437
00441     public void updateFinalPlaytime() {
00442         controller.getPlayer().setPlaytime(timerController.getElapsedTime());
00443     }
00444
00445
00456     public void showGameEndDialog(GameState state, String prize) {
00457         String message = state == GameState.WON ?
00458             "Gratulálok! Megnyerted a főnyereményt: " + prize :
00459             "Sajnos helytelen válasz! Nyereményed: " + prize;
00460
00461         String title = state == GameState.WON ? "Győzelem!" : "Játék vége";
00462
00463         JOptionPane.showMessageDialog(this, message, title, JOptionPane.INFORMATION_MESSAGE);
00464     }
00465
00466
00473     public void closeGame() {
00474         timerController.stopTimer();
00475         controller.getPlayer().updatePlaytime();
00476         dispose();
00477     }
00478 }

```

## 7.47. src/main/java/millionaire/view/LifelineView.java fájlreferencia

### Osztályok

- class [millionaire.view.LifelineView](#)  
A játék segítségével megjelenítő és kezelő felület.

### Csomagok

- package [millionaire.view](#)

## 7.48. LifelineView.java

[Ugrás a fájl dokumentációjához.](#)

```

00001
00002 package millionaire.view;
00003
00004 import millionaire.controller.LifelineController;
00005 import javax.swing.*;
00006 import java.awt.*;
00007 import java.util.Map;
00008
00018 public class LifelineView {
00019     private static final Color BACKGROUND_COLOR = new Color(90, 90, 90);
00020     private static final Color TEXT_COLOR = Color.WHITE;
00021     private static final Color BUTTON_COLOR = new Color(70, 130, 180);
00022     private static final Color BUTTON_HOVER_COLOR = new Color(100, 149, 237);
00023     private static final Color DISABLED_COLOR = new Color(128, 128, 128);
00025     private final JButton fiftyFiftyButton;
00026     private final JButton audienceButton;
00027     private final LifelineController controller;
00028     private final JFrame parentFrame;
00036     public LifelineView(JFrame parentFrame, LifelineController controller) {
00037         this.parentFrame = parentFrame;
00038         this.controller = controller;
00039
00040         this.fiftyFiftyButton = createLifelineButton("Felezés", e -> controller.useFiftyFifty());
00041         this.audienceButton = createLifelineButton("Közönség szavazás", e ->
00042             controller.useAudienceHelp());
00043     }
00044

```

```

00054     private JButton createLifelineButton(String text, java.awt.event.ActionListener listener) {
00055         JButton button = new JButton(text);
00056         styleButton(button);
00057         button.addActionListener(listener);
00058         return button;
00059     }
00060
00061     private void styleButton(JButton button) {
00074         button.setFont(new Font("Arial", Font.BOLD, 14));
00075         button.setForeground(Color.WHITE);
00076         button.setBackground(BUTTON_COLOR);
00077         button.setBorder(BorderFactory.createRaisedBevelBorder());
00078         button.setFocusPainted(false);
00079
00080         button.addMouseListener(new java.awt.event.MouseAdapter() {
00081             @Override
00082             public void mouseEntered(java.awt.event.MouseEvent evt) {
00083                 if (button.isEnabled()) {
00084                     button.setBackground(BUTTON_HOVER_COLOR);
00085                 }
00086             }
00087
00088             @Override
00089             public void mouseExited(java.awt.event.MouseEvent evt) {
00090                 if (button.isEnabled()) {
00091                     button.setBackground(BUTTON_COLOR);
00092                 }
00093             }
00094         });
00095     }
00096
00097     public JPanel createLifelinePanel() {
00107         JPanel panel = new JPanel(new FlowLayout(FlowLayout.LEFT, 20, 10));
00108         panel.setBackground(new Color(90, 90, 90));
00109         panel.add(fiftyFiftyButton);
00110         panel.add(audienceButton);
00111         return panel;
00112     }
00113
00114     public void updateButtonStates(boolean fiftyFiftyAvailable, boolean audienceHelpAvailable) {
00127         fiftyFiftyButton.setEnabled(fiftyFiftyAvailable);
00128         audienceButton.setEnabled(audienceHelpAvailable);
00129
00130         fiftyFiftyButton.setBackground(fiftyFiftyAvailable ? BUTTON_COLOR : DISABLED_COLOR);
00131         audienceButton.setBackground(audienceHelpAvailable ? BUTTON_COLOR : DISABLED_COLOR);
00132     }
00133
00134     public void updateAnswerButtons(JButton[] answerButtons, Map<String, Boolean> visibilityMap) {
00146         for (JButton button : answerButtons) {
00147             String option = button.getText().substring(0, 1);
00148             if (!visibilityMap.get(option)) {
00149                 button.setEnabled(false);
00150                 button.setText(option + ": ");
00151             }
00152         }
00153     }
00154
00155     public void showAudienceResults(Map<String, Integer> percentages) {
00164         JDialog dialog = createAudienceDialog(percentages);
00165         dialog.setVisible(true);
00166     }
00167
00168     private JDialog createAudienceDialog(Map<String, Integer> percentages) {
00182         JDialog dialog = new JDialog(parentFrame, "Közönség szavazás", true);
00183         dialog.setLayout(new BorderLayout());
00184
00185         JPanel barsPanel = new JPanel(new GridLayout(4, 1, 5, 5));
00186         barsPanel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
00187
00188         for (Map.Entry<String, Integer> entry : percentages.entrySet()) {
00189             barsPanel.add(createPercentageBar(entry.getKey(), entry.getValue()));
00190         }
00191
00192         JButton closeButton = new JButton("Bezár");
00193         closeButton.addActionListener(e -> dialog.dispose());
00194
00195         dialog.add(barsPanel, BorderLayout.CENTER);
00196         dialog.add(closeButton, BorderLayout.SOUTH);
00197
00198         dialog.setSize(300, 200);

```

```

00199         dialog.setLocationRelativeTo(parentFrame);
00200
00201         return dialog;
00202     }
00203
00204
00217     private JPanel createPercentageBar(String letter, int percentage) {
00218         JPanel panel = new JPanel(new BorderLayout(10, 0));
00219
00220         JLabel label = new JLabel(String.format("%s: %d%", letter, percentage));
00221         JProgressBar progressBar = new JProgressBar(0, 100);
00222         progressBar.setValue(percentage);
00223         progressBar.setStringPainted(false);
00224
00225         panel.add(label, BorderLayout.WEST);
00226         panel.add(progressBar, BorderLayout.CENTER);
00227
00228         return panel;
00229     }
00230 }

```

## 7.49. src/main/java/millionaire/view/MainMenuView.java fájlreferencia

### Osztályok

- class [millionaire.view.MainMenuView](#)  
*A játék főmenüjének grafikus felülete.*

### Csomagok

- package [millionaire.view](#)

## 7.50. MainMenuView.java

[Ugrás a fájl dokumentációjához.](#)

```

00001
00002 package millionaire.view;
00003
00004 import millionaire.controller.MainMenuController;
00005 import javax.swing.*;
00006 import java.awt.*;
00007
00017 public class MainMenuView extends JFrame {
00018     private static final int WINDOW_WIDTH = 1280;
00019     private static final int WINDOW_HEIGHT = 720;
00020     private static final String GAME_TITLE = "Legyen Ön is Milliomos!";
00021     private static final Color BACKGROUND_COLOR = new Color(90, 90, 90);
00022     private static final Color BUTTON_COLOR = new Color(70, 130, 180);
00023     private static final Color BUTTON_HOVER_COLOR = new Color(100, 149, 237);
00024     private static final Color TEXT_COLOR = Color.WHITE;
00026     private final MainMenuController controller;
00036     public MainMenuView(MainMenuController controller) {
00037         this.controller = controller;
00038         setupWindow();
00039         setupComponents();
00040     }
00041
00042
00053     private void setupWindow() {
00054         setTitle(GAME_TITLE);
00055         setSize(WINDOW_WIDTH, WINDOW_HEIGHT);
00056         setDefaultCloseOperation(WindowConstants.EXIT_ON_CLOSE);
00057         setResizable(false);
00058         setLocationRelativeTo(null);
00059     }
00060
00061
00069     private void setupComponents() {
00070         JPanel mainPanel = createMainPanel();
00071         JLabel titleLabel = createTitleLabel();

```

```

00072         JPanel buttonPanel = createButtonPanel();
00073
00074         mainPanel.add(titleLabel);
00075         mainPanel.add(buttonPanel);
00076
00077         add(mainPanel);
00078     }
00079
00080
00081     private JPanel createMainPanel() {
00082         JPanel panel = new JPanel();
00083         panel.setLayout(new BorderLayout(panel, BorderLayout.Y_AXIS));
00084         panel.setBorder(BorderFactory.createEmptyBorder(50, 25, 50, 25));
00085         panel.setBackground(BACKGROUND_COLOR);
00086         return panel;
00087     }
00088
00089     private JLabel createTitleLabel() {
00090         JLabel label = new JLabel(GAME_TITLE);
00091         label.setFont(new Font("Arial", Font.BOLD, 36));
00092         label.setForeground(TEXT_COLOR);
00093         label.setAlignmentX(Component.CENTER_ALIGNMENT);
00094         return label;
00095     }
00096
00097     private JPanel createButtonPanel() {
00098         JPanel panel = new JPanel();
00099         panel.setLayout(new BorderLayout(panel, BorderLayout.Y_AXIS));
00100         panel.setBorder(BorderFactory.createEmptyBorder(50, 0, 0, 0));
00101         panel.setBackground(BACKGROUND_COLOR);
00102
00103         String[] buttonLabels = {"Új Játék", "Kérdés hozzáadása", "Dicsőséglista", "Kézikönyv",
00104 "Kilépés"};
00105
00106         for (String label : buttonLabels) {
00107             JButton button = createMenuButton(label);
00108             panel.add(button);
00109             panel.add(Box.createRigidArea(new Dimension(0, 20)));
00110         }
00111
00112         return panel;
00113     }
00114
00115     private JButton createMenuButton(String text) {
00116         JButton button = new JButton(text);
00117         styleButton(button);
00118         button.addActionListener(e -> handleButtonClick(text));
00119         return button;
00120     }
00121
00122     private void styleButton(JButton button) {
00123         button.setPreferredSize(new Dimension(200, 50));
00124         button.setMaximumSize(new Dimension(200, 50));
00125         button.setFont(new Font("Arial", Font.PLAIN, 18));
00126         button.setFocusPainted(false);
00127         button.setBackground(BUTTON_COLOR);
00128         button.setForeground(TEXT_COLOR);
00129         button.setBorder(BorderFactory.createRaisedBevelBorder());
00130         button.setAlignmentX(Component.CENTER_ALIGNMENT);
00131
00132         button.addMouseListener(new java.awt.event.MouseAdapter() {
00133             @Override
00134             public void mouseEntered(java.awt.event.MouseEvent evt) {
00135                 button.setBackground(BUTTON_HOVER_COLOR);
00136             }
00137
00138             @Override
00139             public void mouseExited(java.awt.event.MouseEvent evt) {
00140                 button.setBackground(BUTTON_COLOR);
00141             }
00142         });
00143     }
00144
00145     private void handleButtonClick(String buttonText) {
00146         switch (buttonText) {
00147             case "Új Játék":
00148                 controller.startNewGame();
00149                 break;
00150             case "Kérdés hozzáadása":
00151                 controller.showAddQuestionDialog();
00152                 break;
00153             case "Dicsőséglista":

```



```

00221         controller.showScoreboard();
00222         break;
00223     case "Kézikönyv":
00224         controller.showRules();
00225         break;
00226     case "Kilépés":
00227         controller.exitGame();
00228         break;
00229     }
00230 }
00231
00232
00233 public void hideWindow() {
00234     setVisible(false);
00235 }
00236
00237 public void showWindow() {
00238     setVisible(true);
00239 }
00240 }

```

## 7.51. src/main/java/millionaire/view/PlayerView.java fájlreferencia

### Osztályok

- class [millionaire.view.PlayerView](#)

*Játékos létrehozására szolgáló dialógusablak.*

### Csomagok

- package [millionaire.view](#)

## 7.52. PlayerView.java

[Ugrás a fájl dokumentációjához.](#)

```

00001
00002 package millionaire.view;
00003
00004 import millionaire.controller.PlayerController;
00005 import javax.swing.*;
00006 import java.awt.*;
00007
00016 public class PlayerView extends JDialog {
00017     private static final int DIALOG_WIDTH = 300;
00018     private static final int DIALOG_HEIGHT = 150;
00019     private final PlayerController controller;
00020     private JTextField nameField;
00021     private JButton okButton;
00022     private JButton cancelButton;
00023     public PlayerView(JFrame parent, PlayerController controller) {
00024         super(parent, "Új Játék", true);
00025         this.controller = controller;
00026         setupUI();
00027     }
00028
00029     private void setupUI() {
00030         setSize(DIALOG_WIDTH, DIALOG_HEIGHT);
00031         setLocationRelativeTo(getParent());
00032
00033         JPanel panel = new JPanel();
00034         panel.setLayout(new BorderLayout(panel, BorderLayout.Y_AXIS));
00035         panel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
00036
00037         JLabel label = new JLabel("Kérem adja meg a nevét:");
00038         nameField = new JTextField(20);
00039         okButton = new JButton("OK");
00040         cancelButton = new JButton("Mégse");
00041     }

```

```

00066         label.setAlignmentX(Component.CENTER_ALIGNMENT);
00067         nameField.setMaximumSize(new Dimension(200, 25));
00068         nameField.setAlignmentX(Component.CENTER_ALIGNMENT);
00069
00070         JPanel buttonPanel = new JPanel();
00071         buttonPanel.add(okButton);
00072         buttonPanel.add(cancelButton);
00073
00074         okButton.addActionListener(e -> handleOkButton());
00075         cancelButton.addActionListener(e -> handleCancelButton());
00076
00077         panel.add(label);
00078         panel.add(Box.createRigidArea(new Dimension(0, 10)));
00079         panel.add(nameField);
00080         panel.add(Box.createRigidArea(new Dimension(0, 10)));
00081         panel.add(buttonPanel);
00082
00083         add(panel);
00084     }
00085
00086
00092     private void handleOkButton() {
00093         String name = nameField.getText().trim();
00094         controller.createPlayer(name);
00095     }
00096
00097
00103     private void handleCancelButton() {
00104         controller.cancelPlayerCreation();
00105     }
00106
00107
00115     public void showError(String message) {
00116         JOptionPane.showMessageDialog(this, message, "Hiba", JOptionPane.ERROR_MESSAGE);
00117     }
00118
00119
00127     public String getPlayerName() {
00128         return nameField.getText().trim();
00129     }
00130
00131
00137     public void closeDialog() {
00138         dispose();
00139     }
00140 }

```

## 7.53. src/main/java/millionaire/view/RulesView.java fájlreferencia

### Osztályok

- class [millionaire.view.RulesView](#)  
*A játékszabályok megjelenítésére szolgáló ablak.*

### Csomagok

- package [millionaire.view](#)

## 7.54. RulesView.java

[Ugrás a fájl dokumentációjához.](#)

```

00001
00002 package millionaire.view;
00003
00004 import millionaire.controller.RulesController;
00005 import javax.swing.*;
00006 import java.awt.*;
00007
00017 public class RulesView extends JFrame {
00018     private static final int WINDOW_WIDTH = 800;

```

```

00019     private static final int WINDOW_HEIGHT = 600;
00020     private static final Color BACKGROUND_COLOR = new Color(90, 90, 90);
00021     private static final Color TEXT_COLOR = Color.WHITE;
00022     private static final Color BUTTON_COLOR = new Color(70, 130, 180);
00023     private static final Color BUTTON_HOVER_COLOR = new Color(100, 149, 237);
00025     private final RulesController controller;
00026     private JTextArea rulesArea;
00033     public RulesView(RulesController controller) {
00034         this.controller = controller;
00035         setupWindow();
00036         setupComponents();
00037     }
00038
00039
00050     private void setupWindow() {
00051         setTitle("Játékszabályok");
00052         setSize(WINDOW_WIDTH, WINDOW_HEIGHT);
00053         setDefaultCloseOperation(DISPOSE_ON_CLOSE);
00054         setLocationRelativeTo(null);
00055         setResizable(true);
00056     }
00057
00067     private void setupComponents() {
00068         JPanel mainPanel = createMainPanel();
00069         JScrollPane scrollPane = createRulesTextArea();
00070         JButton closeButton = createCloseButton();
00071
00072         mainPanel.add(scrollPane, BorderLayout.CENTER);
00073         mainPanel.add(closeButton, BorderLayout.SOUTH);
00074
00075         add(mainPanel);
00076     }
00077
00084     private JPanel createMainPanel() {
00085         JPanel panel = new JPanel(new BorderLayout(10, 10));
00086         panel.setBackground(BACKGROUND_COLOR);
00087         panel.setBorder(BorderFactory.createEmptyBorder(20, 20, 20, 20));
00088         return panel;
00089     }
00090
00097     private JScrollPane createRulesTextArea() {
00098         rulesArea = new JTextArea();
00099         rulesArea.setEditable(false);
00100         rulesArea.setBackground(BACKGROUND_COLOR);
00101         rulesArea.setForeground(TEXT_COLOR);
00102         rulesArea.setFont(new Font("Arial", Font.PLAIN, 14));
00103         rulesArea.setLineWrap(true);
00104         rulesArea.setWrapStyleWord(true);
00105
00106         JScrollPane scrollPane = new JScrollPane(rulesArea);
00107         scrollPane.setBorder(BorderFactory.createEmptyBorder());
00108         return scrollPane;
00109     }
00110
00117     private JButton createCloseButton() {
00118         JButton button = new JButton("Bezárás");
00119         styleButton(button);
00120         button.addActionListener(e -> controller.handleCloseRequest());
00121         return button;
00122     }
00123
00130     private void styleButton(JButton button) {
00131         button.setFont(new Font("Arial", Font.PLAIN, 14));
00132         button.setFocusPainted(false);
00133         button.setBackground(BUTTON_COLOR);
00134         button.setForeground(TEXT_COLOR);
00135         button.setBorder(BorderFactory.createRaisedBevelBorder());
00136
00137         button.addMouseListener(new java.awt.event.MouseAdapter() {
00138             @Override
00139             public void mouseEntered(java.awt.event.MouseEvent evt) {
00140                 button.setBackground(BUTTON_HOVER_COLOR);
00141             }
00142
00143             @Override
00144             public void mouseExited(java.awt.event.MouseEvent evt) {
00145                 button.setBackground(BUTTON_COLOR);
00146             }
00147         });
00148     }
00149
00150

```

```

00156     public void displayRules(String content) {
00157         rulesArea.setText(content);
00158         rulesArea.setCaretPosition(0);
00159     }
00160
00161
00167     public void showError(String message) {
00168         JOptionPane.showMessageDialog(this, message, "Hiba", JOptionPane.ERROR_MESSAGE);
00169     }
00170
00171
00175     public void closeWindow() {
00176         dispose();
00177     }
00178 }

```

## 7.55. src/main/java/millionaire/view/ScoreboardView.java fájlreferencia

### Osztályok

- class [millionaire.view.ScoreboardView](#)  
A dicsőséglista megjelenítésére szolgáló ablak.

### Csomagok

- package [millionaire.view](#)

## 7.56. ScoreboardView.java

[Ugrás a fájl dokumentációjához.](#)

```

00001
00002 package millionaire.view;
00003
00004 import millionaire.controller.ScoreboardController;
00005 import millionaire.model.PlayerModel;
00006 import javax.swing.*;
00007 import javax.swing.table.DefaultTableModel;
00008 import java.awt.*;
00009 import java.util.List;
00010
00020 public class ScoreboardView extends JFrame {
00021     private static final int WINDOW_WIDTH = 800;
00022     private static final int WINDOW_HEIGHT = 600;
00023     private static final Color BACKGROUND_COLOR = new Color(90, 90, 90);
00024     private static final Color TEXT_COLOR = Color.WHITE;
00025     private static final Color BUTTON_COLOR = new Color(70, 130, 180);
00026     private static final Color BUTTON_HOVER_COLOR = new Color(100, 149, 237);
00028     private final transient ScoreboardController controller;
00029     private final JTable scoreTable;
00030     private final DefaultTableModel tableModel;
00040     public ScoreboardView(ScoreboardController controller) {
00041         this.controller = controller;
00042         this.tableModel = createTableModel();
00043         this.scoreTable = createTable();
00044
00045         setupWindow();
00046         setupComponents();
00047     }
00048
00049
00053     private void setupWindow() {
00054         setTitle("Dicsőséglista");
00055         setSize(WINDOW_WIDTH, WINDOW_HEIGHT);
00056         setDefaultCloseOperation(DISPOSE_ON_CLOSE);
00057         setLocationRelativeTo(null);
00058         setResizable(true);
00059     }
00060
00061
00074     private DefaultTableModel createTableModel() {

```

```

00075         return new DefaultTableModel(
00076             new String[]{"Helyezés", "Név", "Megválaszolt kérdések", "Nyeremény", "Játékidő"},
00077             0
00078         ) {
00079             @Override
00080             public boolean isCellEditable(int row, int column) {
00081                 return false;
00082             }
00083         };
00084     }
00085
00086
00091     private JTable createTable() {
00092         JTable table = new JTable(tableModel);
00093         styleTable(table);
00094         return table;
00095     }
00096
00097
00110     private void styleTable(JTable table) {
00111         table.setBackground(BACKGROUND_COLOR);
00112         table.setForeground(TEXT_COLOR);
00113         table.setFont(new Font("Arial", Font.PLAIN, 14));
00114         table.getTableHeader().setFont(new Font("Arial", Font.BOLD, 14));
00115         table.setRowHeight(25);
00116         table.setGridColor(new Color(70, 130, 180));
00117
00118         // Nincs átrendezés és átméretezés..
00119         table.getTableHeader().setReorderingAllowed(false);
00120         table.getTableHeader().setResizingAllowed(false);
00121     }
00122
00123
00127     private void setupComponents() {
00128         JPanel mainPanel = new JPanel(new BorderLayout(10, 10));
00129         mainPanel.setBackground(BACKGROUND_COLOR);
00130         mainPanel.setBorder(BorderFactory.createEmptyBorder(20, 20, 20, 20));
00131
00132         JScrollPane scrollPane = new JScrollPane(scoreTable);
00133         scrollPane.getViewport().setBackground(BACKGROUND_COLOR);
00134
00135         JButton closeButton = createCloseButton();
00136
00137         mainPanel.add(scrollPane, BorderLayout.CENTER);
00138         mainPanel.add(closeButton, BorderLayout.SOUTH);
00139
00140         add(mainPanel);
00141     }
00142
00143
00149     private JButton createCloseButton() {
00150         JButton button = new JButton("Bezárás");
00151         styleButton(button);
00152         button.addActionListener(e -> controller.handleClick());
00153         return button;
00154     }
00155
00156
00162     private void styleButton(JButton button) {
00163         button.setFont(new Font("Arial", Font.PLAIN, 14));
00164         button.setFocusPainted(false);
00165         button.setBackground(BUTTON_COLOR);
00166         button.setForeground(TEXT_COLOR);
00167         button.setBorder(BorderFactory.createRaisedBevelBorder());
00168
00169         button.addMouseListener(new java.awt.event.MouseAdapter() {
00170             @Override
00171             public void mouseEntered(java.awt.event.MouseEvent evt) {
00172                 button.setBackground(BUTTON_HOVER_COLOR);
00173             }
00174
00175             @Override
00176             public void mouseExited(java.awt.event.MouseEvent evt) {
00177                 button.setBackground(BUTTON_COLOR);
00178             }
00179         });
00180     }
00181
00182
00190     public void updateScores(List<PlayerModel> players) {
00191         tableModel.setRowCount(0);
00192         int rank = 1;
00193         for (PlayerModel player : players) {
00194             tableModel.addRow(new Object[]{
00195                 rank++,
00196                 player.getName(),
00197                 player.getCurrentLevel(),

```

```
00198         player.getFormattedPrize(),
00199         player.getFormattedPlaytime()
00200     });
00201 }
00202 }
00203
00204
00208     public void closeWindow() {
00209         dispose();
00210     }
00211 }
```

## 7.57. src/main/java/millionaire/view/TimerView.java fájlreferencia

### Osztályok

- class [millionaire.view.TimerView](#)  
*Az időmérő megjelenítése.*

### Csomagok

- package [millionaire.view](#)

## 7.58. TimerView.java

[Ugrás a fájl dokumentációjához.](#)

```
00001
00002 package millionaire.view;
00003
00004 import javax.swing.*;
00005 import java.awt.*;
00006 import java.time.Duration;
00007
00015 public class TimerView extends JPanel {
00016     private static final Color BACKGROUND_COLOR = new Color(90, 90, 90);
00017     private static final Color TEXT_COLOR = Color.WHITE;
00018
00019     private final JLabel timeLabel;
00024     public TimerView() {
00025         setBackground(BACKGROUND_COLOR);
00026         setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
00027         setPreferredSize(new Dimension(200, 50));
00028
00029         timeLabel = new JLabel("Játékidő: 00:00");
00030         timeLabel.setForeground(TEXT_COLOR);
00031         timeLabel.setFont(new Font("Arial", Font.BOLD, 16));
00032         timeLabel.setHorizontalAlignment(SwingConstants.CENTER);
00033
00034         add(timeLabel);
00035     }
00036
00041     public void updateDisplay(Duration duration) {
00042         long seconds = duration.getSeconds();
00043         long minutes = seconds / 60;
00044         seconds = seconds % 60;
00045
00046         timeLabel.setText(String.format("Játékidő: %02d:%02d", minutes, seconds));
00047     }
00048 }
```

# Tárgymutató

- addQuestion
  - millionaire.model.AddQuestionModel, [16](#)
  - millionaire.model.QuestionRepository, [95](#)
- AddQuestionController
  - millionaire.controller.AddQuestionController, [12](#)
- AddQuestionModel
  - millionaire.model.AddQuestionModel, [16](#)
- AddQuestionView
  - millionaire.view.AddQuestionView, [22](#)
- addScore
  - millionaire.controller.ScoreboardController, [106](#)
  - millionaire.model.ScoreboardModel, [109](#)
- cancelPlayerCreation
  - millionaire.controller.PlayerController, [71](#)
- CATEGORIES
  - millionaire.model.QuestionRepository, [97](#)
- clearFields
  - millionaire.view.AddQuestionView, [22](#)
- closeDialog
  - millionaire.view.PlayerView, [84](#)
- closeGame
  - millionaire.view.GameView, [42](#)
- closeWindow
  - millionaire.view.AddQuestionView, [22](#)
  - millionaire.view.RulesView, [103](#)
  - millionaire.view.ScoreboardView, [113](#)
- compareTo
  - millionaire.model.PlayerModel, [76](#)
- CORRECT
  - millionaire.model.GameState, [38](#)
- createLifelinePanel
  - millionaire.view.LifelineView, [56](#)
- createPlayer
  - millionaire.controller.PlayerController, [71](#)
- DIFFICULTIES
  - millionaire.model.QuestionRepository, [97](#)
- displayRules
  - millionaire.view.RulesView, [104](#)
- equals
  - millionaire.model.PlayerModel, [77](#)
- exitGame
  - millionaire.controller.MainMenuController, [62](#)
- GameController
  - millionaire.controller.GameController, [25](#)
- GameModel
  - millionaire.model.GameModel, [31](#)
- GameView
  - millionaire.view.GameView, [41](#)
- getAllQuestions
  - millionaire.model.QuestionRepository, [96](#)
- getAnswerButtons
  - millionaire.view.GameView, [42](#)
- getAudienceResult
  - millionaire.model.LifelineModel, [52](#)
- getCategories
  - millionaire.controller.AddQuestionController, [12](#)
  - millionaire.model.AddQuestionModel, [17](#)
- getCategory
  - millionaire.model.Question, [90](#)
- getCorrectAnswer
  - millionaire.model.Question, [90](#)
- getCurrentLevel
  - millionaire.controller.GameController, [26](#)
  - millionaire.model.PlayerModel, [77](#)
- getCurrentPrize
  - millionaire.model.GameModel, [32](#)
- getCurrentQuestion
  - millionaire.controller.GameController, [26](#)
  - millionaire.model.GameModel, [32](#)
- getDifficulties
  - millionaire.controller.AddQuestionController, [13](#)
  - millionaire.model.AddQuestionModel, [18](#)
- getDifficulty
  - millionaire.model.Question, [90](#)
- getElapsedTime
  - millionaire.controller.TimerController, [115](#)
  - millionaire.model.TimerModel, [119](#)
- getFiftyFiftyResult
  - millionaire.model.LifelineModel, [52](#)
- getFormattedPlaytime
  - millionaire.model.PlayerModel, [78](#)
- getFormattedPrize
  - millionaire.model.PlayerModel, [78](#)
- getGameView
  - millionaire.controller.GameController, [26](#)
- getLastSafePrize
  - millionaire.model.GameModel, [32](#)
- getLevel
  - millionaire.model.PrizeLevel, [87](#)
- getLifelinePanel
  - millionaire.controller.LifelineController, [48](#)
- getName
  - millionaire.model.PlayerModel, [78](#)
- getOptionA
  - millionaire.model.Question, [91](#)

- getOptionB
  - millionaire.model.Question, 91
- getOptionC
  - millionaire.model.Question, 91
- getOptionD
  - millionaire.model.Question, 92
- getPlayer
  - millionaire.controller.GameController, 27
  - millionaire.model.GameModel, 33
- getPlayerName
  - millionaire.view.PlayerView, 84
- getPlayers
  - millionaire.model.ScoreboardModel, 110
- getPlaytime
  - millionaire.model.PlayerModel, 79
- getPrize
  - millionaire.model.PlayerModel, 79
  - millionaire.model.PrizeLevel, 87
- getPrizeLevels
  - millionaire.model.GameModel, 33
- getQuestion
  - millionaire.model.Question, 92
- getQuestionRepository
  - millionaire.model.MainMenuModel, 66
- getQuestionsByCategory
  - millionaire.model.QuestionRepository, 96
- getQuestionsByDifficulty
  - millionaire.model.QuestionRepository, 96
- getRulesContent
  - millionaire.model.RulesModel, 101
- getStartTime
  - millionaire.model.PlayerModel, 79
  - millionaire.model.TimerModel, 119
- getView
  - millionaire.controller.TimerController, 115
- handleAnswer
  - millionaire.controller.GameController, 28
- handleCancel
  - millionaire.controller.AddQuestionController, 13
- handleClose
  - millionaire.controller.ScoreboardController, 106
- handleCloseRequest
  - millionaire.controller.RulesController, 99
- handleSubmit
  - millionaire.controller.AddQuestionController, 13
- hideWindow
  - millionaire.view.MainMenuView, 68
- incrementLevel
  - millionaire.model.PlayerModel, 80
- isAudienceHelpAvailable
  - millionaire.model.LifelineModel, 53
- isContentLoaded
  - millionaire.model.RulesModel, 101
- isDeveloperMode
  - millionaire.Main, 59
- isFiftyFiftyAvailable
  - millionaire.model.LifelineModel, 54
- isGameEnding
  - millionaire.model.GameState, 38
- isGameOver
  - millionaire.model.GameModel, 33
- isSafeSpot
  - millionaire.model.PrizeLevel, 87
- isTimeUp
  - millionaire.model.GameModel, 34
- LifelineController
  - millionaire.controller.LifelineController, 48
- LifelineView
  - millionaire.view.LifelineView, 56
- LOST
  - millionaire.model.GameState, 38
- main
  - millionaire.Main, 60
- MainMenuController
  - millionaire.controller.MainMenuController, 62
- MainMenuModel
  - millionaire.model.MainMenuModel, 65
- MainMenuView
  - millionaire.view.MainMenuView, 68
- millionaire, 9
- millionaire.controller, 9
- millionaire.controller.AddQuestionController, 11
  - AddQuestionController, 12
  - getCategories, 12
  - getDifficulties, 13
  - handleCancel, 13
  - handleSubmit, 13
  - showDialog, 14
- millionaire.controller.GameController, 24
  - GameController, 25
  - getCurrentLevel, 26
  - getCurrentQuestion, 26
  - getGameView, 26
  - getPlayer, 27
  - handleAnswer, 28
  - setAnswerButtonsEnabled, 28
  - updateGameTime, 29
- millionaire.controller.LifelineController, 46
  - getLifelinePanel, 48
  - LifelineController, 48
  - setGameContext, 48
  - useAudienceHelp, 49
  - useFiftyFifty, 50
- millionaire.controller.MainMenuController, 60
  - exitGame, 62
  - MainMenuController, 62
  - showAddQuestionDialog, 62
  - showMenu, 62
  - showRules, 63
  - showScoreboard, 63
  - startNewGame, 64
- millionaire.controller.PlayerController, 69
  - cancelPlayerCreation, 71
  - createPlayer, 71



- PlayerController, 70
- showPlayerCreationDialog, 72
- millionaire.controller.RulesController, 98
  - handleCloseRequest, 99
  - RulesController, 98
  - showRules, 99
- millionaire.controller.ScoreboardController, 104
  - addScore, 106
  - handleClose, 106
  - ScoreboardController, 106
  - showScoreboard, 107
- millionaire.controller.TimerController, 114
  - getElapsedTime, 115
  - getView, 115
  - startTimer, 116
  - stopTimer, 116
  - TimerController, 115
- millionaire.Main, 58
  - isDeveloperMode, 59
  - main, 60
- millionaire.model, 10
- millionaire.model.AddQuestionModel, 15
  - addQuestion, 16
  - AddQuestionModel, 16
  - getCategories, 17
  - getDifficulties, 18
  - validateInputs, 18
- millionaire.model.GameModel, 29
  - GameModel, 31
  - getCurrentPrize, 32
  - getCurrentQuestion, 32
  - getLastSafePrize, 32
  - getPlayer, 33
  - getPrizeLevels, 33
  - isGameOver, 33
  - isTimeUp, 34
  - processAnswer, 34
  - selectNextQuestion, 35
  - updatePlayTime, 36
- millionaire.model.GameState, 37
  - CORRECT, 38
  - isGameEnding, 38
  - LOST, 38
  - WON, 38
- millionaire.model.LifelineModel, 51
  - getAudienceResult, 52
  - getFiftyFiftyResult, 52
  - isAudienceHelpAvailable, 53
  - isFiftyFiftyAvailable, 54
- millionaire.model.MainMenuModel, 65
  - getQuestionRepository, 66
  - MainMenuModel, 65
- millionaire.model.PlayerModel, 73
  - compareTo, 76
  - equals, 77
  - getCurrentLevel, 77
  - getFormattedPlaytime, 78
  - getFormattedPrize, 78
  - getName, 78
  - getPlaytime, 79
  - getPrize, 79
  - getStartTime, 79
  - incrementLevel, 80
  - PlayerModel, 75, 76
  - setCurrentLevel, 80
  - setName, 80
  - setPlaytime, 81
  - setPrize, 81
  - updatePlaytime, 81
- millionaire.model.PrizeLevel, 85
  - getLevel, 87
  - getPrize, 87
  - isSafeSpot, 87
  - PrizeLevel, 86
- millionaire.model.Question, 88
  - getCategory, 90
  - getCorrectAnswer, 90
  - getDifficulty, 90
  - getOptionA, 91
  - getOptionB, 91
  - getOptionC, 91
  - getOptionD, 92
  - getQuestion, 92
- millionaire.model.QuestionRepository, 93
  - addQuestion, 95
  - CATEGORIES, 97
  - DIFFICULTIES, 97
  - getAllQuestions, 96
  - getQuestionsByCategory, 96
  - getQuestionsByDifficulty, 96
  - QuestionRepository, 95
- millionaire.model.RulesModel, 100
  - getRulesContent, 101
  - isContentLoaded, 101
  - RulesModel, 100
- millionaire.model.ScoreboardModel, 108
  - addScore, 109
  - getPlayers, 110
  - ScoreboardModel, 109
- millionaire.model.TimerModel, 117
  - getElapsedTime, 119
  - getStartTime, 119
  - start, 119
  - stop, 120
  - TimerModel, 119
  - updateElapsedTime, 120
- millionaire.view, 10
- millionaire.view.AddQuestionView, 19
  - AddQuestionView, 22
  - clearFields, 22
  - closeWindow, 22
  - showError, 23
  - showSuccess, 23
- millionaire.view.GameView, 39
  - closeGame, 42
  - GameView, 41

- getAnswerButtons, 42
  - setAnswerButtonsEnabled, 42
  - showAnswerResult, 43
  - showGameEndDialog, 43
  - stopTimer, 44
  - updateFinalPlaytime, 44
  - updatePrizeLadder, 45
  - updateQuestion, 45
- millionaire.view.LifelineView, 55
  - createLifelinePanel, 56
  - LifelineView, 56
  - showAudienceResults, 57
  - updateAnswerButtons, 57
  - updateButtonStates, 58
- millionaire.view.MainMenuView, 66
  - hideWindow, 68
  - MainMenuView, 68
  - showWindow, 69
- millionaire.view.PlayerView, 82
  - closeDialog, 84
  - getPlayerName, 84
  - PlayerView, 84
  - showError, 85
- millionaire.view.RulesView, 101
  - closeWindow, 103
  - displayRules, 104
  - RulesView, 103
  - showError, 104
- millionaire.view.ScoreboardView, 110
  - closeWindow, 113
  - ScoreboardView, 112
  - updateScores, 113
- millionaire.view.TimerView, 121
  - TimerView, 122
  - updateDisplay, 122
- PlayerController
  - millionaire.controller.PlayerController, 70
- PlayerModel
  - millionaire.model.PlayerModel, 75, 76
- PlayerView
  - millionaire.view.PlayerView, 84
- PrizeLevel
  - millionaire.model.PrizeLevel, 86
- processAnswer
  - millionaire.model.GameModel, 34
- QuestionRepository
  - millionaire.model.QuestionRepository, 95
- RulesController
  - millionaire.controller.RulesController, 98
- RulesModel
  - millionaire.model.RulesModel, 100
- RulesView
  - millionaire.view.RulesView, 103
- ScoreboardController
  - millionaire.controller.ScoreboardController, 106
- ScoreboardModel
  - millionaire.model.ScoreboardModel, 109
- ScoreboardView
  - millionaire.view.ScoreboardView, 112
- selectNextQuestion
  - millionaire.model.GameModel, 35
- setAnswerButtonsEnabled
  - millionaire.controller.GameController, 28
  - millionaire.view.GameView, 42
- setCurrentLevel
  - millionaire.model.PlayerModel, 80
- setGameContext
  - millionaire.controller.LifelineController, 48
- setName
  - millionaire.model.PlayerModel, 80
- setPlaytime
  - millionaire.model.PlayerModel, 81
- setPrize
  - millionaire.model.PlayerModel, 81
- showAddQuestionDialog
  - millionaire.controller.MainMenuController, 62
- showAnswerResult
  - millionaire.view.GameView, 43
- showAudienceResults
  - millionaire.view.LifelineView, 57
- showDialog
  - millionaire.controller.AddQuestionController, 14
- showError
  - millionaire.view.AddQuestionView, 23
  - millionaire.view.PlayerView, 85
  - millionaire.view.RulesView, 104
- showGameEndDialog
  - millionaire.view.GameView, 43
- showMenu
  - millionaire.controller.MainMenuController, 62
- showPlayerCreationDialog
  - millionaire.controller.PlayerController, 72
- showRules
  - millionaire.controller.MainMenuController, 63
  - millionaire.controller.RulesController, 99
- showScoreboard
  - millionaire.controller.MainMenuController, 63
  - millionaire.controller.ScoreboardController, 107
- showSuccess
  - millionaire.view.AddQuestionView, 23
- showWindow
  - millionaire.view.MainMenuView, 69
- src/main/java/millionaire/controller/AddQuestionController.java, 123
- src/main/java/millionaire/controller/GameController.java, 124
- src/main/java/millionaire/controller/LifelineController.java, 126
- src/main/java/millionaire/controller/MainMenuController.java, 127, 128
- src/main/java/millionaire/controller/PlayerController.java, 128, 129

src/main/java/millionaire/controller/RulesController.java, 129  
 src/main/java/millionaire/controller/ScoreboardController.java, 130  
 src/main/java/millionaire/controller/TimerController.java, 131  
 src/main/java/millionaire/Main.java, 132  
 src/main/java/millionaire/model/AddQuestionModel.java, 132, 133  
 src/main/java/millionaire/model/GameModel.java, 133, 134  
 src/main/java/millionaire/model/GameState.java, 135  
 src/main/java/millionaire/model/LifelineModel.java, 135, 136  
 src/main/java/millionaire/model/MainMenuModel.java, 137  
 src/main/java/millionaire/model/PlayerModel.java, 137, 138  
 src/main/java/millionaire/model/PrizeLevel.java, 139  
 src/main/java/millionaire/model/Question.java, 139, 140  
 src/main/java/millionaire/model/QuestionRepository.java, 141, 142  
 src/main/java/millionaire/model/RulesModel.java, 143  
 src/main/java/millionaire/model/ScoreboardModel.java, 144  
 src/main/java/millionaire/model/TimerModel.java, 146  
 src/main/java/millionaire/view/AddQuestionView.java, 146, 147  
 src/main/java/millionaire/view/GridView.java, 149  
 src/main/java/millionaire/view/LifelineView.java, 153  
 src/main/java/millionaire/view/MainMenuView.java, 155  
 src/main/java/millionaire/view/PlayerView.java, 157  
 src/main/java/millionaire/view/RulesView.java, 158  
 src/main/java/millionaire/view/ScoreboardView.java, 160  
 src/main/java/millionaire/view/TimerView.java, 162  
 start  
     millionaire.model.TimerModel, 119  
 startNewGame  
     millionaire.controller.MainMenuController, 64  
 startTimer  
     millionaire.controller.TimerController, 116  
 stop  
     millionaire.model.TimerModel, 120  
 stopTimer  
     millionaire.controller.TimerController, 116  
     millionaire.view.GridView, 44  
  
 TimerController  
     millionaire.controller.TimerController, 115  
 TimerModel  
     millionaire.model.TimerModel, 119  
 TimerView  
     millionaire.view.TimerView, 122  
  
 updateAnswerButtons  
     millionaire.view.LifelineView, 57  
 updateButtonStates  
     millionaire.view.LifelineView, 58  
 updateDisplay  
     millionaire.view.TimerView, 122  
     updateElapsedTime  
         millionaire.model.TimerModel, 120  
     updateFinalPlaytime  
         millionaire.view.GridView, 44  
     updateGameTime  
         millionaire.controller.GameController, 29  
     updatePlayTime  
         millionaire.model.GameModel, 36  
     updatePlaytime  
         millionaire.model.PlayerModel, 81  
     updatePrizeLadder  
         millionaire.view.GridView, 45  
     updateQuestion  
         millionaire.view.GridView, 45  
     updateScores  
         millionaire.view.ScoreboardView, 113  
 useAudienceHelp  
     millionaire.controller.LifelineController, 49  
 useFiftyFifty  
     millionaire.controller.LifelineController, 50  
  
 validateInputs  
     millionaire.model.AddQuestionModel, 18  
  
 WON  
     millionaire.model.GameState, 38